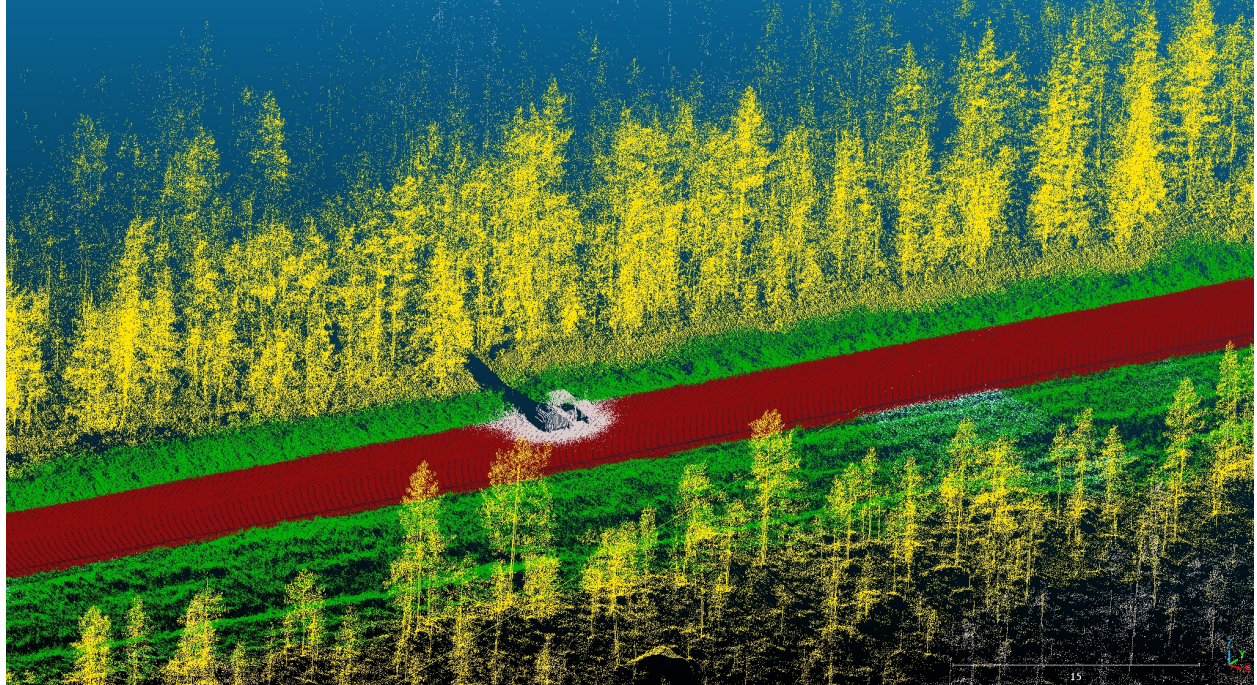




CHALMERS
UNIVERSITY OF TECHNOLOGY



Automatic Classification of LiDAR Point Cloud Data

Master's Thesis in Engineering Mathematics and Computational Science

SHUBHAJIT SAHOO
IVAN WELY

Department of Mathematical Sciences

CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden 2026
www.chalmers.se

MASTER'S THESIS 2026

Automatic Classification of LiDAR Point Cloud Data

SHUBHAJIT SAHOO
IVAN WELY



Department of Mathematical Sciences
CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden 2026

Automatic Classification of LiDAR Point Cloud Data

SHUBHAJIT SAHOO, IVAN WELY

© SHUBHAJIT SAHOO, IVAN WELY, 2026.

Industry Supervisor:
Morgan Larsson, Atritec AB

Academic Supervisor & Examiner:
Aila Särkkä, Chalmers University of Technology

Master's Thesis 2026
Department of Mathematical Sciences
Chalmers University of Technology
SE-412 96 Gothenburg
Sweden
Telephone +46 31 772 1000

Typeset in L^AT_EX
Gothenburg, Sweden 2026

Abstract

Road surveys use laser scanners to create 3D maps of the road and its surroundings, known as point clouds. This research addresses the automatic detection of moving vehicles within these point clouds. Moving vehicles distort the survey while it is being scanned, so Atritec AB, the company conducting these surveys, faced the tedious task of manually removing them from the resulting point clouds. This consequently became a problem of labeling every point in the cloud by category, tackled using machine learning methods that learn this labeling from examples. An end-to-end pipeline was developed around a model called RandLA-Net to carry out this point-by-point labeling, trained on a dataset created by manually annotating Atritec's point cloud recordings. Moving vehicles were far rarer in the dataset than the background classes, an imbalance that required a training approach able to counterbalance it. This was disproportionately penalizing the model on certain classes to make up for their infrequency. Experiments were also conducted with other datasets in an attempt to supplement the native Atritec AB dataset. However, due to fundamental differences between the datasets, this was not to any avail. Experimenting with different model designs and iteratively adjusting their settings led to a pipeline that reached a score of 0.74 for the moving vehicle class on parts of the dataset the model had never seen before. On this scale, a score of 0 means none of the moving vehicles were correctly identified, while a score of 1 means every one of them was.

Keywords: Mobile Laser Scanning, LiDAR, Point Cloud Semantic Segmentation, Machine Learning, RandLA-Net, Dynamic Object Detection, Class Imbalance, Focal Loss, Transfer Learning, Data Classification

Acknowledgements

We would like to express our sincere gratitude to Aila Särkkä, our supervisor and examiner at Chalmers University of Technology, for her valuable feedback throughout our meetings, her guidance in shaping the evaluation design, and her encouragement to document failures with the same rigor as successes, a principle that we hope makes this thesis stronger for its honesty. We would also like to thank Morgan Larsson, our supervisor at Atritec AB, for his operational perspective on production survey workflows and for the practical grounding that kept this research connected to real industrial deliverables. We are grateful for access to Atritec’s proprietary mobile laser scanning data and domain expertise, and for the academic support and resources provided by Chalmers University of Technology, all of which were instrumental in completing this thesis. We further acknowledge the computing resources provided by the Chalmers Centre for Computational Science and Engineering, C3SE, on the Vera high performance computing cluster, without which the experiments described in this thesis, comprising well over one thousand GPU hours, would not have been possible.

Shubhajit Sahoo & Ivan Wely, Gothenburg, October 2026

Declaration of AI-Assisted Technologies

During the preparation of this work, AI tools were used to improve grammar, vocabulary, and readability of the manuscript. After using these tools, the content was carefully reviewed and verified as needed, and full responsibility for the content of the published thesis report is taken by us.

Shubhajit Sahoo & Ivan Wely, Gothenburg, October 2026

List of Acronyms

Below is the list of acronyms used throughout this thesis, in alphabetical order.

Adam	Adaptive Moment Estimation, a gradient based optimiser
ASPRS	American Society for Photogrammetry and Remote Sensing
C3SE	Chalmers Centre for Computational Science and Engineering
CNN	Convolutional Neural Network
FN	False Negative
FP	False Positive
GDPR	General Data Protection Regulation
GNSS	Global Navigation Satellite System
GPU	Graphics Processing Unit
HPC	High Performance Computing
IMU	Inertial Measurement Unit
IoU / mIoU	(mean) Intersection over Union
KNN	K Nearest Neighbours
KPConv	Kernel Point Convolution
LAS	Public point cloud file format maintained by ASPRS
LiDAR	Light Detection and Ranging
LiDAR Scanner	Proprietary scanner used by Atritec AB
MLP	Multi Layer Perceptron
MLS	Mobile Laser Scanning
NPY	NumPy array file format used for the exported training data
RandLA-Net	Point cloud segmentation network that combines random point sampling with local feature aggregation
RGB	Red, Green and Blue colour channels
SemanticKITTI	Public labelled LiDAR dataset used in the transfer study
SLAM	Simultaneous Localisation and Mapping
TN	True Negative
TP	True Positive

Nomenclature

Below is the nomenclature of the recurring symbols used throughout this thesis.

Recurring Symbols

p_i, N	The i th point of a point cloud and the number of points in it
$\mathbf{f} \in \mathbb{R}^{10}$	Per point input feature vector, Equation 3.1
d	Number of features per point, 10 in this work
r, r_{norm}	Scanner range and its per tile normalisation
$n_{\text{ret}}, N_{\text{ret}}$	Return number, number of returns
R, G, B	Normalised red, green, and blue colour channels
C	Number of classes, 8 in this work
w_c, α_c	Static loss weight of class c , w_{18} for moving vehicles, written α_c in the focal loss equation
γ	Focal loss focusing parameter, 2.0 throughout
$\mathcal{L}, \mathcal{L}_{CE}, \mathcal{L}_{FL}, \boldsymbol{\theta}$	Loss, cross entropy loss, focal loss and network weights
p_{18}, p_{road}	Centred oversampling probabilities
K	Neighbourhood size, 32
κ	Decimation factor per encoder stage, 4
η, η_0, η_e	Learning rate, its base value and its value at epoch e
δ	Learning rate decay factor, 0.98
λ	Motion distortion elongation ratio, Equation 2.5
TP_c, TN_c	True positives and true negatives of class c
FP_c, FN_c	False positives and false negatives of class c
IoU_c	IoU of class c , IoU_{18} the primary metric

Contents

List of Acronyms	xi
Nomenclature	xiii
List of Figures	xix
List of Tables	xxi
1 Introduction	1
1.1 Background and Motivation	1
1.1.1 Challenges	2
1.2 Purpose and Scope	3
1.3 Research Questions	4
1.4 Contributions	4
1.5 Relation to the Existing Literature	4
1.6 Ethical Aspects	6
1.7 Data and Code Availability	6
2 Theory	7
2.1 LiDAR Principles and Point Cloud Data	7
2.1.1 Scanner Properties	7
2.1.2 Dynamic Object Distortion	8
2.1.3 Point Cloud Data Structure	9
2.2 Three-Dimensional Semantic Segmentation and Machine Learning Principles	10
2.2.1 Random Point Sampling	12
2.2.2 Local Feature Aggregation	12
2.2.3 Focal Loss	12
2.2.4 Class Imbalance	14
2.2.5 Training and Optimization	14
2.2.6 Transfer Learning	15
2.2.7 Validation and Generalization	15
2.2.8 Evaluation Metrics	16
3 Methodology	19
3.1 Terminology and Experiment Overview	19
3.2 Dataset	20

3.2.1	Data Acquisition and Class Scheme	20
3.2.2	Annotation Protocol	23
3.2.3	Tile Sets per Run	24
3.3	Pipeline Overview	24
3.4	Input Features	26
3.4.1	Ten Feature Vector	26
3.4.2	Feature Count by Iteration	26
3.5	Network Architecture	27
3.5.1	V1 and V2	27
3.6	Training and Evaluation	28
3.6.1	Loss and Class Weights	28
3.6.2	Window Sampling and Oversampling	28
3.6.3	Optimization and Checkpointing	29
3.6.4	Validation and Inference	29
3.7	Iterations and Experiments	30
3.7.1	Iterations 1 to 3	30
3.7.2	Iteration 4	30
3.7.3	Transfer Study	31
3.7.4	RGB Integration Run	31
3.7.5	Iteration 5	31
4	Experimental Results	33
4.1	Iterations 1 to 3	33
4.2	Between Iterations 3 and 4	33
4.3	Iteration 4	34
4.4	Transfer Study	35
4.5	RGB Integration Run	36
4.5.1	Training Instability	36
4.5.2	Learning Rate Overshoot	36
4.5.3	Checkpoint B on Reserved Test Tile 51	37
4.6	Iteration 5, Final Results	38
4.6.1	Validation Trajectories per Profile	38
4.6.2	Best Epoch per Profile	38
4.6.3	Reserved Test Tile Protocol	38
4.6.4	Confusion Matrix and IoU	40
4.6.5	Row Normalized Confusion Heatmap	41
4.6.6	Column Normalized Confusion Heatmap	41
4.6.7	Recall and Precision	42
4.6.8	Reserved Test Tile Render	42
5	Discussion	45
5.1	Iterations Together	45
5.1.1	Iteration Progression	45
5.1.2	Data Volume	45
5.1.3	Checkpoint B to Iteration 5	46
5.2	Q1, Moving Vehicle Detection	47
5.3	Q2, Class Imbalance	48

5.3.1	Weighting Iterations	48
5.3.2	RGB Integration Run Instability	48
5.3.3	Iteration 5 Stability	49
5.4	Q3, Transfer Learning	49
5.4.1	Experiment A, Head Reset	49
5.4.2	Experiment C, Overfitting	50
5.4.3	Domain Gap and Augmentation	50
5.5	Road, Ground and Color	50
5.5.1	RGB Hypothesis	51
5.5.2	Iteration 5, Road and Ground	51
5.6	Relation to Published Results	51
5.7	Limitations	52
6	Conclusion and Future Work	53
6.1	Conclusions	53
6.2	Future Work	54
	References	55
A	Hyperparameter Record	I
B	Core Algorithms	III
C	Computational Environment	VII
D	Iteration 5 Training Logs	IX
D.1	Colored Profile	IX
D.2	Mixed Profile	XII
D.3	Uncolored Profile	XV

List of Figures

1.1	Class composition of the full dataset, with each bar showing that class's percentage share on a logarithmic scale with point count at right.	3
2.1	Each point is assigned a semantic class from the classification scheme, here shown for ground, high vegetation and a moving vehicle.	7
2.2	The RIEGL VUX-1HA-22 LiDAR scanner used to collect the point cloud dataset for this thesis. Source: RIEGL LMS GmbH.	8
2.3	Intensity is the strength of the returned pulse and varies from point to point, shown here as a grayscale value from low (white) to high (black).	10
2.4	A single laser pulse traveling through vegetation can produce multiple returns, one for each surface it reflects from on the way to the ground. The number of returns the scanner can resolve per pulse depends on the pulse repetition rate, reported in Table 2.1.	11
2.5	RandLA-Net encoder decoder architecture. Each of the five encoder stages reduces the point count while enriching the features carried by each remaining point, and each of the five decoder stages restores points through nearest neighbor upsampling, combined with the matching encoder stage of the same resolution through a skip connection.	11
2.6	Local feature aggregation unit (a) and encoder stage (b).	13
2.7	Confusion matrix convention for three generic classes A, B and C. Row i and column j hold n_{ij} , the count of points whose true class is i and predicted class is j	16
2.8	Row normalized confusion heatmap for the three generic classes of Figure 2.7.	17
2.9	Column normalized confusion heatmap for the three generic classes of Figure 2.7.	18
3.1	Runs in chronological order.	21
3.2	Ground truth example of each visible class in one tile.	22
3.3	A manually annotated training tile rendered by classification in Cloud-Compare. Road (IoU_{11}) is annotated as patches (yellow), the moving vehicle (IoU_{18}) distortion is annotated in full (red), vegetation is annotated on both sides (green), and the remaining points carry no label (blue).	23

3.4	Tile split of the three Iteration 5 profiles.	25
3.5	End-to-end pipeline, with gray blocks for the input and output data, and orange for training and evaluation.	25
3.6	V2 architecture overview.	28
3.7	Training, validation and inference loop.	29
4.1	Validation IoU_{18} per epoch for Experiment A (head reset) and Ex- periment C (whole network fine tuning). Experiment C is shown only through epoch 32, when training was terminated. The dots mark the reference epoch of Table 4.3.	36
4.2	RGB integration run validation trajectory. The dot marks the best validation IoU_{18} epoch, Checkpoint B.	37
4.3	Iteration 5 validation mIoU by epoch. The dots mark the best val- idation mIoU epoch of each profile. Since Uncolored averages four classes and the other two profiles seven, their levels are not directly comparable.	38
4.4	Iteration 5 validation IoU_{18} by epoch. The dots mark the best vali- dation mIoU epoch of each profile.	39
4.5	Iteration 5 focal loss of the Colored profile. The vertical axis is loga- rithmic.	39
4.6	Row normalized confusion heatmap on tile 51.	41
4.7	Column normalized confusion heatmap on tile 51.	42
4.8	A training tile annotated under the protocol of Figure 3.3 (left), shown next to the full tile inference on the reserved test tile 51 (right) for comparison.	43
5.1	Validation IoU_{18} at the reference epoch against the number of training tiles.	46
5.2	Research questions mapped to runs and answers.	47
5.3	Design of the transfer study.	49

List of Tables

2.1	Maximum range and maximum returns per pulse at each pulse repetition rate (PRR) of the LiDAR scanner used in this project. Source: RIEGL LMS GmbH.	8
3.1	Overview of the training runs.	20
3.2	Class codes used by the network and in the LAS tiles.	22
3.3	Point counts per class and total point count per tile across all twenty-six tiles, split into training, validation and test sets.	24
3.4	Training, validation and test tiles of every run.	25
3.5	Comparison of V1 and V2 architecture configurations.	27
3.6	Dataset profiles used for Iteration 5 training.	32
4.1	Every training run of this thesis in the order presented.	34
4.2	Iterations 1 to 3 outcomes.	34
4.3	Phase 2 against the from scratch baseline.	35
4.4	Mean color per class on tile 52, in 16 bit RGB channel values, computed over every annotated point.	37
4.5	Iteration 5 best validation results per class.	39
4.6	Confusion matrix of the Colored profile on tile 51, in point counts, with rows as the true class and columns as the predicted class.	40
4.7	Per class IoU on validation and on the reserved test tile 51.	40
5.1	Per class IoU against the published RandLA-Net result.	52
A.1	Hyperparameters of the principal runs.	I
C.1	HPC and software environment.	VII
C.2	Packages installed inside the Apptainer container.	VII
D.1	Iteration 5 Colored profile, per epoch training and validation record: training loss (TrL), validation loss (VaL), mean IoU and per-class IoU, with BEST marking each new best checkpoint and the reported checkpoint in bold.	IX
D.2	Iteration 5 Mixed profile, per epoch training and validation record: training loss (TrL), validation loss (VaL), mean IoU and per-class IoU, with BEST marking each new best checkpoint and the reported checkpoint in bold.	XII

D.3	Iteration 5 Uncolored profile, per epoch training and validation record: training loss (TrL), validation loss (VaL), mean IoU and per-class IoU (N/A where a class is absent), with BEST marking each new best checkpoint and the reported checkpoint in bold.	XV
-----	---	----

1

Introduction

1.1 Background and Motivation

The goal of this research is to assist Atritec AB with processing point cloud data by removing points pertaining to dynamic objects for their clients, such as Trafikverket. Moving vehicles distort the geometry of a road survey while it is being scanned, so removing their points leaves a clean representation of the road surface. In addition to removing dynamic object points, remaining points are to be classified into a set of classes. This boils down to a classification problem whereby classifying all the points leads to trivial removal of undesired points.

LiDAR is a remote sensing technology that measures the distance to a target by timing the return of a laser pulse, which is emitted from the scanner and reflected by the target [1]. Repeated measurements in different directions can be used to build up a three-dimensional picture of the environment. Mobile Laser Scanning (MLS) applies this principle from a moving platform, such as a road vehicle, and produces dense three-dimensional point clouds, which are sets of discrete measurement points in three-dimensional space. Point clouds may have additional attributes associated with each point, such as the returned signal strength. Surfaces are represented directly by point density and arrangement. MLS is being used in surveying, inspection, and modeling applications, including those involving infrastructure. The LiDAR scanner used in this project consists of two rotating RIEGL VUX-1HA-22 units and generates point clouds of high density and geometric accuracy. Atritec AB is the industrial partner in this project and operates the scanner for road corridor mapping across Sweden, and the data is given in LAS format (Section 3.2.1).

During a road scan, an object is scanned multiple times from different angles, resulting in a cluster of points that form a geometrically coherent representation. A moving vehicle will be changing position between successive scan lines, and its points will distort across positions which it did not simultaneously occupy. This produces a distortion or compression artifact in the direction of relative motion, as seen in Section 2.1.2. The artifacts corrupt the geometric integrity of the scene and can inflate clearance measurements, contaminate statistics, and require removal before road surveying can be conducted.

Manual identification and removal of these artifacts is feasible for small datasets, but is impractical at survey production scale. The current manual workflow for removal involves loading a tile, a rectangular section of the surveyed point cloud, into a point cloud visualization tool, visually inspecting the point cloud, and manually identifying and deleting any vehicle distortions. This process can take several hours

for one tile, and some artifacts may be missed. Given the growing survey volume undertaken by Atritec AB, manual dynamic object removal is becoming a significant production bottleneck, with cost scaling linearly with survey size and with no analytical value to the data cleansing process. A pipeline that can reliably detect and remove moving vehicle points would have significant practical value, and developing its detection stage is the problem which this thesis seeks to address.

1.1.1 Challenges

Autonomous vehicles typically rely on Simultaneous Localization and Mapping (SLAM) [2], which lets them know their location in the environment as they build up a point cloud map. Since these algorithms work at about ten scans per second [3], anything that has shifted since a fraction of a second earlier can be flagged as moving. That is not the case with MLS survey data, where successive moments are not compared. The survey vehicle uses the Global Navigation Satellite System (GNSS) and inertial sensors only to place the final point cloud correctly on the map. Each stretch of road is typically scanned in a single pass, so a moving vehicle can only be recognized from the distortion it leaves behind rather than by observing it move. Autonomous vehicles must predict where a moving vehicle will go next within milliseconds and respond accordingly to proceed safely. Here, no such prediction is made. Instead, the trace a moving vehicle has already left in the final point cloud has to be recognized, so that it can be removed. This difference in purpose, together with the single-pass nature of the MLS data, explains why frame-comparison methods cannot be applied directly, and motivates geometry-based moving vehicle detection, meaning detection based on the shape of the distortion.

The five challenges highlighted below are the major constraints of this task.

Scale. Although the format of the data is highly regular, a single tile contains tens of millions of points, and many more points are present across multiple tiles. This places severe constraints on per-point algorithms, as they must be kept computationally tractable.

Extreme Class Imbalance. In this dataset, the moving points are less than one percent of all points, as shown in Figure 1.1. The twenty-six tiles hold about 2.20 billion points, of which about 590 million are annotated. Among these annotated points, the moving vehicle points are about 2.8 percent, and therefore moving vehicle points contribute only a few of the points.

Geometric Irregularity. A main distinction between autonomous driving perception and geometry-based moving vehicle detection lies in the nature of distortions, the biggest clue to the presence of a moving vehicle. The shape of a distortion depends directly on the speed and trajectory of its source relative to the survey vehicle, so no single, fixed shape can be assumed. A high-speed oncoming car leaves a drastically different artifact from a slow-moving truck in the adjacent lane.

Scarcity of Labeled Training Data. Deep segmentation networks are usually trained on thousands of labeled scenes [4], [5]. However, labeled data is scarce because preparing it is costly and far from trivial. Classifying each point is easily achieved in a three-dimensional editor, but this takes many hours of annotation and is only suitable for relatively small data volumes. Labels assigned in this way to

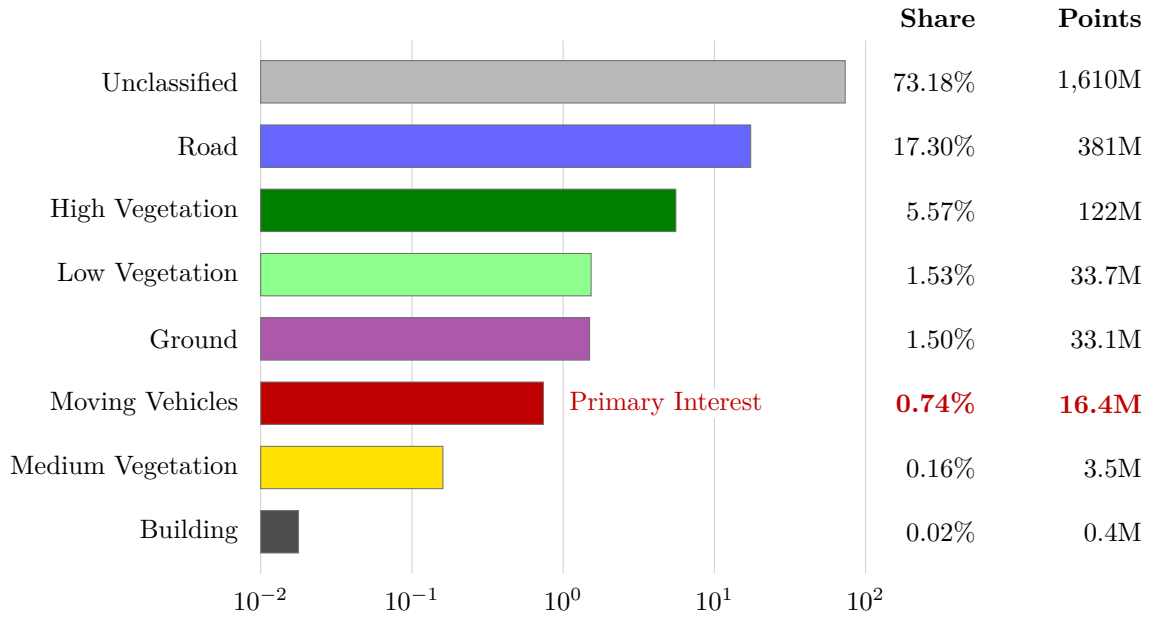


Figure 1.1: Class composition of the full dataset, with each bar showing that class’s percentage share on a logarithmic scale with point count at right.

points over a small area may be used to train a segmentation network, which can then process much larger volumes of data.

Spectral Ambiguity. The LiDAR scanner of this project operates at a single wavelength of about 1550 nm and outputs a measure of the strength of the reflected laser signal, called intensity, with no color channel of its own. Road surfaces and nearby ground can reflect in a similar manner, so intensity alone may be of little help in distinguishing the road surface from the ground immediately adjacent to it. When both road surface and ground are essentially flat and low, there is little or no distinction between them, and only geometry can be used as a discriminant. Due to this, color channels (RGB) were integrated as features in Iteration 5, the fifth in the thesis’ sequence of training iterations, reported in Chapter 4.

1.2 Purpose and Scope

The purpose of this thesis is to develop an automated pipeline for detecting dynamic objects in MLS point clouds in order to remove them. The pipeline uses a semantic segmentation model, RandLA-Net [6], and the network is configured to classify each point into eight classes (Table 3.2). The task of removing the dynamic objects then simply boils down to running inference and deleting all points classified as Class 18, that is, the label for moving vehicles.

The data Atritec AB has collected consists of MLS tiles along Swedish roads using the aforementioned LiDAR scanner configuration, and this was done with an accompanying inertial measurement unit (IMU) for scan vehicle trajectory data. Only the point cloud of each tile is used. The dataset and the LAS format are described in full later in Section 3.2.

1.3 Research Questions

This thesis will attempt to answer the following research questions.

- Q1.** How reliable are moving vehicle detections when training on MLS point clouds acquired with a proprietary LiDAR scanner?
- Q2.** To what extent does the imbalance between the moving vehicle class and the background classes affect training stability and detection accuracy? How can it be handled without leading to over prediction of the moving vehicle class?
- Q3.** Would pretraining on SemanticKITTI, a publicly available semantic segmentation dataset, improve performance on the proprietary LiDAR scanner dataset? If not, what factors might account for this outcome?

1.4 Contributions

The main contributions are the following.

1. An end-to-end pipeline for dynamic object detection on LiDAR scanner data, from LAS files through a purpose-designed feature representation, containerized high performance computing (HPC) training, and full tile inference writing classified LAS output compatible with existing survey tooling.
2. A systematic experimental study across multiple iterations on a proprietary industrial dataset, documenting the final configuration and the failures encountered along the way. These included the model collapsing when class weights were set too high, interference from oversampling, defects in the earlier code, and several problems with transfer learning and training stability. Each was analyzed and, where its cause was found, turned into a concrete design lesson.
3. A characterization of the domain gap between the Velodyne sensor of SemanticKITTI and the LiDAR scanner of this project, identifying differences in scan pattern, point density, and how return information is recorded as the likely reasons a SemanticKITTI-pretrained model did not transfer well to LiDAR scanner data in the transfer study, which was not repeated on the corrected code.
4. A final training configuration, Iteration 5, trained from scratch on corrected training and evaluation code as described in Section 2.2.7, reached a moving vehicle IoU (Intersection over Union) of 0.78 with color on the validation tiles as reported in Section 4.6, and was evaluated once on a single reserved test tile, giving a moving vehicle IoU of 0.74 as reported in Section 4.6.4. The same evaluation of the best earlier checkpoint gave a moving vehicle IoU close to zero, and the comparison on that tile is the headline result of this thesis.

1.5 Relation to the Existing Literature

Road environments have been studied extensively using MLS [7], where point cloud methods have been explored for detecting moving vehicles [8], [9]. Feature-based methods have been used by Lehtomäki et al. [7], who also addressed an uneven distribution of points arising from the point density of the helical scan pattern. Guan

et al. [10] extracted road markings from MLS data automatically. Semantic segmentation of point clouds has recently been the subject of extensive research, where initial approaches reduce point cloud data by converting it into three-dimensional voxels or images and using standard convolutional neural networks (CNNs) [4], [11]. However, benefits of working on unordered point sets surfaced with the advent of PointNet [12], which is also known for good training convergence rates. PointNet++ [13] outperformed PointNet by learning hierarchical local features. Other methods have used kernel point convolution (KPConv) [14], or a random point sampling plus a local feature aggregation approach taken by RandLA-Net [6], which will serve as this thesis' architecture. The public benchmark SemanticKITTI [5] is used for comparison in other researchers' published work, but in this thesis it is instead used as the source of pretraining data in Chapter 3. For detection of moving vehicles, the continuous scanning distortion described by Dewan et al. [8] and Mertz et al. [9] is used. It results in an elongated distortion whose shape encodes relative velocity, and this distortion defines the moving vehicle class, Class 18, in this thesis' context. The closest published work on vehicle detection from georeferenced MLS data was presented by Xiao et al. [15], while Schauer and Nüchter [16] detected and removed dynamic objects by traversing a voxel occupancy grid built from several observations of the same scene, which would have been possible in this thesis' context with repeated passes, but such data was unavailable. To address class imbalance, focal loss, which down-weights easy examples so that training focuses on hard ones, was used throughout this thesis [17]. Surveys of deep learning on point clouds [4], [11] give context on the datasets and the annotation effort involved.

Transfer learning between domains [18] has positive results whenever a model trained on a source domain can be used to help a target task. When applied to point cloud data, it often comes down to a mismatch in the representation between source and target [19]. Qin et al. [19] propose an adaptation network to match different aspects of the representation. In this thesis' two-stage transfer study, Phase 1 and Phase 2, described in Section 3.7.3, no explicit domain adaptation module is added, but rather a pretrained network is fine-tuned directly. The negative result of this fine tuning described in Section 5.4 is consistent with the general observation in the domain adaptation literature that a model pretrained on one sensor transfers poorly to another when the two differ substantially in geometry and point density [19]. In two ways this thesis differs from the closest published work. In contrast to Xiao et al. [15], here deep neural network representations are employed instead of engineered geometric features, and results are reported on a reserved test tile that was never trained on. The results of Hu et al. [6] are measured on SemanticKITTI, a large benchmark with Velodyne LiDAR scans, while here SemanticKITTI is only used as pretraining data, and the target is the smaller proprietary dataset. The two are therefore not directly comparable, and Section 5.6 places the final checkpoint beside the published per class values.

1.6 Ethical Aspects

Three safeguards keep the use of third-party data collected during MLS acquisition within the ethical boundaries expected for data collected on public roads, where vehicles with number plates may be legible at the point densities of this data. First, all tile data is handled in line with Atritec’s existing data processing agreement and stored only on the access-controlled storage system provided by the Chalmers Centre for Computational Science and Engineering (C3SE), so no result data is stored outside it. Second, the intended removal of the detected moving vehicle points leaves less personal information in the output than in the input, consistent with the data minimization principle of the General Data Protection Regulation (GDPR) [20]. Third, Class 18 is a purely geometric category, and nothing in the pipeline identifies, tracks or re-identifies any road user. SemanticKITTI is used under its academic license and none of its data is redistributed.

1.7 Data and Code Availability

The point cloud tiles used in this thesis were captured with the LiDAR scanner and supplied by Atritec AB under a data processing agreement. The tiles, including their manual annotations, and the training, export and inference code developed for this thesis are proprietary and will not be shared. Only the hyperparameters of the principal runs are shared, in Appendix A, together with the pseudocode of the export, training and inference procedures in Appendix B, the computational environment in Appendix C and the per epoch training records of the final iteration in Appendix D. The public SemanticKITTI dataset used for pretraining in Section 3.7.3 is available under its own academic license from its maintainers.

2

Theory

This chapter elaborates the physics to explain why distortions exist and which features are physically meaningful. It gives deep learning theory to explain why we have chosen the architecture in Section 3.5.1. It covers theories to explain why we choose the training strategy. Finally, it proposes metrics to explain why we evaluate in the particular way.

2.1 LiDAR Principles and Point Cloud Data

LiDAR measures the distance by producing a laser pulse and timing the return of the reflected signal. For a pulse produced at time t_0 and returning at time t_1 , the slant range r is

$$r = \frac{c \cdot (t_1 - t_0)}{2} \quad (2.1)$$

where c is the speed of light. The LiDAR scanner works in full waveform mode. It records the entire reflected signal over time. This helps the scanner detect more than one object or surface along the laser's path and detect objects that are partly hidden by vegetation, such as trees or bushes. Figure 2.1 shows a point cloud with points assigned to the ground, vegetation and moving vehicle classes.

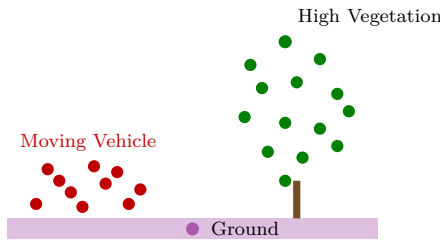


Figure 2.1: Each point is assigned a semantic class from the classification scheme, here shown for ground, high vegetation and a moving vehicle.

2.1.1 Scanner Properties

The MLS system used here has two LiDAR scanner units installed on the survey vehicle at fixed angles (e.g. 45 degrees and vertical). As the vehicle moves along the survey line, the units independently rotate around their respective axes. Figure 2.2 shows the scanner unit used.



Figure 2.2: The RIEGL VUX-1HA-22 LiDAR scanner used to collect the point cloud dataset for this thesis. Source: RIEGL LMS GmbH.

For a single unit rotating at angular velocity ω with the platform moving at velocity v_p , each complete rotation produces a helical scan pattern. The scan line spacing along the trajectory direction is

$$d_{line} = \frac{v_p}{\omega/(2\pi)} \quad (2.2)$$

The point density at range r from a single unit is approximately

$$\rho(r) \approx \frac{\omega \cdot f_{PRF}}{2\pi \cdot v_p \cdot r} \quad (2.3)$$

where f_{PRF} is the pulse repetition frequency. With a maximum scan speed of 250 rotations per second and a pulse repetition frequency of up to 1.8 million pulses per second per unit, and with two units operating simultaneously at different mounting angles, the combined point density at a given location is the sum of the contributions from the units. The inverse relationship between density and range in the equation above is the characteristic of MLS data that distinguishes it from fixed scanner terrestrial surveys. Table 2.1 lists the manufacturer's maximum range and maximum returns per pulse at each pulse repetition rate, since a higher rate trades range and multi-target resolution for a denser scan.

Table 2.1: Maximum range and maximum returns per pulse at each pulse repetition rate (PRR) of the LiDAR scanner used in this project. Source: RIEGL LMS GmbH.

PRR (kHz)	300	500	1000	1250	1500	1800
Max Range, 10% Reflectivity (m)	170	130	85	85	85	85
Max Range, 80% Reflectivity (m)	475	370	235	235	235	235
Max Returns per Pulse	15	15	9	7	5	4

2.1.2 Dynamic Object Distortion

For a point on a stationary object at position $\mathbf{p}_0 = (x_0, y_0, z_0)$, all scanner measurements of that point return the same position in the georeferenced frame. Here $\mathbf{p}_0$

is the true, fixed position of the point in three-dimensional space, and x_0, y_0, z_0 are its coordinates.

As discussed by Dewan et al. [8] and Mertz et al. [9], for a moving vehicle, the apparent position of a point measured at time t_i is

$$\mathbf{p}_i^{apparent} = \mathbf{p}_{0,i} + \mathbf{v}_o t_i \quad (2.4)$$

where $\mathbf{p}_i^{apparent}$ is the position at which the point is recorded in the georeferenced point cloud, $\mathbf{p}_{0,i}$ is the position of that point at the start of the observation, $\mathbf{v}_o$ is the velocity of the moving vehicle relative to the ground, and t_i is the time at which that point was measured, relative to the start of the observation. Because the object has moved by the time each successive point is captured, while the platform's motion is accounted for during georeferencing, the recorded position differs from $\mathbf{p}_{0,i}$ as t_i increases.

The collection of apparent positions across the measurement interval can form a compressed cluster when the vehicle moves opposite to the direction in which its length is sampled. The length ratio λ is approximately

$$\lambda = \frac{L_{apparent}}{L_{true}} \approx \left| 1 + \frac{(\mathbf{v}_o \cdot \hat{\mathbf{s}}) T_{scan}}{L_{true}} \right| \quad (2.5)$$

where $L_{apparent}$ is the length of the object as it appears in the distorted point cloud, L_{true} is the physical length of the object, T_{scan} is the time over which the object's length was sampled, and $\hat{\mathbf{s}}$ points from the first to the last sampled part of the object. A value of λ close to 1 indicates little or no length distortion, while a value below 1 indicates compression. In this case, the compression of the vehicle's point cloud reflects its motion during the scan and can help identify it as a moving object.

2.1.3 Point Cloud Data Structure

A point cloud $\mathcal{P} = \{p_i\}_{i=1}^N$ is an unordered set of N points [12], where each point $p_i \in \mathbb{R}^d$ carries d features. In the final configuration of this thesis each point carries ten input features as described in Section 3.4.1

$$p_i = (x_i, y_i, z_i, \text{intensity}_i, r_i, \text{return_num}_i, \text{num_returns}_i, R_i, G_i, B_i) \quad (2.6)$$

where x_i, y_i and z_i are the spatial coordinates of the point, intensity_i is the strength of the returned laser signal, r_i is the range from the scanner to the point, return_num_i and num_returns_i record which return within its pulse the point is and how many returns that pulse produced, and R_i, G_i and B_i are the co-registered camera color values.

Besides its position, each point of an MLS survey carries attributes that the LAS format, a standardized binary format for storing point clouds maintained by ASPRS defines for every return [21]. The intensity is the strength of the returned pulse, which depends on the reflectivity of the surface at the laser wavelength, on the angle at which the pulse meets the surface and on the distance traveled [1]. Intensity is therefore not a property of the material alone, and two different surfaces can return

a similar intensity. This reflectance overlap can limit how well intensity separates classes such as road and ground, discussed in Section 5.5. Figure 2.3 illustrates this per point variation.

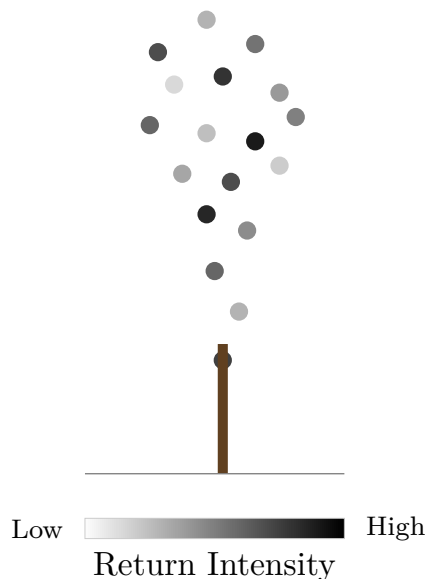


Figure 2.3: Intensity is the strength of the returned pulse and varies from point to point, shown here as a grayscale value from low (white) to high (black).

A laser pulse has a footprint of finite size and can be reflected by several surfaces along its path, for example by the leaves of a tree and then by the ground beneath. Each reflection is recorded as a separate return, the return number gives the position of a return within its pulse, and the number of returns gives how many returns the pulse produced [21]. Pulses that meet vegetation therefore often produce several returns, while pulses that meet a road usually produce one. A scanner that records a single return per pulse gives constant values for both attributes, which is one of the differences behind the domain gap, the mismatch between the sensor used for training and the one used at deployment, discussed in Section 5.4.3. Figure 2.4 illustrates this for a single pulse passing through vegetation.

Color is added to the LAS tiles, the rectangular sections a point cloud is divided into for processing, in a post processing step, in which co-registered RGB values are assigned to each point [21]. It describes the reflection of visible light rather than of the laser wavelength, so it can separate surfaces that return a similar intensity, at the price of depending on the illumination and on the accuracy of the co-registration.

2.2 Three-Dimensional Semantic Segmentation and Machine Learning Principles

RandLA-Net [6] is an encoder decoder network for semantic segmentation of large outdoor point clouds. The encoder repeatedly reduces the number of points while enriching the features each remaining point carries, and the decoder expands the compressed representation to the original number of points so that every point re-

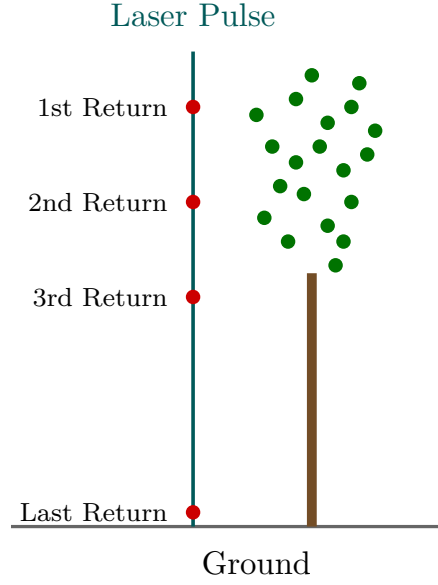


Figure 2.4: A single laser pulse traveling through vegetation can produce multiple returns, one for each surface it reflects from on the way to the ground. The number of returns the scanner can resolve per pulse depends on the pulse repetition rate, reported in Table 2.1.

ceives a class label, as shown in Figure 2.5. It was selected because random point sampling and local feature aggregation let it process tiles of tens of millions of points within practical time and memory. Figure 3.5 in Section 3.3 shows where the network sits in the pipeline of this work.

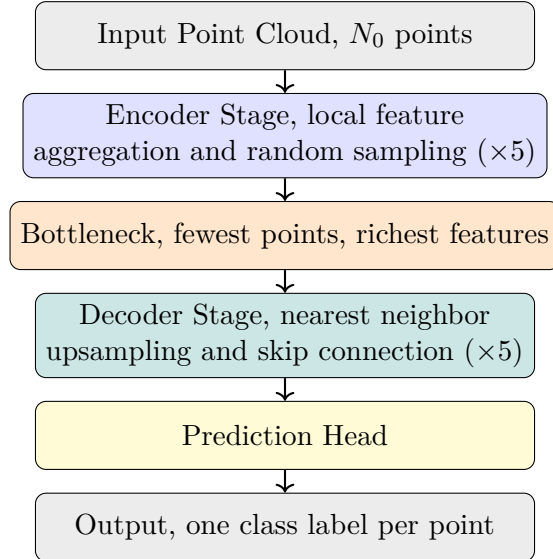


Figure 2.5: RandLA-Net encoder decoder architecture. Each of the five encoder stages reduces the point count while enriching the features carried by each remaining point, and each of the five decoder stages restores points through nearest neighbor upsampling, combined with the matching encoder stage of the same resolution through a skip connection.

2.2.1 Random Point Sampling

At each encoder stage the point cloud is randomly sub-sampled by a decimation factor κ , so that after l stages the number of points remaining is

$$N_l = \frac{N_0}{\kappa^l} \quad (2.7)$$

where N_0 is the number of input points, κ is the decimation factor, taking the value 16 or 32 depending on the network version, and l runs from 1 to the number of encoder stages L . Random sampling costs constant time per point, $O(1)$, whereas farthest point sampling, which iteratively selects the point farthest from those already chosen, costs $O(N^2)$, which makes the former practical at this scale [6].

2.2.2 Local Feature Aggregation

For each point p_i at a given encoder stage, its K nearest neighbors, the K points whose spatial coordinates are closest to p_i by Euclidean distance, are found by exact nearest neighbor search. For each neighbor p_j the relative position and the Euclidean distance are

$$\Delta p_{ij} = p_i^{xyz} - p_j^{xyz}, \quad d_{ij} = \|\Delta p_{ij}\| \quad (2.8)$$

where p_i^{xyz} and p_j^{xyz} are the spatial coordinates of the center point and the neighbor. These are concatenated with the two feature vectors and encoded by a shared multilayer perceptron (MLP), a small network of one or more fully connected layers with a nonlinear activation between them, applied identically to every neighbor,

$$\tilde{\mathbf{f}}_{ij} = \text{MLP}([\Delta p_{ij} \parallel d_{ij} \parallel \mathbf{f}_i \parallel \mathbf{f}_j]) \quad (2.9)$$

where $\mathbf{f}_i$ and $\mathbf{f}_j$ are the feature vectors of the center point and the neighbor, $\parallel$ denotes concatenation and $\tilde{\mathbf{f}}_{ij}$ is the encoded contribution of that neighbor. Attentive pooling [6] then weights each contribution by a learned score, using the softmax function so that the K raw scores of a point become positive weights that sum to one

$$s_{ij} = \text{softmax}_j(\text{MLP}(\tilde{\mathbf{f}}_{ij})), \quad \mathbf{F}_i = \sum_{j=1}^K s_{ij} \tilde{\mathbf{f}}_{ij} \quad (2.10)$$

where s_{ij} is the weight of neighbor j , positive and summing to one over the K neighbors, and $\mathbf{F}_i$ is the aggregated feature of p_i . Two such units are stacked per encoder stage with a residual shortcut [22] that adds the stage input to its output, which keeps gradients flowing through the deep network. Figure 2.6 summarizes one such unit, panel (a), and the encoder stage built from two of them, panel (b). The aggregated feature passes through a second shared MLP before it leaves the unit.

2.2.3 Focal Loss

Standard cross entropy loss for multi class segmentation is

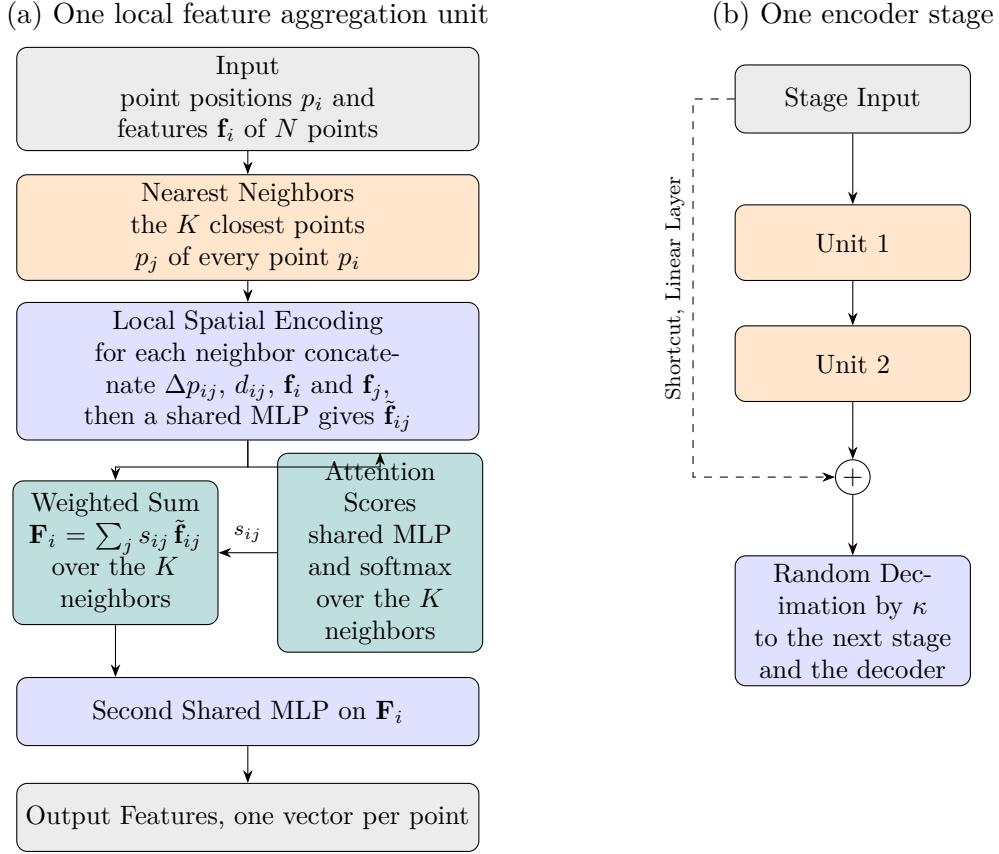


Figure 2.6: Local feature aggregation unit (a) and encoder stage (b).

$$\mathcal{L}_{CE} = - \sum_{c=1}^C y_c \log(\hat{p}_c) \quad (2.11)$$

where C is the number of classes, y_c is the one-hot label, equal to 1 for the true class and 0 otherwise, and $\hat{p}_c$ is the predicted probability of class c . On an imbalanced dataset this loss is dominated by the majority classes. Focal loss [17] adds a modulating factor that reduces the contribution of points the network already classifies confidently

$$\mathcal{L}_{FL} = - \sum_{c=1}^C \alpha_c (1 - \hat{p}_c)^\gamma y_c \log(\hat{p}_c) \quad (2.12)$$

where α_c is the class weight, a non-negative value set per class and ranging between 1 and 10, and $\gamma \geq 0$ is the focusing parameter that controls how strongly confident predictions are down weighted, with $\gamma = 0$ recovering weighted cross entropy. In this work $\gamma = 2$, the class weights α_c are the vector of Equation 3.2, and Class 0, unclassified, is excluded from the loss. The moving vehicle weight was set empirically, as described in Section 4.1.

2.2.4 Class Imbalance

Imbalance also acts on how training data is sampled. If training windows are drawn uniformly at random from a tile, the expected number of moving vehicle points in a window of n points is

$$\mathbb{E}[N_{18}] = n \cdot \pi_{18} \quad (2.13)$$

where π_{18} is the proportion of moving vehicle points in the tile. When π_{18} is small, most windows contain few or no minority points, whatever the class weight. Centered oversampling therefore centers a configurable proportion of windows on a randomly chosen minority class point, so that complete examples of that class are seen at a controlled frequency, described in Section 3.6.2.

2.2.5 Training and Optimization

Training minimizes the loss $\mathcal{L}$, the focal loss defined in Section 2.2.3, over the network weights θ by repeated small steps against its gradient

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta_t) \quad (2.14)$$

where t counts the updates and η_t is the learning rate, the size of each step. A rate that is too large makes the loss oscillate or diverge, and a rate that is too small makes learning slow, so the rate is reduced during training, here by an exponential decay

$$\eta_e = \eta_0 \delta^e \quad (2.15)$$

where e is the epoch, η_0 is the base rate and $0 < \delta < 1$ is the decay factor, with $\delta = 0.98$ in this work. Adaptive Moment Estimation (Adam) [23] is the optimizer used, and it keeps running averages of the gradient and of its square to scale the step of every weight separately. An epoch is normally one pass over the training data, and in this work it is defined by the window sampling of Section 3.6.2.

Four further mechanisms limit overfitting, the state in which the network fits the training data more closely than it generalizes, and instability. Weight decay adds a penalty on the squared weights to the loss, which shrinks the weights and limits how closely the network can fit the training set [24]. Dropout [25] sets a random share of the activations to zero during training so that no unit can rely on one particular other unit. Gradient clipping rescales the gradient whenever its norm exceeds a threshold, so that one unusual batch cannot cause a large step [24]. Batch normalization [26] normalizes the activations of each channel over the batch and rescales them with learned parameters, which stabilizes training but means that the network must be switched to evaluation mode at inference, so that stored running statistics replace the statistics of the batch. In this work dropout is applied in the classification head only, the gradient clipping threshold is 1.0, and weight decay was tried once, described in Section 4.5.1, and is not part of the final configuration.

Data augmentation applies random transformations that keep the labels valid, so that the network meets more variation than the tiles contain [24]. For point clouds the usual transformations are a rotation about the vertical axis, a small random

displacement of every point, called jitter, and a random scaling of the coordinates. Rotation reduces dependence on the driving direction of the survey vehicle. In this work all three are applied to every training window, described in Section 3.6.2, and none is applied during validation or inference. Figure 3.7 shows how these steps fit together in the training loop of this work.

2.2.6 Transfer Learning

Transfer learning trains a model on a source task and adapts it to a target task [18], in point cloud segmentation usually by pretraining on a large public dataset and fine tuning on a smaller domain specific one, on the assumption that low level geometric features transfer. In this thesis, the source task is semantic segmentation on SemanticKITTI [5], a large public benchmark collected with a Velodyne spinning LiDAR mounted on a driving car, and the target task is semantic segmentation on the LiDAR scanner dataset used throughout this project, collected with the RIEGL VUX-1HA-22 scanner mounted on a survey vehicle. The domain gap is the mismatch that breaks this assumption when source and target sensors differ in scan pattern, point density or radiometric calibration [19], the conversion of raw measurements into physical values. Two fine tuning risks are independent of the gap. Resetting the classification head, the final layer that assigns labels, discards class specific knowledge, which matters most when source and target share a labeling scheme. Training the head much faster than the rest of the network on a small target set lets it overfit and overwrite the pretrained mapping, a form of catastrophic forgetting [27] driven by the learning rate schedule rather than by a change of task. Figure 5.3 in Section 5.4 shows the transfer study of this work, whose two fine tuning strategies differ in exactly these two respects.

2.2.7 Validation and Generalization

The aim of training is generalization, that is, good performance on points the network has never seen. The tiles are therefore divided into training tiles, which update the weights, validation tiles, which are scored after every epoch to follow the training and to select a checkpoint, and a test tile used only for the final evaluation [24]. A checkpoint is the set of weights saved at the end of an epoch, and the best checkpoint is the one with the highest validation score. Whole tiles are kept out of training as validation tiles and as the test tile because neighboring points of one tile are almost identical, so a random split of points would let training information leak into validation.

As mentioned in Section 2.2.5, overfitting shows as a training loss that keeps falling while the validation loss stops falling or rises. The generalization gap is the difference between the score on validation data and the score on unseen data. Since the best checkpoint is picked from many epochs, its validation score is somewhat optimistic, and for a rare class it varies strongly from epoch to epoch because it rests on few points. The test tile must not influence any choice. Otherwise, the gap can no longer be measured honestly.

Validation can score every annotated point of the validation tiles, here called ex-

haustive validation, or a random selection of windows, called sampled validation. A random selection gives a noisy estimate that can repeat some points and miss others, so its values change from epoch to epoch even for an unchanged network, whereas exhaustive validation is repeatable for a given checkpoint. The two schemes are described in Section 3.6.4.

2.2.8 Evaluation Metrics

Per class performance is measured by the Intersection over Union (IoU)

$$\text{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c} \quad (2.16)$$

where TP_c , FP_c and FN_c are the true positive, false positive and false negative counts of class c defined in Section 2.2.8.1, so that exact agreement gives 1 and no overlap gives 0. Mean IoU (mIoU) averages IoU_c over the classes present in the ground truth, excluding the unclassified class. IoU is used rather than pointwise accuracy because accuracy is dominated by the true negatives of a rare class, so it stays high even when the rare class is never detected. Recall and precision are defined together with the confusion matrix in Section 2.2.8.1.

2.2.8.1 Confusion Matrices

A confusion matrix records what every prediction became, correct or wrong. For C classes it has C rows, one per true class, and C columns, one per predicted class, and entry n_{ij} counts the points of true class i predicted as class j . Figure 2.7 shows the convention for three generic classes.

	A	B	C
A	n_{AA}	n_{AB}	n_{AC}
B	n_{BA}	n_{BB}	n_{BC}
C	n_{CA}	n_{CB}	n_{CC}

Figure 2.7: Confusion matrix convention for three generic classes A, B and C. Row i and column j hold n_{ij} , the count of points whose true class is i and predicted class is j .

The following four counts are defined for one class at a time, and every other class is treated as a combined alternative. For a class c , for example moving vehicles, they are read from the matrix as follows.

- A true positive, TP_c , is a point of class c that the network predicted as class c . It is a correct detection and is the diagonal entry n_{cc} .
- A false negative, FN_c , is a point of class c that the network predicted as some other class. It is a missed detection and is the sum of the other entries in row c , $\sum_{j \neq c} n_{cj}$.
- A false positive, FP_c , is a point of some other class that the network predicted as class c . It is a false alarm and is the sum of the other entries in column c , $\sum_{i \neq c} n_{ic}$.

- A true negative, TN_c , is a point of some other class that the network did not predict as class c . It is the sum of every entry outside row c and column c .

Recall and precision are defined using these counts as

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \quad \text{Precision}_c = \frac{TP_c}{TP_c + FP_c} \quad (2.17)$$

Recall asks, of all the true points of class c , how many the network found. It is read along a row of the confusion matrix. Precision asks, of all the points the network called class c , how many truly belong to it. It is read down a column. A high recall with a low precision means that a class is found reliably but over predicted, and the reverse means that the class is rarely wrong but often missed. As an illustration, if a tile holds 100 true moving points and the network marks 300 points as moving, of which 90 are truly moving, then $TP = 90$, $FN = 10$ and $FP = 210$. The recall is $90/100 = 0.90$, the precision is $90/300 = 0.30$, and the IoU of Section 2.2.8 is $90/310 \approx 0.29$, which is low because the false positives count against it. True negatives do not enter recall, precision or IoU, which is one reason they are less useful for a rare class, where almost every point is a true negative.

A raw confusion matrix holds point counts, so a large class dominates the picture and a rare class is hard to read. Row normalization divides every entry by the total of its row

$$\tilde{n}_{ij}^{\text{row}} = \frac{n_{ij}}{\sum_k n_{ik}} \quad (2.18)$$

so that each row sums to one and describes what happened to the points of one true class i . The diagonal entry $\tilde{n}_{ii}^{\text{row}}$ is the recall of class i , and each off diagonal entry is the share of the class i points that were mistaken for class j . This view answers where the points of a class went, and it does not depend on how large the class is. It does not show how many points of other classes were predicted as class i , so a class can have a high recall in this view while being heavily over predicted. In a heatmap, every cell is shaded by its value, so that the large shares stand out without reading each number, as in Figure 2.8.

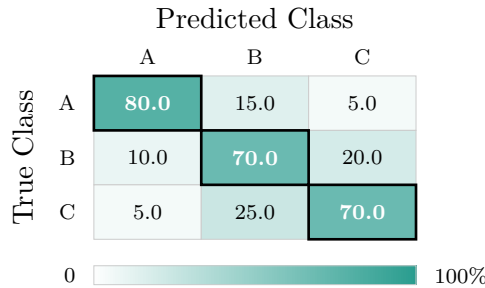


Figure 2.8: Row normalized confusion heatmap for the three generic classes of Figure 2.7.

Column normalization divides every entry by the total of its column

$$\tilde{n}_{ij}^{\text{col}} = \frac{n_{ij}}{\sum_k n_{kj}} \quad (2.19)$$

so that each column sums to one and describes what the points predicted as class j truly were. The diagonal entry $\tilde{n}_{jj}^{\text{col}}$ is the precision of class j , and each off diagonal entry is the share of the class j predictions that truly belong to class i . This view answers how far a prediction of a class can be trusted, and it exposes the over prediction of a rare class. It does not show how many points of a class were missed, so the two views are read together. A class absent from the ground truth has an empty row and a class never predicted has an empty column, and these have no normalized values because their total is zero. Figure 2.9 draws this matrix as a heatmap in the same way.

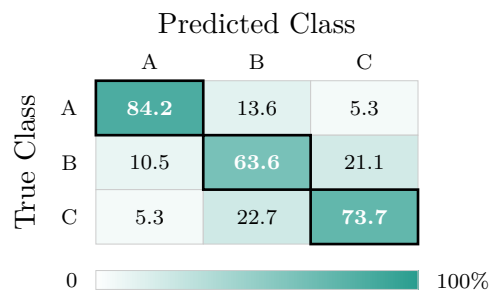


Figure 2.9: Column normalized confusion heatmap for the three generic classes of Figure 2.7.

3

Methodology

This chapter helps to get familiar with the vocabulary in Section 3.1, the dataset in Section 3.2 and an overview of the pipeline in Section 3.3. It covers how the training was implemented, using how many input features in Section 3.4, which network architecture in Section 3.5, which training strategy in Section 3.6 and the number of iterations it took to achieve the result in Section 3.7. The results of the runs are written in the same order mentioned here in Chapter 4.

3.1 Terminology and Experiment Overview

Five iterations were carried out during this thesis. The first four each consist of a single training run, and the fifth consists of three parallel runs. A version describes a network architecture, and is used to describe what changes from one version to another at each encoder stage, described in Section 3.5. Experiments were also carried out between Iteration 4 and Iteration 5 using V2. The experiments included a transfer study in Section 3.7.3 and an RGB integration run in Section 3.7.4. A checkpoint consists of the network’s weights, saved at the end of an epoch, as described in Section 3.6.3, and is distinct from the per class weights w used in the loss function. To produce a prediction, a point cloud is passed through the network’s encoder and decoder stages, and the prediction head reads out a class score for each point using the checkpoint’s saved weights. Figure 3.1 shows all the runs in chronological order, with Checkpoint A defined in Section 3.7.2 and Checkpoint B in Section 3.7.4.

A checkpoint consists of weights of the implemented network saved at the end of an epoch, as described in Section 3.6.3. A dataset profile is a selection of tiles based on color availability, used in Iteration 5 and distinguished into three types, Colored, Mixed and Uncolored tiles. A tile is the representation of a LAS file, the point cloud format described in Section 3.2, delivered by Atritec AB, identified by the name in Table 3.3. A sequence is the corresponding unit of SemanticKITTI [5], one continuous drive recorded as many consecutive scans, and it is used only in the transfer study. The reserved test tile is the tile on which final inference was run and it was never used for training or validation. Checkpoint B was also run on it once as a diagnostic, described in Section 4.5.3.

Sampled validation and exhaustive validation are the two validation schemes used in the experiment, and are defined in Section 3.6.4.

Class numbers are defined using the LAS classification codes throughout the thesis, listed in Table 3.2. Classes are used along with IoU with the two digit LAS code as

Table 3.1: Overview of the training runs.

Run	Data Used	What Changed	Purpose or Lesson	Defined	Reported
Iteration 1	Six tiles, sampled validation	V1, six input features, scratch training, moving vehicle weight 3.0	The moving class flattens early	3.7.1	4.1
Iteration 2	As Iteration 1	Training continued, first full tile inference, four epoch smoke test at weight 10	Validation IoU hides over prediction	3.7.1	4.1
Iteration 3	As Iteration 1	Moving vehicle weight 1.5	Too conservative. The usable weight lies between 1.5 and 3.0	3.7.1	4.1
Between Iterations 3 and 4	Eleven tiles	Range feature added, neighborhood widened, window reduced, road oversampling tested, V2 completed	Road oversampling collapses IoU ₁₈ . A shared architecture definition is adopted	3.7.1	4.2
Iteration 4	Eleven tiles, sampled validation	V2, seven input features, scratch training, weight 2.5	Road and ground unresolved	3.7.2	4.3
Phase 1, Variants Z, O, D	SemanticKITTI sequences	V2, three return encoding and augmentation variants	Variant Z epoch 12 feeds Experiment A, Variant O adopted and its epoch 36 feeds Experiment C, Variant D rejected	3.7.3	4.4
Phase 2, Experiments A, C	Eleven tiles, validation on tile_06	Fine tuning from Phase 1 checkpoints, head reset in A, discriminative rates in C	Neither strategy reaches the Iteration 4 baseline	3.7.3	4.4
RGB Integration Run	Twenty-two tiles, sampled validation	Ten input features with color, epoch 83 checkpoint resumed at a lower rate	Resumed training needs a learning rate of 10^{-3} . Checkpoint B. Road and ground improve with color	3.7.4	4.5
Iteration 5, Three Profiles	Colored 17 tiles, Mixed 22 tiles, Uncolored 4 tiles, exhaustive validation	Corrected training and evaluation code, from scratch at the lower learning rate	Final configuration. Colored has the best checkpoint	3.7.5	4.6
Reserved Test Tile	Tile 51	Iteration 5 Colored checkpoint, epoch 130	Best checkpoint of this thesis	3.6.4	4.6.4

subscript. The same subscript is used for the class weights w and the oversampling probabilities p .

Table 3.1 lists all the training runs, with pointers to the corresponding network configuration and to the outcome.

3.2 Dataset

3.2.1 Data Acquisition and Class Scheme

The LAS tiles were produced by Atritec AB. LAS is a standardized binary file format for storing unstructured point clouds, defined and maintained by the American Society for Photogrammetry and Remote Sensing (ASPRS) [21]. ASPRS defines the fixed set of codes to classify the point clouds, making them easier to share and interpret across software tools and organizations. Figure 2.1, shown in Section 2.1.3, illustrates the general idea before Table 3.2 lists the eight classes used in this thesis. Eight codes following the ASPRS convention, also called classes, were defined jointly with Atritec (Table 3.2). Figure 3.2 shows an example of each visible class in an

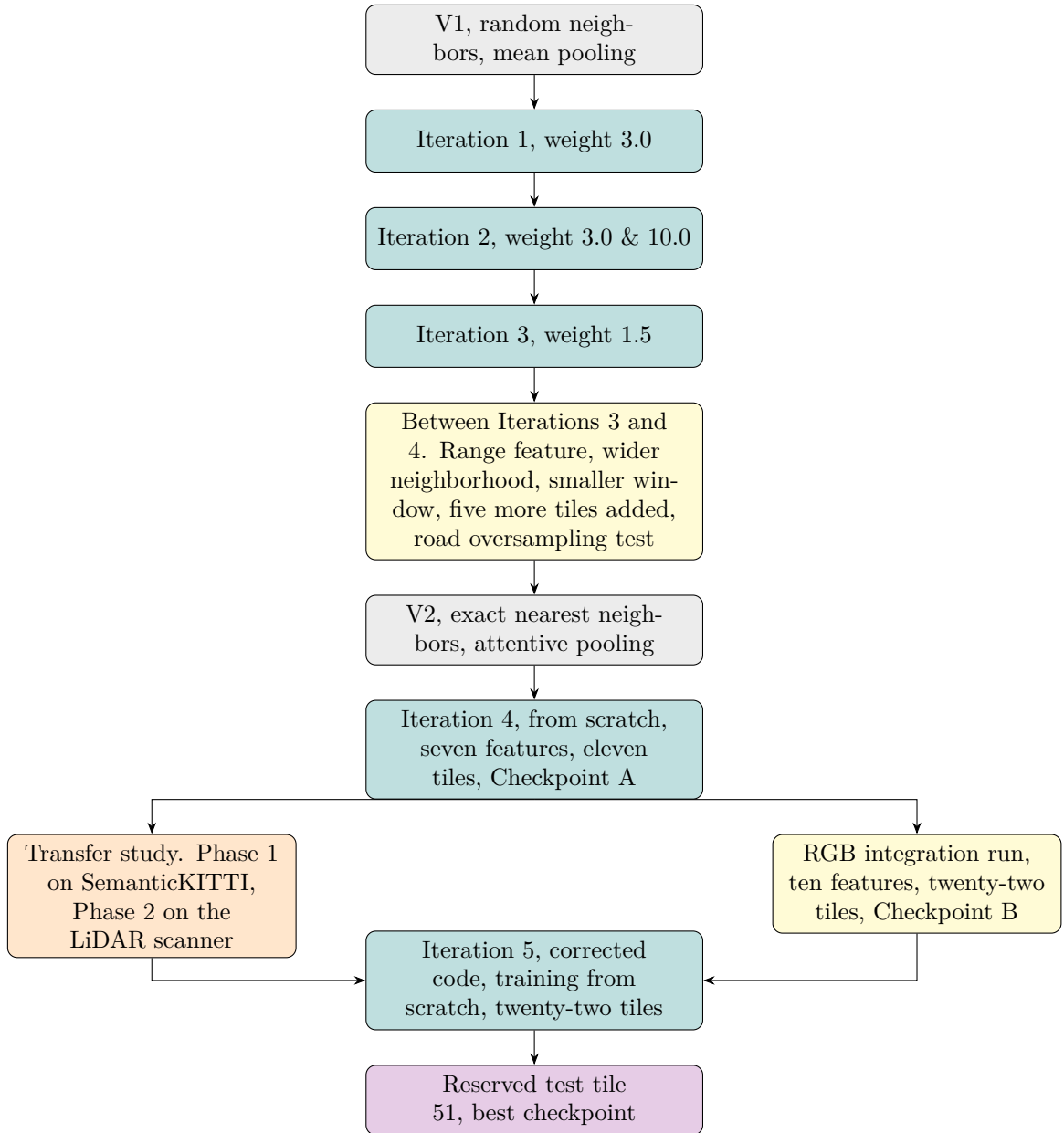


Figure 3.1: Runs in chronological order, with teal for the iterations, yellow for transitional changes and for the RGB integration run, gray for the network versions, orange for the transfer study, and purple for the evaluation.

RGB colored tile. Each box marks a ground truth example of one class, labeled with its LAS code. Building (IoU_{06}) does not occur in this view. The moving vehicle (IoU_{18}) at the bottom is the distortion left by its motion during the scan. The tile has been rendered using CloudCompare [28], a point cloud visualization and editing software.

Table 3.2: Class codes used by the network and in the LAS tiles.

LAS Code	Model Index	Name
0	0	Unclassified
2	1	Ground
3	2	Low vegetation
4	3	Medium vegetation
5	4	High vegetation
6	5	Building
11	6	Road
18	7	Moving vehicle

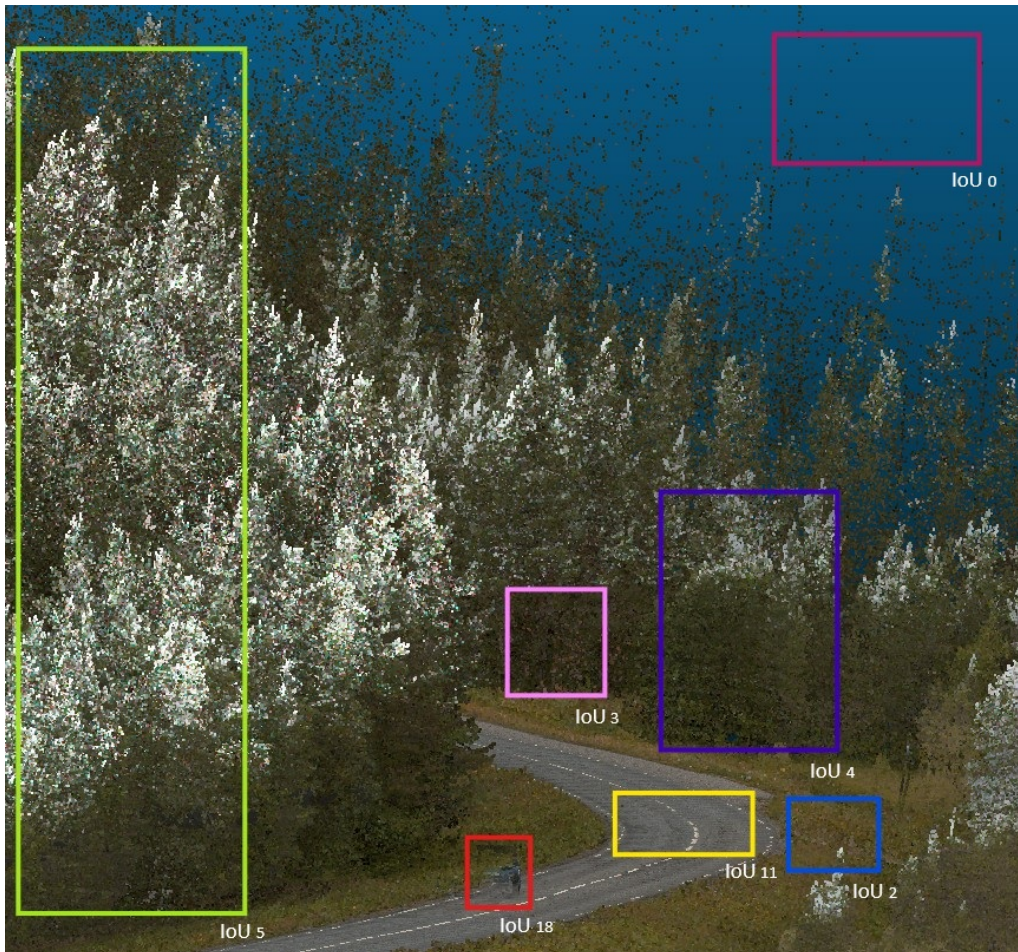


Figure 3.2: Ground truth example of each visible class in one tile.

3.2.2 Annotation Protocol

Annotations were done manually by assigning the codes directly into the LAS tile and were validated using the classification tool in CloudCompare 2.13 [28]. Priority was set on annotating moving vehicles and road while annotating other classes sparsely. Figure 3.3 shows a training tile annotated under this protocol.

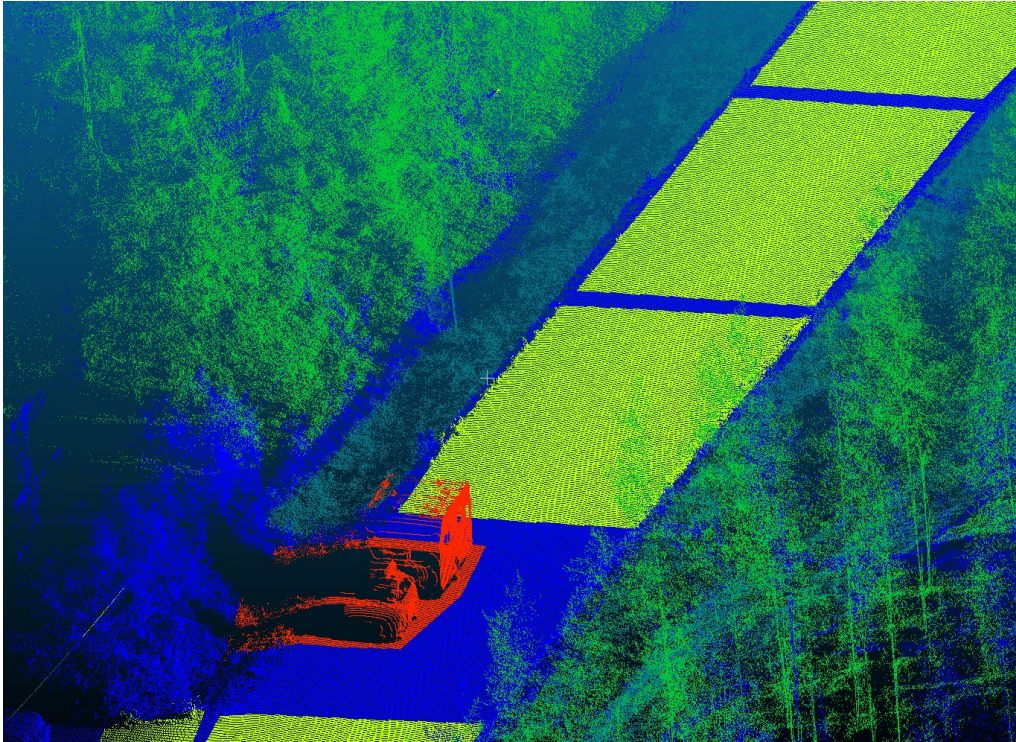


Figure 3.3: A manually annotated training tile rendered by classification in CloudCompare. Road (IoU_{11}) is annotated as patches (yellow), the moving vehicle (IoU_{18}) distortion is annotated in full (red), vegetation is annotated on both sides (green), and the remaining points carry no label (blue).

Moving vehicles, Class 18, cover all vehicles seen as point clusters distorted along their trajectory, as explained in Section 2.1.2. Once several tiles were found to depict vehicles at different positions and orientations, all visible vehicles were annotated with the annotation tool. The distortions were annotated in full, including faint point clusters along their edges caused by the trailing distortions. This procedure was carried out to capture every Class 18 point. Road, Class 11, was treated as the driving surface. The full road width was annotated continuously over the entire length of the road segments on each tile, with at least 30 patches per tile. Ground, Class 2, consists of bare earth and vegetated verges along both road sides. To capture the ground at different elevations, 10 to 15 sparsely placed patches per tile in both road verges were sufficient. Vegetation was divided into low, Class 3, medium, Class 4 and high, Class 5, based on height. Building, Class 6, received the least attention since buildings appeared in only a few tiles, and is not visible in Figure 3.2. The result of this annotation protocol strongly influences the evaluation of the validation and test tiles in Chapter 4.

3.2.3 Tile Sets per Run

Table 3.3 lists all twenty-six tiles, and Figure 1.1 shows the composition of the full dataset by class. Twenty-two tiles form the training set, three tiles form the validation set, and one tile forms the reserved test tile. The export script stops if a tile occurs in more than one split or if an export from an earlier run remains in a split folder, so no tile is used in two splits.

Table 3.3: Point counts per class and total point count per tile across all twenty-six tiles, split into training, validation and test sets.

Tile	Total	Ground	Low Veg	Med Veg	High Veg	Building	Road	Moving	Split
10	74,475,352	0	541,812	0	612,932	0	14,709,314	309,163	Train
11	77,843,416	0	478,594	0	2,337,439	0	21,597,265	2,420,720	Train
12	72,769,197	0	256,644	0	10,654,627	0	21,562,539	3,017,839	Train
13	76,170,113	0	1,379,498	0	11,849,144	0	20,004,383	6,652,177	Train
15	74,475,352	0	2,997,661	0	2,466,220	0	19,213,676	1,399,315	Train
20	74,673,391	3,470,094	0	0	0	0	17,187,545	45,084	Train
21	73,872,463	3,237,776	0	0	0	0	12,358,596	16,801	Train
22	75,267,132	2,167,725	0	0	0	0	15,015,318	33,346	Train
23	80,079,992	5,895,700	0	0	0	0	15,178,119	24,170	Train
26	79,080,328	4,410,515	0	0	0	0	13,747,043	68,571	Train
27	79,739,597	3,180,534	0	0	0	0	13,172,090	67,877	Train
28	88,849,488	2,036,114	0	0	0	0	14,281,062	90,784	Train
29	44,728,869	1,552,937	0	0	0	0	9,781,259	16,108	Train
50	69,925,506	0	4,251,244	629,985	19,194,787	121,879	33,478,079	122,461	Train
52	92,015,710	739,745	7,544,187	216,084	30,357,433	114,183	36,257,354	103,480	Train
tile_0	105,906,101	92,024	263,819	12,966	3,480,412	0	6,062,886	0	Train
tile_02	103,216,190	185,259	519,636	235,087	242,120	0	7,256,291	200,357	Train
tile_03	108,418,102	124,838	618,659	17,233	3,443,484	0	8,964,797	0	Train
tile_04	122,554,657	77,113	907,086	1,455,945	2,376,535	27,925	1,281,474	220,157	Train
tile_05	114,121,425	141,690	655,187	36,960	397,530	0	3,428,858	147,235	Train
tile_07	51,402,551	186,198	2,011,847	306,766	853,240	0	3,035,733	0	Train
tile_01	122,769,716	198,360	323,757	224,740	2,794,484	0	4,622,290	592,031	Train
24	78,936,984	2,462,968	0	0	0	0	13,400,835	100,470	Validation
25	79,888,843	2,306,751	0	0	0	0	14,537,161	15,602	Validation
tile_06	93,706,539	129,063	979,769	393,579	3,673,801	33,880	4,561,558	613,182	Validation
51	84,741,029	455,112	9,988,079	0	27,726,462	95,158	35,816,963	84,621	Test (locked)

The manual annotations were not uniform across tiles. The focus was on what mattered most and not necessarily on being exhaustive everywhere. This explains why some tiles were sparsely annotated, while others contained dense labeled points. Not every training run used all tiles. Table 3.4 shows the tile counts for each run, and the validation and test schemes are defined in Section 3.6.4. The training set grew from six tiles in Iterations 1 to 3, to eleven in Iteration 4 and the transfer study, to twenty-two in the RGB integration run and Iteration 5.

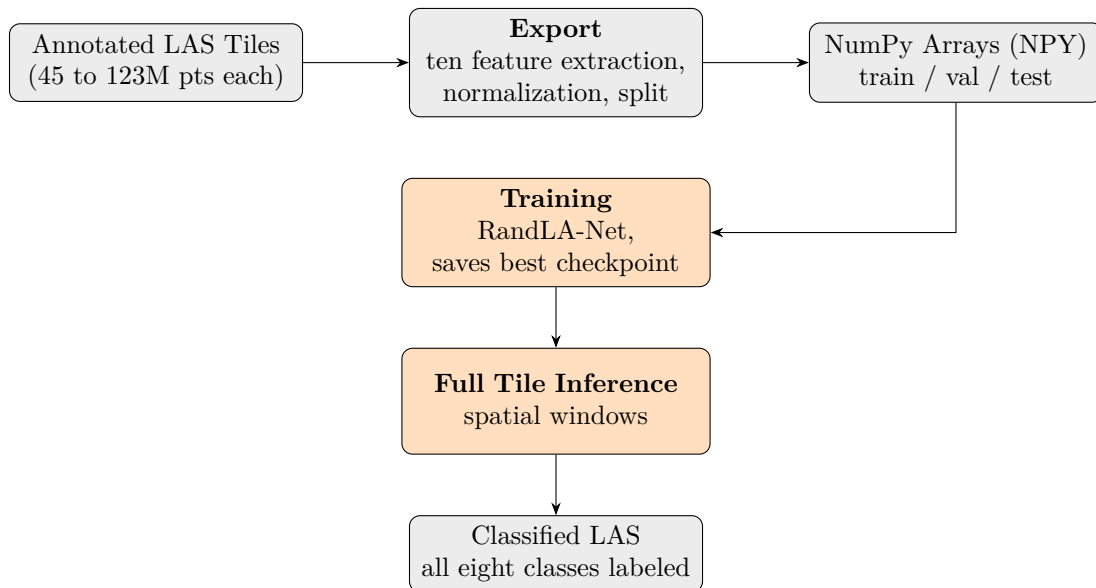
3.3 Pipeline Overview

Figure 3.5 shows the end-to-end pipeline of the RandLA-Net [6] as described in Section 2.2. This includes data preparation, learned segmentation, and full tile inference. Full tile inference and the exhaustive validation scheme of Section 3.6.4 share the spatial window and vote averaging step, combining overlapping window predictions for the same point by majority vote, shown in the Full Tile Inference block and given in Algorithm 3 in Appendix B.

Table 3.4: Training, validation and test tiles of every run.

Run	Train Tiles	Validation Tiles	Validation Scheme	Test Tiles
Iterations 1 to 3	6	1	sampled	1
Iteration 4	11	1	sampled	1
Phase 2, Exp. A & C	11	1	sampled	1
RGB Integration Run	22	3	sampled	1
Iteration 5, Colored	17	3	exhaustive	1
Iteration 5, Mixed	22	3	exhaustive	1
Iteration 5, Uncolored	4	1	exhaustive	1

All 26 Tiles	Colored	Mixed	Uncolored
Tiles 10 to 13 4 tiles, no color	not used	train	train
Tile 15 1 tile, no color	not used	train	validation
Colored Tiles, 17 tiles 20 to 23, 26 to 29, 50, 52, tile_0 to tile_05, tile_07	train	train	not used
Validation Tiles, 3 tiles 24, 25, tile_06, with color	validation	validation	not used
Reserved Test Tile, 1 tile 51	test	test	test
Tiles per Profile	17 train 3 validation 1 test	22 train 3 validation 1 test	4 train 1 validation 1 test

Figure 3.4: Tile split of the three Iteration 5 profiles.**Figure 3.5:** End-to-end pipeline, with gray blocks for the input and output data, and orange for training and evaluation.

3.4 Input Features

3.4.1 Ten Feature Vector

The input features described above are concatenated into a single vector per point. Every point enters the network as

$$\mathbf{f} = [x', y', z', i_{\text{norm}}, r_{\text{norm}}, n_{\text{ret}}, N_{\text{ret}}, R_{\text{norm}}, G_{\text{norm}}, B_{\text{norm}}] \in \mathbb{R}^{10}. \quad (3.1)$$

Here, $\mathbf{f}$ is the ten dimensional input feature vector assigned to each point in each LAS tile. x' , y' , and z' are the spatial coordinates of the point after centering, described in Algorithm 1. i_{norm} is the normalized laser return intensity at that point, and r_{norm} is its normalized range, the distance from the scanner to the point. n_{ret} is the return number, recognizing which echo within its pulse this point corresponds to, and N_{ret} is the total number of returns produced by that pulse. Finally, R_{norm} , G_{norm} , and B_{norm} are the normalized red, green, and blue color values of each point. Range $r = \sqrt{x^2 + y^2 + z^2}$ is calculated from the uncentered coordinates and normalized by the largest range in the tile as r_{norm} . Intensity is normalized by its maximum to the range zero to one. The return attributes n_{ret} and N_{ret} are taken without changes as integers ranging from one to five. The color channels $(R_{\text{norm}}, G_{\text{norm}}, B_{\text{norm}})$ are the color values normalized to the range zero to one in the same manner as intensity, described further in Section 3.4.2. Algorithm 1 in Appendix B explains the export procedure. The order in which these features are computed matters, since range is taken from the uncentered coordinates so that it measures distance to the scanner rather than to the tile centroid.

3.4.2 Feature Count by Iteration

The feature vector started with six features and evolved into ten after experiments and iterations. Six features were present during Iterations 1 to 3, seven from Iteration 4 including the transfer study, and finally ten during the RGB integration run and Iteration 5.

The first generation used six features, x' , y' , z' , i_{norm} , n_{ret} , N_{ret} , without r_{norm} . After inference error analysis, false negatives concentrated at long range were found where point density drops. This means the network lacked the means to adapt its decision behavior to density. To address this issue, r_{norm} was added.

Later, the feature vector was extended from seven to the ten dimensions of Equation 3.1 for the RGB integration run and Iteration 5. Not all the LAS tiles contained RGB. To use this ten feature vector everywhere, an RGB value of 0 was assigned to the tiles without color, while the tiles with color kept their measured values.

Changing the input features altered the width of the first local feature aggregation layer in the first encoder stage. This changes the shape of the first layer weights while the rest of the network's weights remain unaffected. The shape mismatch is the reason a seven feature checkpoint could not be loaded into a ten feature model. Rather than reinitialize only the first layer, training was done from scratch.

3.5 Network Architecture

3.5.1 V1 and V2

RandLA-Net [6] is the encoder decoder architecture described in Section 2.2. V1 followed RandLA-Net with two modifications inside each local feature aggregation (LFA) block, random selection of neighbors in place of the exact nearest neighbor search and mean pooling in place of attentive pooling, both for computational efficiency. The upsampling in the decoder used a repeat interleave operation that duplicates the elements of the feature vector without assigning the duplicated element to a location, a limitation since the position vectors computed in Equation 2.8 describe the relationship between a point and a random point which is often far away. V2 is the successor of V1, restoring exact nearest neighbor search and attentive pooling as in the original RandLA-Net. It is the network architecture used to train from Iteration 4 onward, with only the training data, the input features and the initial weights varied between the runs. Table 3.5 compares both designs.

Table 3.5: Comparison of V1 and V2 architecture configurations.

Parameter	V1 (Iterations 1 to 3)	V2 (Iteration 4 Onward)
Neighbor Selection	Random	Exact, chunked KNN
Aggregation Method	Mean pooling	Attentive, two units + residual
Upsampling Method	Repeat interleave	Nearest neighbor interpolation
Decimation Factor	4	4
Encoder Stages	5	5
Output Classes	8	8
Total Parameters	52,176	85,925

V2 completes the design in four respects, implementing the complete architecture of Section 2.2 shown in Figure 3.6, with the local feature aggregation unit detailed in Figure 2.6, as follows.

- Exact nearest neighbor search was done in chunks. Instead of comparing every point in a window with other points, the network takes the points in chunks of 2,048, avoiding memory conflicts.
- Two local feature aggregation units are placed one after another at each encoder stage, starting from an input window of $N = 65,536$ points (Section 3.6.2).
- Upsampling is based on nearest neighbors. When the decoder needs to bring back the points that were removed, each restored point copies the features of whichever remaining point is closest to it.
- A prediction head made of a shared MLP, 64 features wide, followed by dropout [25] of 0.5 to reduce overfitting, and a linear layer that outputs the eight class scores for every point.

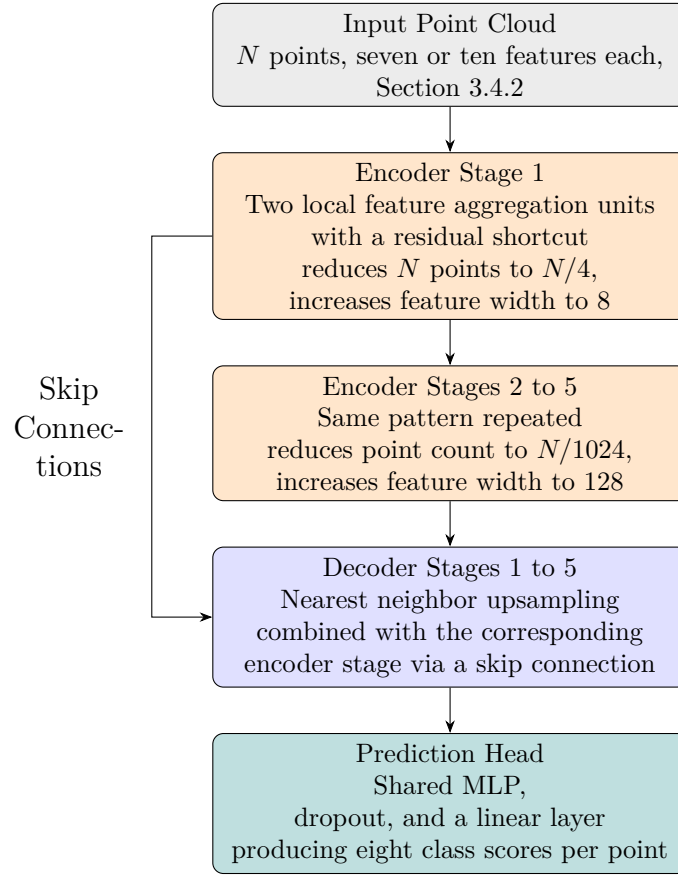


Figure 3.6: V2 architecture overview.

3.6 Training and Evaluation

3.6.1 Loss and Class Weights

All the runs have used focal loss [17] with $\gamma = 2$ as defined in Section 2.2.3, weighted per class by the vector $\mathbf{w}$, indexed by LAS code and defined as follows.

$$\mathbf{w} = [w_0, w_{02}, w_{03}, w_{04}, w_{05}, w_{06}, w_{11}, w_{18}] = [0, 1, 1, 1, 1, 1, 1.2, 2.5]. \quad (3.2)$$

After the weight bracketing of Iterations 1 to 3, described in Sections 3.7.1 to 3.7.1, the static weights of Equation 3.2 were used for every training run from Iteration 4 onward, with unclassified points excluded and moving vehicles assigned the empirically stable weight of 2.5. The road weight was set to $w_{11} = 1.2$. Phase 1 pretraining on SemanticKITTI used its own weights, $w_{18} = 3.0$ and $w_{11} = 1.0$, given in Section 3.7.3 and Table A.1.

3.6.2 Window Sampling and Oversampling

The window size was 65,536 points from Iteration 4 onward. This size is limited to a multiple of 4^5 by the five decimation stages at a batch size of 8. Windows are drawn with probability $p_{18} = 0.5$ centered on a random Class 18 point, with

the search radius scaled so the window remains compact. Otherwise, points are drawn uniformly from the tile. In Iteration 5 one epoch draws 100 windows for every training tile, so the number of windows per epoch grows with the number of training tiles. Every training window is rotated by a random angle about the vertical axis, and its coordinates are perturbed by Gaussian noise with a standard deviation of 0.01 m and scaled by a random factor between 0.9 and 1.1, while the remaining input features are left unchanged. The full training procedure is given as Algorithm 2 in Appendix B.

3.6.3 Optimization and Checkpointing

Network weights are updated using Adam [23], as described in Section 2.2.5. Model selection uses validation mIoU, the mean IoU across classes defined in Section 2.2.8. The best checkpoint is overwritten whenever a new epoch beats the running best validation mIoU, and epoch checkpoints are saved every ten epochs to resume running if needed. Figure 3.7 shows how the steps of this section fit together in one loop.

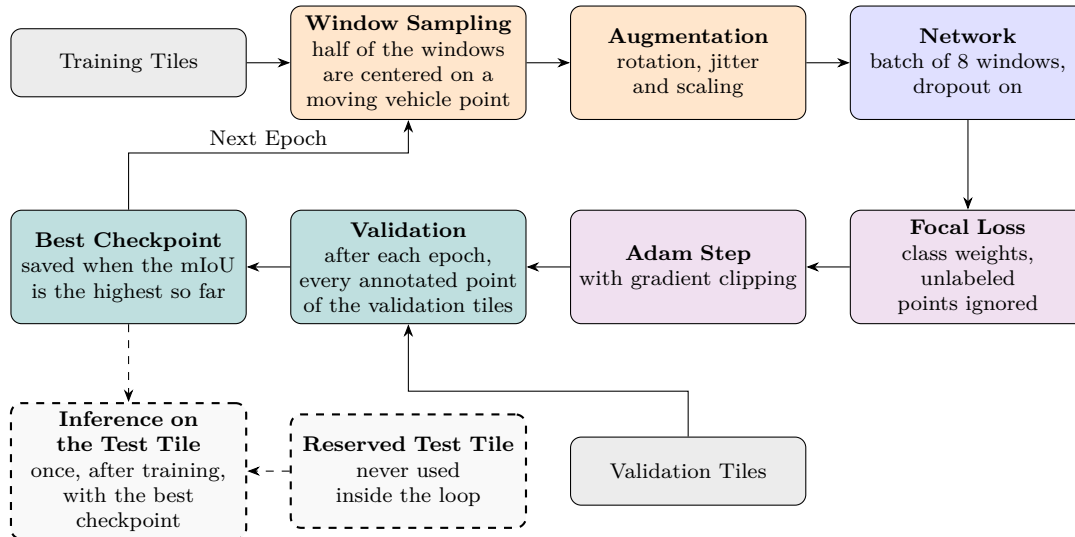


Figure 3.7: Training, validation and inference loop.

3.6.4 Validation and Inference

Two validation schemes are used. Sampled validation uses a random selection of windows drawn from each validation tile at the end of every epoch, and applies to every run except Iteration 5. Exhaustive validation uses every annotated point of every validation tile present in Iteration 5.

Inference was run on the reserved test tile with the best checkpoint. During inference, the tile was divided into spatial windows of a size similar to training, each point in the window was classified, and the classification codes were written into a copy of the same LAS tile. Algorithm 3 in Appendix B gives the procedure. On a single graphics processing unit (GPU) of the cluster described in Appendix C, inference on reserved test tile 51 took approximately 26 minutes to classify about 85

million points. The total time to generate a classified tile depends on the number of points and spatial windows in the LAS tile. The reserved test tile is compared with the classified tile to understand the per class IoU between the ground truth and predicted points. The comparison is developed into a confusion matrix, available in Table 4.6.

3.7 Iterations and Experiments

This section helps to understand the configuration of the training listed in Table 3.1. The subsections describe the changes with respect to the previous run. The results are in Chapter 4.

3.7.1 Iterations 1 to 3

Iteration 1 was the first training run. It trained V1 with six input features, the vector of Equation 3.1 without range and color, on the six tiles, using random initialization at an initial learning rate of $\eta_0 = 10^{-2}$ under sampled validation on one validation tile. The moving vehicle weight was set to $w_{18} = 3.0$.

Iteration 2 training was initiated with the Iteration 1 best checkpoint and a full tile inference was run on a training tile. The objective was to see the behavior of the network. Within the same iteration a smoke test, a short trial run to catch obvious problems early, was run at $w_{18} = 10$, following the inverse frequency intuition to analyze the effect of the weight.

Iteration 3 lowered the moving vehicle weight to $w_{18} = 1.5$ with the rest as in Iteration 1. It was built to analyze the effect of the weight from below after the over prediction in Iteration 2.

Several experiments were run and changes were made in response to observations. Range was added as the seventh input feature, described in Section 3.4.2. The number of neighbors K in local feature aggregation was raised from 16 to 32, since elongated distortions need a wider local region. The training window was reduced to 65,536 points due to memory limitations after exact nearest neighbor search and attentive pooling were introduced. Further, road oversampling $p_{\text{road}} = 0.3$ was tested, with the outcome reported in Section 4.5.1. The network upgrade was a new network version V2.

3.7.2 Iteration 4

Iteration 4 was the first training on V2. It trained with seven input features from random initialization on the eleven tile set with the moving vehicle weight set to $w_{18} = 2.5$ as the compromise between the weight of 3.0 used in Iteration 1 and the weight of 1.5 used in Iteration 3, moving vehicle oversampling at $p_{18} = 0.5$ and $\eta_0 = 5 \times 10^{-3}$. Validation was sampled on tile_06. Its epoch 131 checkpoint is Checkpoint A, the Iteration 4 checkpoint, and it is marked as the baseline against which the transfer study and the RGB integration run are judged.

3.7.3 Transfer Study

Phase 1 consists of training V2 on SemanticKITTI [5], a large public dataset of labeled scans from a Velodyne scanner, with its classes assigned to the eight class scheme of Table 3.2. A full SemanticKITTI sequence held up to 590 million points, so each sequence was reduced to 40 million points by a sub-sampling method that retained every moving vehicle and road point and sampled the other classes uniformly at random. Three variants were trained, differing in how the return attributes n_{ret} and N_{ret} of Section 3.4.1 were encoded, since the Velodyne data records a single return per pulse, and in whether density augmentation, randomly varying the point density of a tile during training, was applied.

Phase 2 was designed to fine tune Phase 1 checkpoints on the eleven tile training set of Iteration 4 with sampled validation on tile_06, so that its values are comparable with those of Iteration 4. Two strategies were evaluated, differing in whether the pretrained classification head was reset and how learning rates were assigned across the parameter groups of Section 3.5.1. Figure 5.3 in Section 5.4 shows the design of the study.

3.7.4 RGB Integration Run

The training was similar to previous iterations except the feature vector was updated to ten dimensions and random initialization was used on twenty-two tiles with $w_{18} = 2.5$, $p_{18} = 0.5$. Sampled validation on three tiles was used. Since the seven feature checkpoint’s input layer shape does not match this ten feature model, it could not be loaded to initialize training, as mentioned in Section 3.4.2. Training was therefore done from scratch.

The run proceeded as a series of sub runs, reported in that order in Section 4.5. The first sub run, from random initialization at $\eta_0 = 10^{-2}$, reached epoch 83 and produced the best checkpoint used for future training. A sub run then introduced two changes at once, an L_2 weight decay of 10^{-4} on all optimizer parameters and centered oversampling for the ground class, $p_{\text{ground}} = 0.15$, to improve the IoU. The collapse that followed and the rollback of both changes, along with the subsequent resumption of the epoch 83 checkpoint at a lower rate, are described in Section 4.5.1. Epoch 34 from that last sub run is Checkpoint B, the best result before Iteration 5. Two engineering changes were made during this run. Data loading was changed from a single worker process to six workers with pinned memory, host memory reserved so it can be transferred to the GPU faster, and the validation loop was batched. This was done to reduce GPU idle time without reintroducing the memory problem that had motivated the single worker. Epoch times are recorded in Table A.1 in Appendix A. Checkpoint B was run once on the reserved test tile as a diagnostic, reported in Section 4.5.3.

3.7.5 Iteration 5

Iteration 5 is the final configuration of this thesis. Before Iteration 5 training started, the entire network code was inspected with respect to the issues encountered in previous iterations. Three defects were found and they were fixed. First, validation

was using a random selection of windows per tile, so the validation IoU was measured on a random subset that could repeat points and miss others. This was replaced by the exhaustive validation of Section 3.6.4, which used every annotated point once. Second, the focal loss computed its modulating factor from the class weighted cross entropy rather than from the unweighted class probability, and averaged over every point in a window including the unlabeled points, so the class weights of Equation 3.2 entered the loss twice and the gradient of the annotated points was diluted. The loss computes the modulating factor from the unweighted probability, applies the class weight once, and averages over annotated points only. Third, window sampling and validation were made deterministic under fixed seeds, a check was added that no tile occurs in both the training and the validation set, and the batches with no annotated point were skipped.

The configuration of Iteration 5 is V2 with ten input features, trained from scratch at $\eta_0 = 10^{-3}$ from the first epoch, with a 150 epoch schedule and exhaustive validation, and the remaining settings as in Section 3.6 and Table A.1. The three profiles of Table 3.6 were trained in parallel, with one question in mind, namely which set of tiles identifies the classes with the highest IoU.

Table 3.6: Dataset profiles used for Iteration 5 training.

Profile	Train Tiles	Validation Tiles	Color Channels	Test Tile
Colored	17	3	real values on all tiles	tile 51
Mixed	22	3	real values on 17 tiles, zero values on 5 tiles	tile 51
Uncolored	4	1	zero values on all tiles	tile 51

The Colored checkpoint was picked for the inference on the reserved test tile, described in Section 4.6.3, based on the validation evidence of Section 4.6. The Colored profile had the best validation values across most classes. The Uncolored profile had a high value for moving vehicles alone while its vegetation classes lagged, and the Mixed profile had a similar result to Colored on most classes but its moving vehicle IoU stayed below that of Colored.

4

Experimental Results

This chapter reports the outcome of the training runs defined in Section 3.7. Sections 4.1 to 4.2 cover Iterations 1 to 3 and the changes before Iteration 4. Section 4.3 covers Iteration 4. Section 4.4 covers the transfer study. Section 4.5 covers the RGB integration run, and Section 4.6 covers Iteration 5 along with the reserved test tile evaluation. Two quantities beyond IoU appear in the test tile sections, recall and precision, as defined in Section 2.2.8.1. Recall that IoU ranges from 0 to 1, with values closer to 1 indicating better agreement between predicted and ground truth points. Points with no ground truth label, the unclassified Class 0, are excluded from the loss and from every IoU value, and they appear only as the unclassified row of a confusion matrix. Table 4.1 places all the training runs on one page, in the order of Figure 3.1. Rows measured under sampled and exhaustive validation, or on different validation tiles, are not directly comparable. The last two rows are the only values measured on unseen data.

4.1 Iterations 1 to 3

Iteration 1, at $w_{18} = 3.0$, started to learn the moving class but its validation curve flattened quickly. Iteration 2 started from the Iteration 1 best checkpoint and ran the first full tile inference on a training tile. Upon inspection, a failure was found. More than half of all the points were predicted as moving vehicles while the validation IoU_{18} was about the same as the Iteration 1 value. The smoke test at $w_{18} = 10$ had produced the same failure, with nearly every point predicted as Class 18. Next, the weight was lowered to $w_{18} = 1.5$ while running Iteration 3. Over prediction decreased, and training was stable, which was an improvement over the previous run, but the network became conservative, and the best IoU_{18} dropped to about a third of the Iteration 1 value. It was therefore understood that the usable weight lies between 1.5 and 3.0, reported in Table 4.2, and 2.5 was adopted for Iteration 4.

4.2 Between Iterations 3 and 4

When the tiles with high road density were introduced along with road oversampling at $p_{\text{road}} = 0.3$, random windows drawn from them were already dominated by road surfaces. This made IoU_{18} collapse to zero for forty epochs. Road oversampling had to be removed.

Table 4.1: Every training run of this thesis in the order presented.

Run	Evaluation	mIoU (Epoch)	IoU ₁₈ (Epoch)
Iteration 1	sampled, one tile	0.0824 (36)	0.1347 (36)
Iteration 2, continued	sampled, one tile	—	0.1315
Iteration 3	sampled, one tile	0.0881	0.0468
Iteration 4	sampled, tile_06	0.1026 (131)	0.3131 (93)
Phase 1, Variant Z	SemanticKITTI	0.3379 (41)	0.4708 (47)
Phase 1, Variant O	SemanticKITTI	0.3452 (18)	0.4655 (36)
Phase 1, Variant D	SemanticKITTI	0.2229	0.2291
Phase 2, Experiment A	sampled, tile_06	0.0819 (85)	0.1966 (85)
Phase 2, Experiment C	sampled, tile_06	0.0707 (6)	0.1723 (8)
RGB Integration Run, Checkpoint B	sampled, three tiles	0.1057 (34)	0.3386 (34)
Iteration 5, Uncolored	exhaustive, one tile	0.5761 (145), 4 classes	0.9213 (145)
Iteration 5, Mixed	exhaustive, three tiles	0.6455 (149), 7 classes	0.5848 (149)
Iteration 5, Colored	exhaustive, three tiles	0.6767 (130), 7 classes	0.7844 (130)
Checkpoint B on tile 51	test, diagnostic	0.1842 (34), 6 classes	0.0018 (34)
Iteration 5 Colored on tile 51	test, final	0.5710 (130), 6 classes	0.7430 (130)

Table 4.2: Iterations 1 to 3 outcomes.

Run	w_{18}	IoU ₁₈	Class 18 Prediction
Iteration 1	3.0	0.1347	not measured
Iteration 2, continued	3.0	0.1315	59.7 percent
Smoke test, four epochs	10	—	99.6 percent
Iteration 3	1.5	0.0468	reduced over prediction

4.3 Iteration 4

Iteration 4, described in Section 3.7.2, is the first run on the new architecture, V2, introduced in Section 3.5.1 with nearest neighbor search and attentive pooling in each local feature aggregation (LFA) block. Table 4.1 records its values.

The epoch 131 checkpoint improved on the previous best checkpoint. This was named Checkpoint A. This served as the reference for the transfer study and the RGB integration run. IoU₁₈ was volatile from epoch to epoch, and road and ground remained far below vegetation and moving vehicles, which is the reason the RGB

integration run was implemented to bridge this gap as reported in Section 4.5. Buildings were never predicted correctly in the validation tile.

4.4 Transfer Study

Iteration 4 left the moving vehicle IoU too low, so pretraining on a large public dataset aimed to raise it. The transfer learning approach, described in Section 3.7.3, checks whether pretraining V2 on SemanticKITTI [5] improves on Checkpoint A.

Three variants were trained in Phase 1 and are scored on the SemanticKITTI validation split, reported in Table 4.1. Variant Z confirmed that the architecture learns the semantics properly, with road IoU above 0.9 in every logged epoch, while two of the seven input channels carried no values. After debugging, it was found that both return attributes were encoded as zero. Variant O corrected this by encoding both return attributes as one, and started higher on moving vehicles and on vegetation at matched epochs and no longer recorded dead channels. Its epoch 36 checkpoint was the source for Experiment C. Variant D, which added density augmentation, reached less than half the moving vehicle IoU of Variants Z and O, and its checkpoint was discarded.

Both the experiments in Phase 2 mentioned in Section 3.7.3 and shown in Figure 5.3 are fine tuned under the Iteration 4 configuration with a sampled validation on a single validation tile and are compared with Checkpoint A in Table 4.3. Experiment A loaded the Variant Z epoch 12 checkpoint and continued to train. Its IoU_{18} stayed below 0.02 through epoch 27 before rising, reaching its best value at epoch 85. This value was below Checkpoint A. In the next Experiment C, the Variant O epoch 36 checkpoint was loaded and trained with discriminative learning rates, described in Section 3.7.3. This experiment was able to detect moving vehicles at epoch 2, and it peaked at epoch 8, but then it started to drop. By the time training had reached epoch 32, its IoU_{18} had fallen to a tenth of the peak value while training loss continued to drop and validation loss had started to rise. The training had to be terminated. Neither strategy reached the from scratch baseline. Figure 4.1 plots the validation IoU_{18} of both experiments by epoch.

Table 4.3: Phase 2 against the from scratch baseline.

Run	Initial Weights	IoU_{18} at Reference Epoch	Relative to Checkpoint A
Iteration 4, Checkpoint A	random	0.3012 (131)	baseline
Experiment A	Variant Z epoch 12, head reset	0.1966 (85)	35 percent below
Experiment C	Variant O epoch 36, whole network	0.1723 (8)	43 percent below

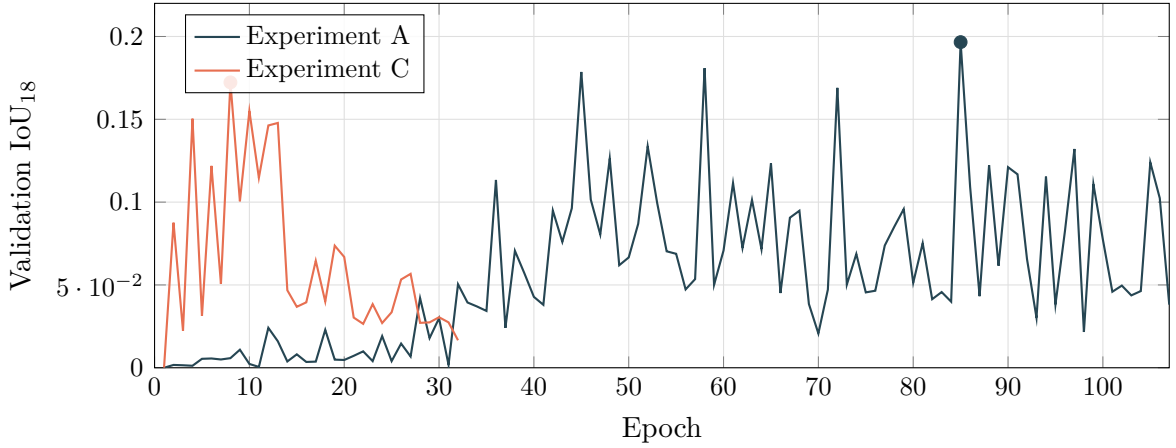


Figure 4.1: Validation IoU₁₈ per epoch for Experiment A (head reset) and Experiment C (whole network fine tuning). Experiment C is shown only through epoch 32, when training was terminated. The dots mark the reference epoch of Table 4.3.

4.5 RGB Integration Run

The RGB integration run in Section 3.7.4 addresses the road and ground identification confusion that Iteration 4 left unresolved. In this section and in Chapter 5, two checkpoints are compared, Checkpoint A that was produced from Iteration 4 at epoch 131, and Checkpoint B that was produced from the RGB integration run at epoch 34 of the sub run resumed from the epoch 83 checkpoint at $\eta_0 = 10^{-3}$, and Checkpoint B was the best checkpoint until Iteration 5 was introduced, reported in Table 4.1. Both have the same architecture but differ in the number of input features and number of training and validation tiles, making a per class comparison meaningful.

4.5.1 Training Instability

When weight decay and ground oversampling were introduced during a sub run, the training collapsed within its first fourteen epochs. High vegetation IoU was stuck at zero and IoU₁₈ started with a small value but in further epochs stalled. Removing weight decay did not resolve the issue, and later ground oversampling was disabled and a new training was initiated, which also collapsed. Since none of the sub runs worked, the investigation returned to the last known best epoch 83 checkpoint.

4.5.2 Learning Rate Overshoot

One more training was run with a higher learning rate of $\eta_0 = 10^{-2}$ using the epoch 83 best checkpoint, but that did not produce a better result. IoU₁₈ developed above Checkpoint A at epoch 3 but later kept fluctuating within a range with no class producing zero IoUs. A similar fluctuating range was observed for training and validation loss as well. As a correction, the learning rate was reduced to $\eta_0 = 10^{-3}$, with no weight decay and no ground oversampling. This produced the trajectory as shown in Figure 4.2, which covers 150 epochs under the sampled validation method

with the best epoch marked. IoU_{18} stayed in a stable band and achieved its best checkpoint at epoch 34 that was named Checkpoint B. Road produced a good result at roughly four times its Iteration 4 best value, and ground produced a result about five times its Iteration 4 best value. While the other classes improved, high vegetation stuck at roughly an eighth of what the Iteration 4 model reached. This vegetation related shortfall is discussed in Section 5.5.1, using the mean RGB values per class in the 16 bit range in Table 4.4, computed over every annotated point of tile 52, an RGB colored training tile.

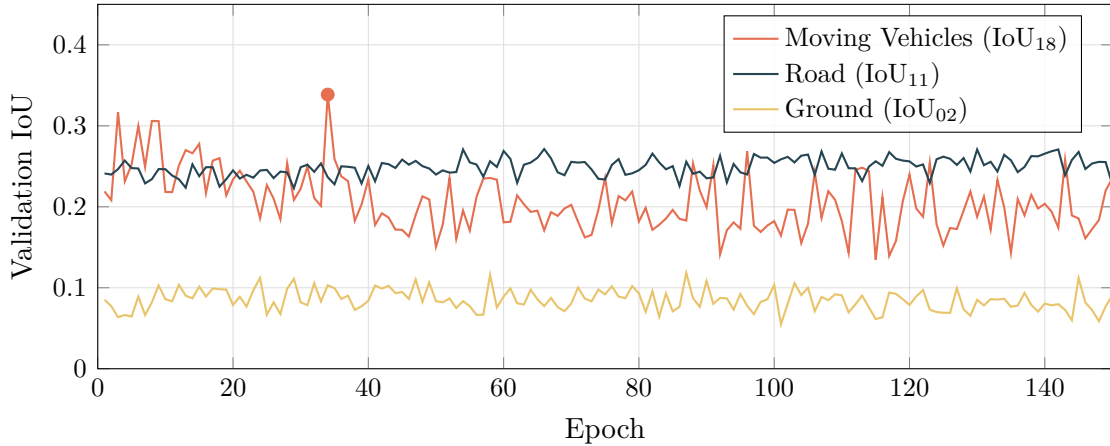


Figure 4.2: RGB integration run validation trajectory. The dot marks the best validation IoU_{18} epoch, Checkpoint B.

Table 4.4: Mean color per class on tile 52, in 16 bit RGB channel values, computed over every annotated point.

Class	Points	Mean R	Mean G	Mean B
High Vegetation	30,357,433	18,677	18,220	13,621
Low Vegetation	7,544,187	19,874	17,995	10,081
Ground	739,745	25,977	21,869	17,284
Road	36,257,354	25,412	25,152	24,067

4.5.3 Checkpoint B on Reserved Test Tile 51

Inference was run on the reserved test tile using Checkpoint B as a diagnostic under the protocol of Section 3.6.4, to see whether its validation values carried over to a tile it had never seen. The result was analyzed and reported in the second column of Table 4.7 in Section 4.6.4, where it is compared against the best checkpoint of the thesis. The road was recalled almost fully, but most of the ground points were predicted as road and vegetation points as ground. High vegetation and moving vehicles were recalled in well under a fifth of cases, with half of the true moving points predicted as road. The moving vehicle IoU on this reserved test tile was 0.0018 against a validation value of 0.3386 for the same checkpoint. Iteration 5 was built to close this gap, discussed in Section 5.1.3.

4.6 Iteration 5, Final Results

Iteration 5, described in Section 3.7.5, was running on the updated code, on the three dataset profiles of Figure 3.4 in parallel using the exhaustive validation as mentioned in Section 3.6.4. The Uncolored profile completed its 150 epoch schedule with its best epoch at 145. The Colored profile reached its best validation epoch at 130 and the Mixed profile at 149. The complete per epoch record of the three profiles is given in Appendix D. This section covers the validation trajectories, the best epoch per profile, the protocol of the reserved test tile evaluation, and the evaluation itself.

4.6.1 Validation Trajectories per Profile

Figure 4.3 plots validation mIoU across all the epochs for the three profiles, Figure 4.4 isolates the moving class, and Figure 4.5 shows the losses for the Colored profile without divergence between the training and validation loss, which gives no sign of overfitting in the loss.

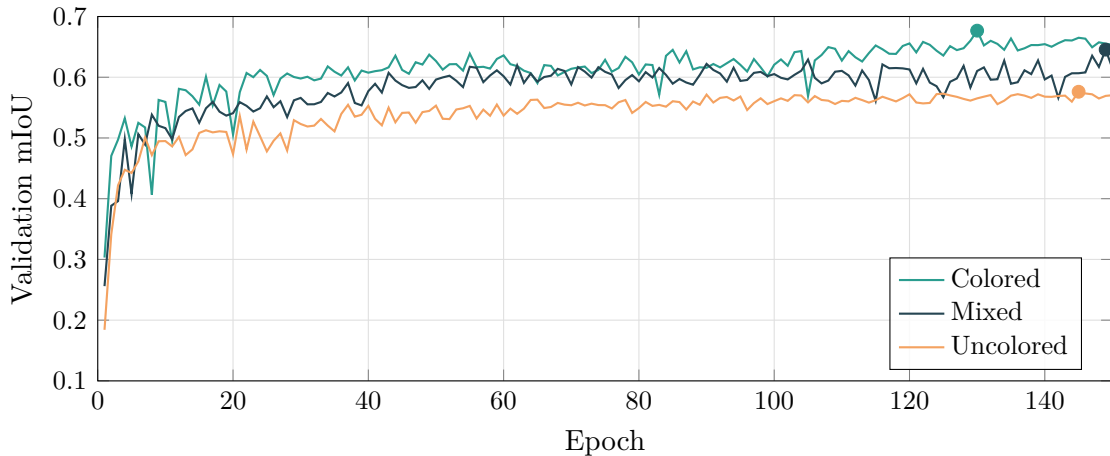


Figure 4.3: Iteration 5 validation mIoU by epoch. The dots mark the best validation mIoU epoch of each profile. Since Uncolored averages four classes and the other two profiles seven, their levels are not directly comparable.

4.6.2 Best Epoch per Profile

Table 4.5 shows the best validation mIoU results of the Iteration 5 training using three dataset profiles. Every profile’s mIoU is well above those of the earlier iterations, although the values are not directly comparable since validation changed from a sampled to an exhaustive scheme. N/A marks a class with no ground truth points, and the best value per class is in bold.

4.6.3 Reserved Test Tile Protocol

The final evaluation used the Iteration 5 Colored best checkpoint, reported in Table 4.5, and that choice was reserved before the test was run, based on validation

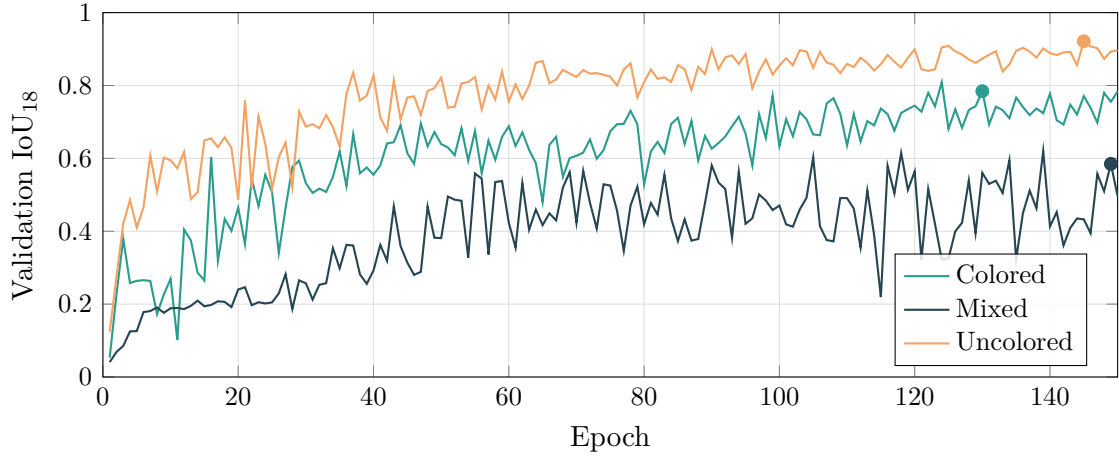


Figure 4.4: Iteration 5 validation IoU_{18} by epoch. The dots mark the best validation mIoU epoch of each profile.

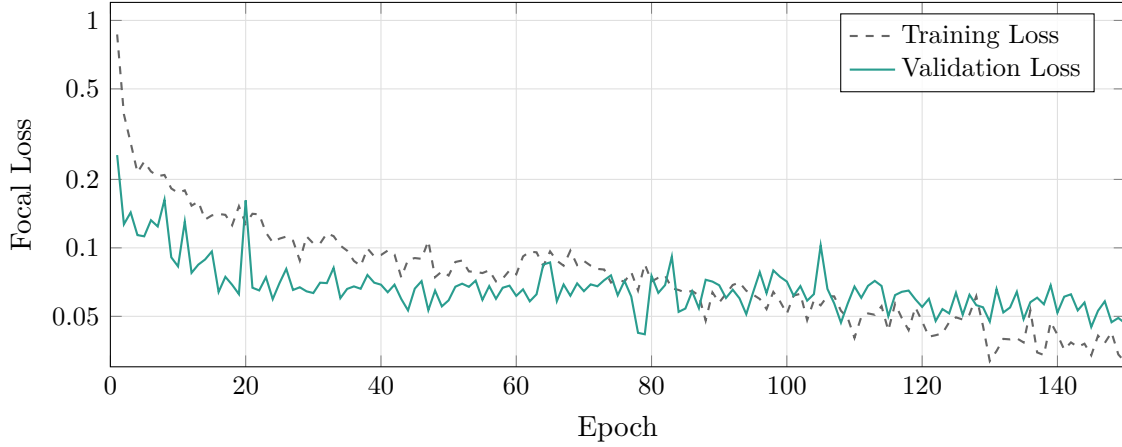


Figure 4.5: Iteration 5 focal loss of the Colored profile. The vertical axis is logarithmic.

Table 4.5: Iteration 5 best validation results per class.

Profile	Epoch	mIoU (Classes)	Ground	Low Veg	Med Veg	High Veg	Building	Road	Moving
Uncolored	145	0.5761 (4)	N/A	0.0000	N/A	0.4197	N/A	0.9633	0.9213
Mixed	149	0.6455 (7)	0.9469	0.9426	0.0014	0.8959	0.1658	0.9814	0.5848
Colored	130	0.6767 (7)	0.9845	0.9607	0.1142	0.8992	0.0000	0.9939	0.7844

scores only, described in Section 3.7.5. Inference with this checkpoint was performed on tile 51 exactly once, using the same full tile inference method used everywhere else, described in Section 3.6.4. Tile 51 was used for the Checkpoint B test as a diagnostic, described in Section 4.5.3, so that two results can be compared side by side. Unclassified points were excluded from IoU, as described in Section 2.2.8. Since medium vegetation does not appear on this test tile, it is marked N/A in Table 4.7 and left out of the average, meaning that the test score is an average across six classes, while the validation score averages across seven, as the numbers in brackets in the last rows of Table 4.1 show.

4.6.4 Confusion Matrix and IoU

Table 4.6 presents the confusion matrix of the Iteration 5 Colored best checkpoint on the reserved test tile, and Table 4.7 shows the per class IoU comparison of Checkpoint B and the Colored checkpoint, both derived from their confusion matrices with the unclassified row dropped, next to the validation values of Table 4.5. Rows are the true class and columns the predicted class, as defined in Section 2.2.8.1, counted in points, and the unclassified row holds the points with no ground truth label. The unclassified and building columns are zero throughout because the model never predicts either class on this tile.

Table 4.6: Confusion matrix of the Colored profile on tile 51, in point counts, with rows as the true class and columns as the predicted class.

	Unclass.	Ground	Low Veg	Med Veg	High Veg	Building	Road	Moving
Unclassified	0	155,540	1,945,919	42	5,359,552	0	2,995,720	117,861
Ground	0	9,543	19,024	0	662	0	425,675	208
Low Veg	0	530,307	8,762,553	127	389,807	0	294,794	10,491
Med Veg	0	0	0	0	0	0	0	0
High Veg	0	10,612	293,618	3	27,387,809	0	1,719	32,701
Building	0	478	622	0	84,064	0	4,173	5,821
Road	0	9,612	46,393	0	0	0	35,760,057	901
Moving	0	0	2,945	0	0	0	2,186	79,490

Table 4.7: Per class IoU on validation and on the reserved test tile 51.

Class	Checkpoint B Validation	Checkpoint B Diagnostic Tile 51	Colored Best Checkpoint Validation	Colored Best Checkpoint Final Tile 51
Ground	0.1030	0.0001	0.9845	0.0515
Low Vegetation	0.0131	0.1485	0.9607	0.8731
Medium Vegetation	N/A	N/A	0.1142	0.0000
High Vegetation	0.0488	0.1274	0.8992	0.9715
Building	0.0000	0.0000	0.0000	0.0000
Road	0.2365	0.8276	0.9939	0.9802
Moving Vehicles	0.3386	0.0018	0.7844	0.7430
mIoU	0.1057	0.1842	0.6767	0.5710

The final checkpoint in Table 4.7 has improved with respect to Checkpoint B for every class present in the test tile except building, which stays at zero, marking a clear improvement for the vegetation and moving classes. Comparing the columns in the table shows that the final checkpoint carries road, low vegetation and high vegetation values almost unchanged from validation to the unseen tile, while ground and moving vehicles still underperform. To understand the root cause behind them, a confusion matrix was produced which locates both the gaps, shown in Table 4.6 and Figure 4.6. Ground is predicted as road in 93.53 percent of cases, down from 98.4 percent under Checkpoint B, but ground is still mostly merged with road. Low vegetation is predicted as ground in 5.31 percent of cases, so the confusion shown by Checkpoint B between vegetation and ground has been largely resolved. High

vegetation shows one clear diagonal value for recall. Moving vehicles are recalled in 93.94 percent of cases. This shows that the confusion of moving vehicles with road has almost disappeared. The remaining gap of the moving class is in precision, discussed in Section 4.6.7. Building is never predicted correctly in this tile, and most of its true points are assigned to high vegetation.

4.6.5 Row Normalized Confusion Heatmap

Figure 4.6 shows Table 4.6 with every row divided by its total, as defined in Section 2.2.8.1, and with each cell given in percent and shaded so that a darker cell holds a larger share. Each row therefore shows where the points of one true class were sent, and the diagonal gives the recall of that class. The unclassified row shows how the points without a ground truth label were distributed over the predicted classes. The unclassified column is zero because the network never predicts that class. The medium vegetation row is marked N/A because the tile holds no medium vegetation points.

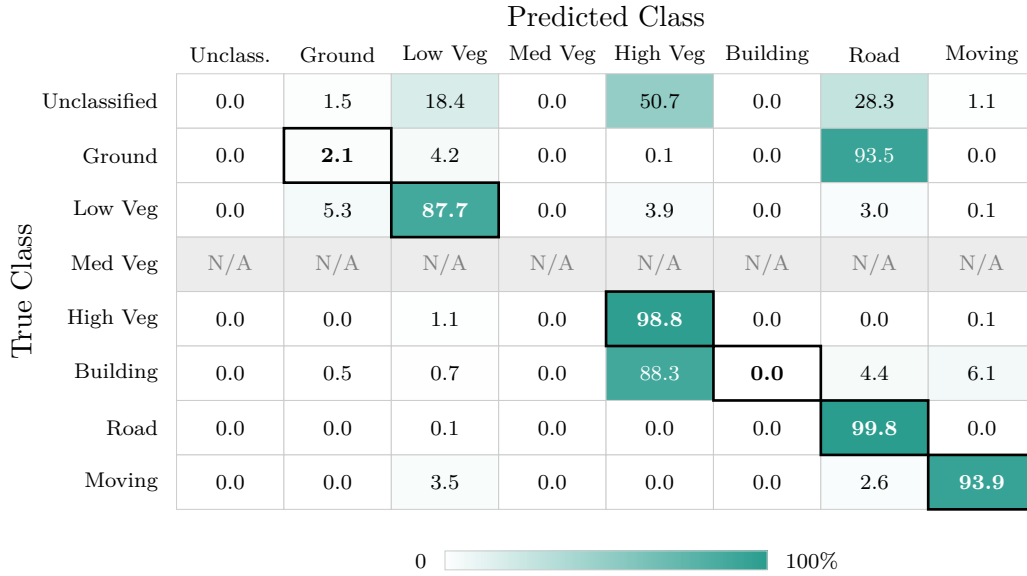


Figure 4.6: Row normalized confusion heatmap on tile 51.

4.6.6 Column Normalized Confusion Heatmap

Figure 4.7 shows Table 4.6 with every column divided by its total, as defined in Section 2.2.8.1, and shaded in the same way as Figure 4.6. Each column therefore shows what the points predicted as one class truly were, and the diagonal gives the precision of that class. The unclassified row is kept in the totals, so a share of every column is made up of points without a ground truth label. The unclassified and building columns are marked N/A because the network predicted no points as these classes on this tile.

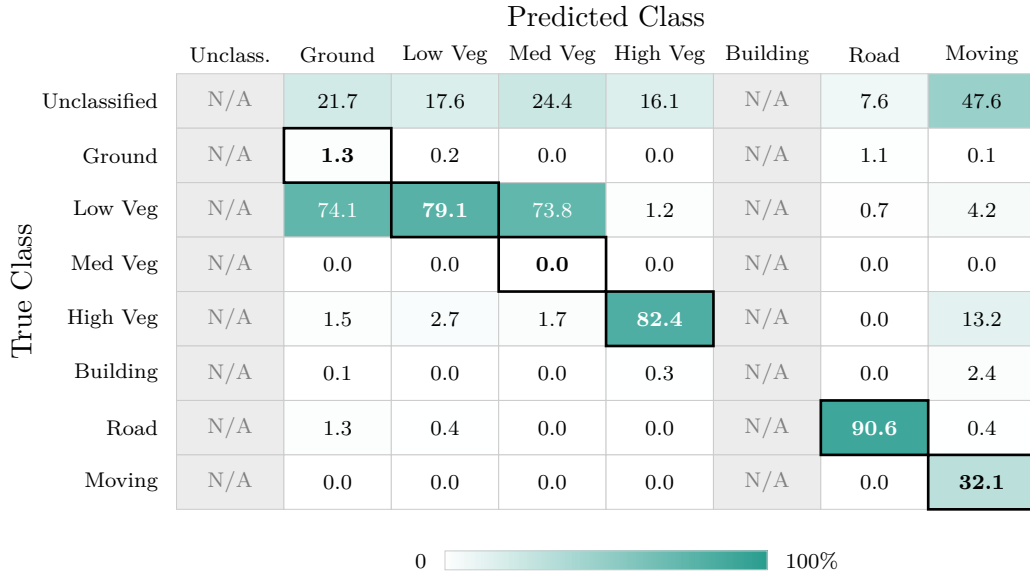


Figure 4.7: Column normalized confusion heatmap on tile 51.

4.6.7 Recall and Precision

The confusion matrix in Table 4.6 gives recall per true class when read by row, shown in Figure 4.6, and read by column gives precision per predicted class, shown in Figure 4.7. The unclassified row shows the share of each class’s predictions that landed on points without a ground truth label. When read together, both readings show an asymmetric failure mode for the moving vehicle class. Recall is high, so the network rarely misses a true moving vehicle, but precision is much lower. Of all points predicted as moving, only 32.12 percent are moving vehicles when the unclassified points are counted as false positives. After the removal of unclassified points, the same becomes 61.33 percent. Close to half of the moving predictions fall on points with no ground truth label and about one eighth on high vegetation. The moving vehicles are over predicted on unseen data rather than missed, and the interpretation of the origin of those points is given in Section 5.2.

4.6.8 Reserved Test Tile Render

Figure 4.8 shows the full tile inference result of the Iteration 5 Colored best checkpoint on the reserved test tile, rendered in CloudCompare [28] with the color scheme of Figure 3.3, shown alongside it for direct comparison. Road is recovered as a continuous surface rather than as the annotated patches of Figure 3.3, and the moving vehicle distortion is recovered, which includes its trailing edge, and this is also the recall result of Table 4.6.

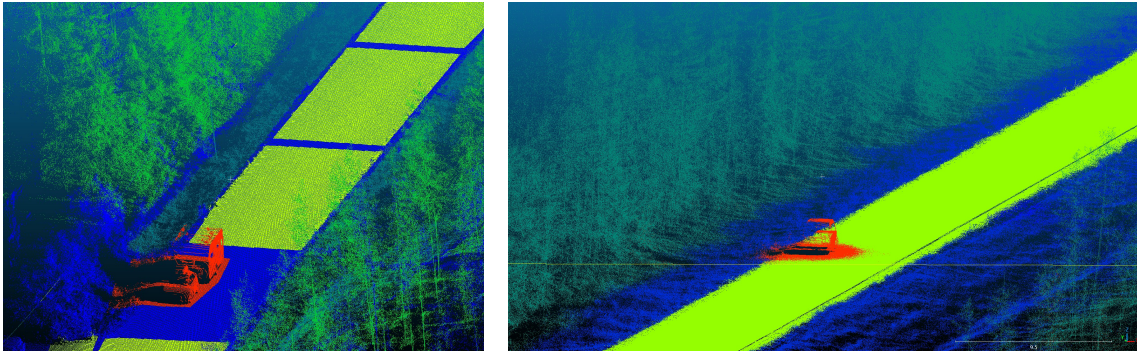


Figure 4.8: A training tile annotated under the protocol of Figure 3.3 (left), shown next to the full tile inference on the reserved test tile 51 (right) for comparison.

5

Discussion

Here, we discuss the results obtained in Chapter 4. Section 5.1 follows the five iterations as a single progression and lists what changed from the earlier best result to Iteration 5. Sections 5.2, 5.3 and 5.4 answer the three research questions of Section 1.3. Figure 5.2 maps each question to the runs that address it and to the sections that answer it. Section 5.5 discusses the road and ground question, Section 5.6 brings the final checkpoint and published results together, and Section 5.7 lists the limitations.

5.1 Iterations Together

5.1.1 Iteration Progression

As seen in Table 4.1, the first three iterations confirmed that the moving vehicle class can be learned by a simple network, with the loss weight bracketed from both sides, meaning pushed too high and too low in turn to find the usable range, as reported in Section 4.1. Changing several things, among them the architecture from V1 to V2, which replaced the decoder’s repeat interleave upsampling that duplicated point features without assigning them to a location, as described in Section 3.5.1, and the tile set from six to eleven tiles, improved the results. Iteration 1 is the right comparator for that gain, since the weight of Iteration 3 was deliberately conservative and Iteration 2 over predicted on a full tile. Neither transfer strategy, head reset or whole network fine tuning as discussed in Section 5.4, improved on Checkpoint A, while the RGB integration run, Checkpoint B, which tested co-registered RGB color as discussed in Section 5.5.1, improved slightly on validation, but its diagnostic on the reserved test tile showed that the validation value did not carry over to unseen data. After the errors, present in every run through Checkpoint B, were fixed, Iteration 5 roughly doubled the moving vehicle IoU on validation (Table 4.1), with the unseen data result discussed in Section 5.2. All these are traced to individual changes identified in Section 3.7, and the step from Checkpoint B to Iteration 5 is discussed in Section 5.1.3.

5.1.2 Data Volume

Between Iterations 1 to 3 and Iteration 4, the architecture and the tile set changed together. More tiles were added to increase the number of moving vehicle points, and the V2 architecture removed a structural limitation that no amount of data could have compensated for, as described in Section 3.5.1. Both contributed, and this thesis cannot measure which one mattered more.

Figure 5.1 plots the validation IoU_{18} at the reference epoch of Iteration 1, Iteration 4, the RGB integration run and the three Iteration 5 profiles against the number of training tiles. The runs before Iteration 5 use sampled validation and are drawn as orange circles joined by a dashed line, and the three Iteration 5 profiles use exhaustive validation and are drawn as teal squares at their best validation epoch. Doubling the tile set from Iteration 4 to the RGB integration run gave only a modest gain, and the Iteration 5 points break the pattern. The Mixed profile uses the same tiles as the RGB integration run yet reaches a higher value, and Colored, with fewer tiles than Mixed, is higher again. The Iteration 5 points were measured under exhaustive validation on the corrected code.

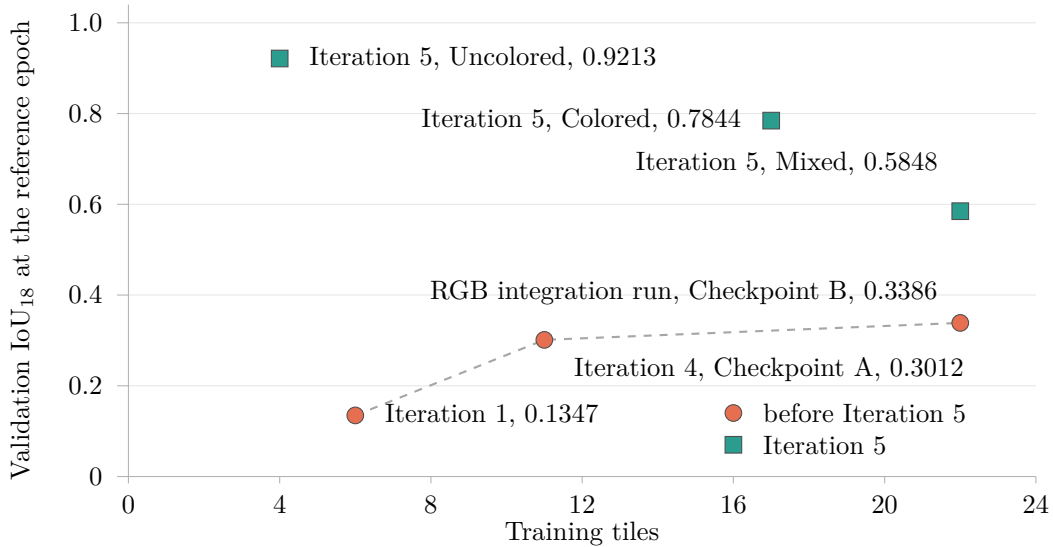


Figure 5.1: Validation IoU_{18} at the reference epoch against the number of training tiles.

5.1.3 Checkpoint B to Iteration 5

The lower learning rate was used when the model was trained from scratch. The validation scheme was upgraded from sampled to exhaustive. The data was organized into three profiles, namely Colored, Uncolored and Mixed. The earlier validation scored a random subset of windows as explained in Section 2.2.7, so the validation values of every run before Iteration 5 were measured on a shifting sample, which is consistent with the epoch to epoch volatility recorded as early as Iteration 4 and with the failure of Checkpoint B’s validation value to carry over to the test tile. The earlier focal loss [17] also carried two implementation defects that diluted its effective class weighting, described in Section 3.7.5.

Both Checkpoint B and the Iteration 5 Colored checkpoint were evaluated on reserved test tile 51 by full tile inference, giving the cleanest comparison since it does not depend on the validation scheme, so the two tile 51 columns of Table 4.7 (Section 4.6.4) are directly comparable. The validation comparison mixes a real improvement with a change of measurement, since exhaustive validation uses every annotated point where sampled validation uses a subset, making the size of the

validation gain indicative rather than exact.

As the four changes were introduced together, a matched ablation should be performed to understand the cause of the difference, and this is listed as future work in Section 6.2.

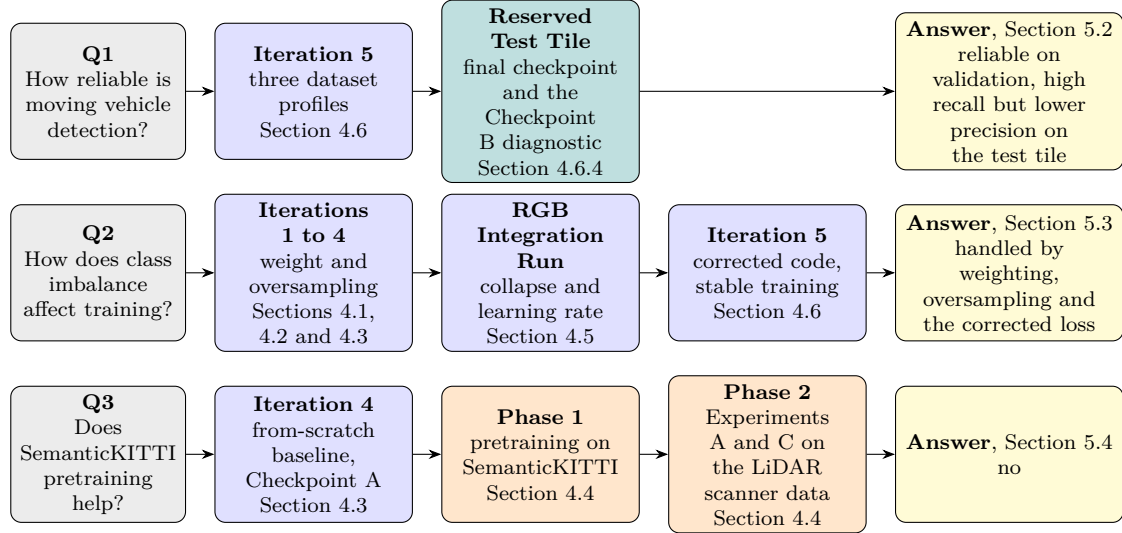


Figure 5.2: Research questions mapped to runs and answers.

5.2 Q1, Moving Vehicle Detection

The answer to Q1, concerning the reliability of the moving vehicle detection approach, is divided into two parts. The moving vehicles get detected on recall but not fully on precision. On recall, the true moving points get detected on a tile the model never saw, as shown in Table 4.6 and Figure 4.6. The confusion with road, present in earlier runs, has almost disappeared. However, on precision, a large portion of the points predicted as moving are not moving vehicles, as shown in Figure 4.7, causing the moving vehicle IoU on the reserved test tile to be 0.7430 while the validation IoU stands at 0.7844 for the same checkpoint. The test tile IoU falls behind the validation IoU for Checkpoint B, the RGB integration run, but the comparison of the two gaps must not be over read since the RGB integration run used sampled validation while Iteration 5 used exhaustive validation. The only like for like comparison in this thesis, Checkpoint B against the Colored checkpoint, Iteration 5, on tile 51, shows an improvement, as reported in Table 4.7.

For moving vehicles, many of the predictions were false positives. Where the false positives come from matters for how the result should be used. Close to half of the moving predictions fall on points with no ground truth label, discussed in Section 4.6.7. The annotation protocol of Section 3.2.2 says that moving vehicles alone were annotated exhaustively, so unlabeled points are more likely to be unannotated ground, vegetation or road, and the false positives of Section 4.6.7 are most likely genuine, and the precision figure that counts them is the relevant one. About one eighth of the moving predictions land on high vegetation, the class whose distortion-like fragments most resemble a moving vehicle in shape.

Ground behaves differently, best resolved on validation yet near zero on the test tile, because on reserved test tile 51 almost all true ground is predicted as road. This road and ground confusion is a known weakness, discussed as part of the earlier spectral ambiguity in Section 5.5. It does not explain why the confusion is worse on this unseen tile than on validation, and that difference is not established by the available evidence. For the industrial use case the asymmetry between high recall and lower precision is workable. A detector that rarely misses a moving vehicle but over predicts it can serve as a first stage whose output is reviewed. Closing the precision gap on unseen data is the immediate next step and is listed in Section 6.2.

5.3 Q2, Class Imbalance

5.3.1 Weighting Iterations

Iterations 1 to 3 were an empirical search to find the right weight for the moving vehicle class to keep the training run stable under the fixed focal loss. The suitable range was between 1.5 and 3.0, far below the 10 that inverse frequency weighting, scaling each class’s weight by how rare it is, would suggest, as reported in Table 4.2. The cause is the class weight failure described in Section 4.1. At a weight of 3.0, missing a moving vehicle point is so costly that predicting most points as moving becomes cheaper for the loss on a full tile.

When a weight of 2.5 was chosen, it produced a stable result to detect the moving vehicle class in every later run. The better result of the later iterations came from the network design and the data along with the weight tuning. Two practical lessons follow. Validation IoU alone did not show the over prediction, while full tile inference did, since IoU is computed only over annotated points and so does not count moving predictions that land on unannotated points. It is therefore important to perform full tile inference as a part of evaluation. The second lesson is that the road oversampling failure in Section 4.2 shows that when batch gradients are averaged, a class already well represented in a batch can also overwhelm a rare class once it is pushed even higher. Oversampling probabilities should therefore be set by carefully studying the class distribution in a window, not from the total percentage at the tile level.

5.3.2 RGB Integration Run Instability

The RGB integration run exposed two further failure modes, described in Sections 4.5.1 and 4.5.2. The first failure, described in Section 4.5.1, happened when weight decay and ground oversampling were introduced together. The working hypothesis is that weight decay applied to every parameter stopped the network from forming confident class boundaries, but this remains unconfirmed. The second failure has a clear cause. Resuming an already well trained checkpoint at the from scratch learning rate made the network reach a good result early and then lose it, while the training loss kept falling. This is the sign of a learning rate too large for a model already close to a good solution, so the optimizer steps past the best point instead of settling into it. A from scratch rate suits a randomly initialized model, which starts far from any good solution, unlike the resumed checkpoint above, whose

corrected rate gave a steady trajectory, shown in Figure 4.2.

5.3.3 Iteration 5 Stability

Iteration 5 shows none of the failure modes above. The Colored loss curves fall together without divergence, shown in Figure 4.5. The answer to Q2, concerning how class imbalance affects training, is therefore a concrete configuration, not a general principle. The configuration of Section 3.7.5, with the weights of Equation 3.2, keeps training stable, without the collapse or extreme over prediction seen in the earlier runs. Full tile inference remains a required check alongside validation, as the weighting iterations established.

5.4 Q3, Transfer Learning

Neither Phase 2 strategy, head reset (Experiment A) or keeping the pretrained head with whole network fine tuning (Experiment C), approached the from-scratch baseline, reported in Table 4.3, despite the pretrained model’s strong in-domain performance on SemanticKITTI [5]. The two strategies failed in different ways, and the gap between the two sensors most likely accounts for the rest. Figure 5.3 recalls the design of the study.

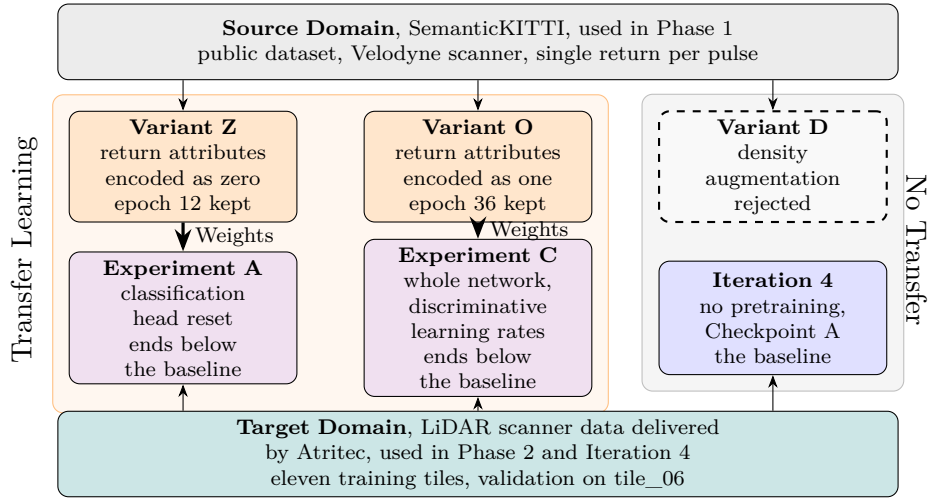


Figure 5.3: Design of the transfer study.

5.4.1 Experiment A, Head Reset

The head reset runs against a common assumption in transfer learning, that the final layer is task-specific and the encoder is the reusable part. Here the source and target datasets shared the same eight class labeling scheme by design, so the head’s learned mapping from features to moving vehicles was the most transferable part of the model. Discarding it left the network detecting no moving vehicles for roughly the first 28 epochs, and the run never recovered enough to beat the baseline, as reported in Section 4.4. When source and target tasks use matching class definitions,

the head should be kept.

5.4.2 Experiment C, Overfitting

Experiment C, which fine tuned the whole network with discriminative learning rates, kept the head. With the head trained fifty times faster than the encoder, the aggressively trained head overwrote its pretrained mapping with patterns fitted to the training set, a form of catastrophic forgetting [27]. The run needed stronger controls on the head, a lower rate, weight decay or early stopping, and the small parameter mismatch of Table A.1 may have made it worse. That it outperformed Experiment A from epoch 2 to epoch 8, as reported in Section 4.4, shows the idea initially worked, and that its failure set in more slowly than Experiment A’s, though this attempt broke down too.

5.4.3 Domain Gap and Augmentation

Behind both the head reset failure of Section 5.4.1 and the overfitting failure of Section 5.4.2 most likely sits the gap between the two sensors, discussed in Section 2.2.6. The Velodyne scanner behind SemanticKITTI [3], [5] records a single return per pulse, while the LiDAR scanner records multiple. As Section 4.4 reports, filling the missing return attributes with zero, the encoding used in Variant Z, leaves two of the seven input channels dead during pretraining, so the corrected encoding of Variant O, which encoded the return attributes as one instead, was adopted. Variant D showed that closing the density gap by adding interpolated points does not work, since interpolating at the midpoint of random pairs creates physically impossible points, and the resulting label noise sets a hard floor under the training loss, reported in Table 4.1. Any further density augmentation should be sub-sampling only, removing points from the denser data. The answer to Q3, whether SemanticKITTI pretraining helps, is therefore a clear no for the strategies tried, with one caveat. The study was not repeated under the corrected code of Iteration 5, which is left open in Section 5.7 and listed as future work in Section 6.2.

5.5 Road, Ground and Color

Road and ground were the classes with the lowest IoU in Iteration 4, with a road IoU an order of magnitude below the same architecture’s road IoU on SemanticKITTI in Phase 1. Two possible causes were considered at the time. The overlapping 1550 nm reflectance of asphalt and soil removes the spectral separation between them, and the helical scan pattern lacks the ring edge discontinuities that outline the road geometrically in Velodyne data. High vegetation, distinctive in shape, was learned well by the same model. The remedy proposed at the time was co-registered RGB, to restore the spectral separation between road and ground.

5.5.1 RGB Hypothesis

The RGB integration run tested this hypothesis and partly confirmed it. At Checkpoint B the road IoU rose to about four times and the ground IoU to about five times their Iteration 4 values, shown in Figure 4.2, both consistent with the reflectance overlap diagnosis, though far short of the values reached in Iteration 5. A further finding was not anticipated, that high vegetation fell to a fraction of its Iteration 4 value in this run. The color statistics of Table 4.4 show that road and ground differ clearly in mean blue and had no separating signal before color, so color gave them one, while high and low vegetation have similar mean color in every channel, so color adds nothing for that pair. The statistics do not show that color caused the drop in high vegetation, and this thesis did not isolate its cause. Other candidates are the two extra validation tiles without vegetation labels, the training instability of this run and the code errors corrected before Iteration 5. The drop did not persist, since the Iteration 5 Colored checkpoint, with the same ten input features, reaches a high vegetation IoU of 0.8935 on validation, reported in Table 4.5. Whether color affects vegetation could be tested by the matched ablation of Section 6.2.

5.5.2 Iteration 5, Road and Ground

Iteration 5 changes the findings of the previous section. Colored and Mixed datasets reach road IoU above 0.98 on validation, reported in Table 4.5, consistent with color restoring the spectral separation, but Uncolored also reaches 0.96, close enough that color cannot be the primary explanation for the improvement over Checkpoint B. The more likely causes are the changes of Section 5.1.3 (the code corrections, the exhaustive validation scheme and the new data profiles), which cannot be separated with the runs available. The validation scheme change contributes to the size of the measured gain. Color still contributes. Colored dataset road IoU exceeds Uncolored's, and Colored resolves low vegetation, a class on which Uncolored scores zero. Ground cannot be compared across profiles, because the Uncolored single validation tile contains no ground labels.

5.6 Relation to Published Results

Table 5.1 places the final checkpoint beside the published RandLA-Net result on SemanticKITTI [6]. The validation column is the Colored profile at its best epoch, reported in Table 4.5, and the test column is the same checkpoint on the reserved test tile 51, reported in Table 4.7. The published values are the RandLA-Net single scan results, with terrain, vegetation and road used as the nearest equivalents of ground, high vegetation and road. On the validation tiles this thesis exceeds the published road and vegetation values, consistent with the distinctive shape of vegetation discussed in Section 5.5. Ground and moving vehicles tell a different story, discussed in Section 5.2, since both fall from validation to the reserved test tile. The published benchmark reports no comparable moving vehicle class. The two datasets differ in sensor, point density and annotation protocol, and the comparison is offered as one further way to see where the validation to test gap of Section 5.2 sits relative

to the wider field.

Table 5.1: Per class IoU against the published RandLA-Net result.

Class	This Thesis, Validation	This Thesis, Reserved Test Tile	SemanticKITTI, Published
Ground	0.9800	0.0095	0.668
High Vegetation	0.8935	0.9712	0.814
Road	0.9905	0.9785	0.907
Moving Vehicles	0.7111	0.5899	not reported

5.7 Limitations

1. Medium vegetation and buildings sit at or near zero IoU across every Iteration 5 profile with ground truth for them, reported in Table 4.5, consistent with the test tile confusion matrix of Table 4.7. The likely cause is the small share of these classes in the annotated data, discussed in Section 3.2.2. The network sees too few examples of either class to learn a stable representation, and the validation and test tiles hold too few of their points to score them reliably.
2. Windows of 65,536 points can truncate same-direction distortions exceeding the window’s spatial extent, described in Section 2.1.2. Larger windows were not used because they would not fit in memory at the decimation granularity this thesis required.
3. Transfer learning is untested against Iteration 5. No Phase 2 strategy beat the from-scratch baseline available at the time, discussed in Section 5.4, and the study was never repeated under Iteration 5’s corrected code, because of limits on time and storage, so it remains unknown whether the failure modes identified would recur.
4. The final result on unseen data rests on one reserved test tile, tile 51, so its variation across other tiles is unknown. Evaluation on several further reserved test tiles, listed in Section 6.2, would show how stable the result is.

6

Conclusion and Future Work

6.1 Conclusions

This thesis aims to automate the detection of moving vehicles in LiDAR road survey point clouds, and the result is a pipeline that runs end-to-end, is documented from the LAS file, the format that stores the point cloud, to the classified output, and has been evaluated on a reserved test tile. Removing the detected moving vehicle points is a separate step that was not evaluated. The work was carried out as five iterations, each built from the failure analysis of the one before. Each research question is answered in turn, and Figure 5.2 maps each question to the runs behind its answer. The answers rest on a single, limited study, and a larger study would give more definite answers.

Q1, how reliable moving vehicle detection is on a proprietary LiDAR scanner dataset. As discussed in Section 5.2, moving vehicle detection is reliable on the validation tiles and usable on unseen data as a first stage whose flagged points a person checks before removal. The final checkpoint, trained on the corrected code, improves on the earlier checkpoints. On unseen data it rarely misses a moving vehicle point but over predicts the class. The result rests on a single reserved test tile. Reducing the over prediction and increasing the precision are the next steps. The evidence is in Section 5.2.

Q2, to what extent class imbalance affects training and how it can be handled without over prediction. The severe imbalance between moving vehicles and background classes affects both training stability and detection accuracy, and its extent was not measured, as the failure modes analyzed in Section 5.3 show. A stable configuration of loss design, class weighting and targeted sampling avoids the collapse and the extreme over prediction, and full tile inference is a required check alongside validation, since validation metrics alone did not reveal the over prediction. The final checkpoint still over predicts the class on the reserved test tile, so the imbalance is handled only in part. The evidence is in Section 5.3.

Q3, whether pretraining on SemanticKITTI improves performance on the proprietary LiDAR scanner dataset. It did not improve moving vehicle detection. The transfer study, whose design is shown in Figure 5.3, tried two fine tuning strategies, and both finished below the from scratch baseline, one by discarding the transferable part of the model and the other by overfitting the classification head. The rest of the shortfall is most likely due to differences in scan pattern, point density and return recording between the two sensors. The study was carried out on the earlier code and was not repeated on the corrected code because of limits on time and storage,

so the answer holds only for the strategies tried and the repeat is left as future work. The evidence is in Section 5.4.

6.2 Future Work

There are several possible directions for future research.

First, the precision on unseen data needs a fix. Two classes follow from the confusion matrix as mentioned in Section 4.6.7. Points with no ground truth label absorb a large share of the moving vehicle predictions, and annotating the unlabeled points of the test tile would confirm how many of these predictions are genuine errors. High vegetation is the largest identifiable source of false positives, and adding a geometry based post filter on the predicted moving points could address it.

Second, the result on unseen data rests on one reserved test tile, tile 51. Several further reserved test tiles from different regions would show how well the model has learned the task and how much it over predicts at new locations. The confusion matrix of each tile would be compared with that of tile 51, discussed in Section 4.6.7. Third, the pipeline should also be evaluated by cross validation across several validation sets instead of a fixed validation set of tiles. The network would be trained several times, each time with a different group of tiles for validation and the rest for training, and the performance compared across the models. Consistent performance would show a stable representation of the task. Large variation would show that the model is sensitive to the scenes it is checked against.

Fourth, the effect of color on the results is still open. A matched ablation would train Colored and Uncolored on matched tile sets and matched validation tiles, differing only in whether the color channels are populated. This would isolate the contribution of color to road, ground and vegetation and make the moving vehicle values of the two profiles comparable.

Finally, repeating the transfer study, Phase 1’s pretraining on SemanticKITTI and Phase 2’s two fine tuning strategies, described in Section 4.4, with the corrected code, as noted above, could bring an improvement, but it is unclear whether either strategy’s shortfall against the baseline would recur. A repeat also needs careful handling, since extracting SemanticKITTI produces tens of thousands of files and the storage allocation limits their number in HPC.

References

- [1] G. Petrie and C. K. Toth, “Terrestrial laser scanners,” in *Topographic Laser Ranging and Scanning: Principles and Processing*, J. Shan and C. K. Toth, Eds., 2nd ed., CRC Press, 2018, pp. 29–88. DOI: 10.1201/9781315154381-2
- [2] H. Durrant-Whyte and T. Bailey, “Simultaneous localization and mapping: Part I,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99–110, 2006. DOI: 10.1109/MRA.2006.1638022
- [3] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, “Vision meets robotics: The KITTI dataset,” *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [4] Y. Guo, H. Wang, Q. Hu, H. Liu, L. Liu, and M. Bennamoun, “Deep learning for 3D point clouds: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 12, pp. 4338–4364, 2021. DOI: 10.1109/TPAMI.2020.3005434
- [5] J. Behley et al., “SemanticKITTI: A dataset for semantic scene understanding of lidar sequences,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 9296–9306.
- [6] Q. Hu et al., “RandLA-Net: Efficient semantic segmentation of large-scale point clouds,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 108–11 117.
- [7] M. Lehtomäki et al., “Object classification and recognition from mobile laser scanning point clouds in a road environment,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 54, no. 2, pp. 1226–1239, 2016. DOI: 10.1109/TGRS.2015.2476502
- [8] A. Dewan, T. Caselitz, G. D. Tipaldi, and W. Burgard, “Motion-based detection and tracking in 3D LiDAR scans,” in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 4508–4513. DOI: 10.1109/ICRA.2016.7487649
- [9] C. Mertz et al., “Moving object detection with laser scanners,” *Journal of Field Robotics*, vol. 30, no. 1, pp. 17–43, 2013. DOI: 10.1002/rob.21430
- [10] H. Guan, J. Li, Y. Yu, C. Wang, M. Chapman, and B. Yang, “Using mobile laser scanning data for automated extraction of road markings,” *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 87, pp. 93–107, 2014. DOI: 10.1016/j.isprsjprs.2013.11.005
- [11] S. A. Bello, S. Yu, C. Wang, J. M. Adam, and J. Li, “Review: Deep learning on 3D point clouds,” *Remote Sensing*, vol. 12, no. 11, p. 1729, 2020. DOI: 10.3390/rs12111729

- [12] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “PointNet: Deep learning on point sets for 3D classification and segmentation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 652–660.
- [13] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “PointNet++: Deep hierarchical feature learning on point sets in a metric space,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [14] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas, “KPConv: Flexible and deformable convolution for point clouds,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019, pp. 6411–6420.
- [15] W. Xiao, B. Vallet, K. Schindler, and N. Paparoditis, “Street-side vehicle detection, classification and change detection using mobile laser scanning data,” *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 114, pp. 166–178, 2016. DOI: 10.1016/j.isprsjprs.2016.02.007
- [16] J. Schauer and A. Nüchter, “The peopleremover—removing dynamic objects from 3-D point cloud data by traversing a voxel occupancy grid,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1679–1686, 2018. DOI: 10.1109/LRA.2018.2801797
- [17] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, “Focal loss for dense object detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [18] K. Weiss, T. M. Khoshgoftaar, and D. Wang, “A survey of transfer learning,” *Journal of Big Data*, vol. 3, no. 1, p. 9, 2016.
- [19] C. Qin, H. You, L. Wang, C.-C. J. Kuo, and Y. Fu, “Pointdan: A multi-scale 3D domain adaption network for point cloud representation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [20] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 of the European Parliament and of the Council,” *Official Journal of the European Union*, vol. L 119, pp. 1–88, 2016.
- [21] American Society for Photogrammetry and Remote Sensing, *LAS specification, version 1.4, R15*, Bethesda, Maryland, 2019.
- [22] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90
- [23] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [24] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, ISBN: 978-0-262-03561-3.
- [25] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [26] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 37, PMLR, 2015, pp. 448–456.

- [27] J. Kirkpatrick et al., “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [28] CloudCompare, *CloudCompare (version 2.13) [GPL software]*, 2023. [Online]. Available: <http://www.cloudcompare.org/>
- [29] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

A

Hyperparameter Record

Table A.1 lists the hyperparameters of the principal runs. The Iteration 5 column applies to all three profiles. The IoU_{18} row gives the value at the reference epoch of each run, Checkpoint A for Iteration 4, the epoch 36 source checkpoint for Phase 1, the best IoU_{18} epoch for Experiment C, Checkpoint B for the RGB integration run and epoch 130 of Colored for Iteration 5. Parameter counts are as logged by each run and have not been reconciled across runs. Epoch times are approximate wall-clock times per epoch on the assigned node. The Iteration 5 value is that of the Colored profile. The Mixed profile takes about 91 minutes and the Uncolored profile about 26 minutes.

Table A.1: Hyperparameters of the principal runs.

Parameter	Iteration 4	Phase 1 (O)	Exp. C	RGB Run	Iteration 5
Architecture	V2	V2	V2	V2	V2
Parameters (Logged)	85,925	190,387	186,993	187,338	187,338
Input Features	7	7	7	10	10
Classes	8	8	8	8	8
K Neighbors	32	32	32	32	32
Decimation	4	4	4	4	4
Window Points	65,536	65,536	65,536	65,536	65,536
Batch Size	8	8	8	8	8
Epochs (Run)	150	66 logged	32, terminated	150 after resumption	150 scheduled
η_0	0.005	0.010	head 0.005, encoder 0.0001	0.001, resumed	0.001
Learning Rate Decay	0.98	0.98	0.98	0.98	0.98
w_{18}	2.5	3.0	2.5	2.5	2.5
w_{11}	1.2	1.0	1.2	1.2	1.2
p_{18}	0.50	0.30	0.50	0.50	0.50
p_{road}	0	0	0	0	0
Focal γ	2.0	2.0	2.0	2.0	2.0
Gradient Clipping Threshold	1.0	1.0	1.0	1.0	1.0
Head Reset	—	—	no	—	—
Validation Scheme	sampled	sampled	sampled	sampled	exhaustive
Validation Tiles	tile_06	sequences 08, 10	tile_06	24, 25, tile_06	24, 25, tile_06, one tile for Uncolored
IoU_{18} at Reference Epoch	0.3012	0.4655 (SemanticKITTI)	0.1723	0.3386	0.7844 (Colored)
Epoch Time	≈ 18 min	≈ 36 min	≈ 71 min	≈ 26 min	≈ 89 min

B

Core Algorithms

This appendix collects the pseudocode for the three top level procedures of the pipeline described in Chapter 3, namely the per tile export into training arrays, Algorithm 1, the network training loop, Algorithm 2, and the full tile inference procedure, Algorithm 3. Supporting subroutines these procedures call, the chunked nearest neighbor search, the chunked upsampling step, the network’s forward pass, and the oversampling window draw, are described in prose alongside the network architecture and training strategy in Chapter 3 rather than repeated as separate pseudocode.

Algorithm 1 Export procedure into NPY, per tile.

Require: Annotated LAS tile T , a single input file containing raw point measurements and their manually assigned classification codes. Split assignment $s(T)$, a function that determines whether tile T belongs to the training, validation, or test set.

- 1: Read $x, y, z, i, n_{\text{ret}}, N_{\text{ret}}, R, G, B, \text{cls}$ from T , the raw coordinates, intensity, return attributes, color channels, and classification code of every point in the tile
 - 2: $r \leftarrow \sqrt{x^2 + y^2 + z^2}$
 - 3: $r_{\text{norm}} \leftarrow r / \max(r)$ ▷ range computed before centring
 - 4: $x' \leftarrow x - \bar{x}$
 - 5: $y' \leftarrow y - \bar{y}$
 - 6: $z' \leftarrow z - \bar{z}$
 - 7: $i_{\text{norm}} \leftarrow i / \max(i)$
 - 8: $R_{\text{norm}}, G_{\text{norm}}, B_{\text{norm}} \leftarrow$ normalize color channels to $[0, 1]$
 - 9: $P \leftarrow \text{stack}[x', y', z', i_{\text{norm}}, r_{\text{norm}}, n_{\text{ret}}, N_{\text{ret}}, R_{\text{norm}}, G_{\text{norm}}, B_{\text{norm}}]$, the assembled feature array for the tile ▷ $P \in \mathbb{R}^{N \times 10}$, float32, where N is the number of points in the tile
 - 10: $L \leftarrow \text{cls}$, the label array for the tile ▷ raw LAS codes, remapped at load time via $\{0:0, 2:1, 3:2, 4:3, 5:4, 6:5, 11:6, 18:7\}$
 - 11: Write P, L to `numpy/s(T)/`
-

Algorithm 2 Training procedure.

Require: Training tiles $\mathcal{T}$, validation tiles $\mathcal{V}$, number of epochs E , window size $n = 65,536$, class weights $\mathbf{w}$, learning rate η , decay factor δ

- 1: Initialize the network, loading a compatible checkpoint if one exists
- 2: best mIoU $\leftarrow$ the validation mIoU of the loaded checkpoint, or 0 for a fresh network
- 3: **for** epoch = 1 to E **do**
- 4: **for** each training batch **do**
- 5: Draw a window from a randomly selected tile, centered on a random Class 18 point with probability p_{18} and otherwise drawn uniformly from the tile
- 6: Apply random rotation, jitter, and scaling to the spatial coordinates only
- 7: Compute the forward pass through the network
- 8: Compute the focal loss over the annotated points of the batch, applying the class weights $\mathbf{w}$ once
- 9: Backpropagate, clip gradients to a maximum norm of 1.0, and update network parameters
- 10: **end for**
- 11: Score every validation tile in $\mathcal{V}$ under the run’s validation scheme, sampled windows before Iteration 5 and the exhaustive window scheme of Algorithm 3 in Iteration 5, without gradient tracking
- 12: Compute per class IoU and mean IoU over all validation predictions, excluding points with no ground truth label
- 13: **if** current mIoU > best mIoU **then**
- 14: Save the current network parameters as the best checkpoint
- 15: best mIoU $\leftarrow$ current mIoU
- 16: **end if**
- 17: Decay the learning rate, $\eta \leftarrow \eta \cdot \delta$
- 18: **end for**
- 19: **return** the best checkpoint saved during training

Algorithm 3 Full tile inference procedure.

Require: A full annotated or unannotated tile with N points, the best trained checkpoint, window point budget $n = 65,536$, a core cell size and a halo width

- 1: Load the tile and extract the ten feature channels described in Section 2.1.3
- 2: Load the trained network parameters from the checkpoint
- 3: Partition the horizontal extent of the tile into core cells, each extended by its halo, so that every point lies in the core of exactly one cell
- 4: Initialize an accumulator of class probabilities, one row per point of the tile
- 5: **for** each core cell **do**
- 6: Gather the points of the cell and its halo, up to n points, repeating points as padding when fewer are present
- 7: Compute class probabilities for the window through the network, without gradient tracking
- 8: Add the probabilities of every real point in the window to its row of the accumulator
- 9: **end for**
- 10: Assign each point the class with the highest accumulated probability
- 11: **if** ground truth labels are available for this tile **then**
- 12: Compute the confusion matrix, per class IoU and mean IoU, dropping points whose ground truth label is Class 0
- 13: **end if**
- 14: Write the predicted class of every point back into a copy of the original tile, replacing its classification field
- 15: **return** the classified tile and, if available, the confusion matrix and IoU results

C

Computational Environment

Table C.1 lists the cluster and the software environment used for training and inference, and Table C.2 lists the packages inside the container.

Table C.1: HPC and software environment.

Component	Detail
Cluster	Vera, Chalmers C3SE
GPU	NVIDIA A100-SXM4-40GB (1 to 2 per job)
Wall-time limit	3 to 9 days per allocation
Container	Apptainer image
Python / PyTorch	3.10 / 2.x
Point I/O	laspy 2.4
DataLoader	<code>num_workers = 6</code> , batched validation
Storage	Project no-quota filesystem
Session mgmt	<code>sbatch</code> batch jobs
Checkpointing	Best by mIoU and every 10 epochs

Table C.2: Packages installed inside the Apptainer container.

Package	Version or purpose
Python	3.10
PyTorch [29]	2.x, with CUDA support
NumPy	Array operations and feature construction
laspy	2.4, reading and writing LAS point cloud files
tqdm	Training and inference progress bars
SciPy	Supporting geometric computations

D

Iteration 5 Training Logs

This appendix reproduces the per epoch validation record of the three Iteration 5 dataset profiles, Colored, Mixed and Uncolored, exactly as logged by the training script on the Vera cluster. Columns show the epoch, training loss, TrL, validation loss, VaL, mean IoU, and the IoU of ground, IoU₀₂, low vegetation, IoU₀₃, medium vegetation, IoU₀₄, high vegetation, IoU₀₅, building, IoU₀₆, road, IoU₁₁, and moving vehicles, IoU₁₈. The column Note marks the epochs at which a new best checkpoint by validation mIoU was saved, labeled BEST. N/A denotes a class absent from the validation ground truth of that profile. All three profiles were validated under the exhaustive scheme of Section 3.6.4, that is, on every annotated point of every validation tile, and the row of the checkpoint reported in Chapter 4 is in bold. Table D.1, Table D.2 and Table D.3 hold the records of the Colored, Mixed and Uncolored profiles respectively, and the tiles and settings of the three profiles are given in Table 3.6 and Figure 3.4. The tables are reproduced in full, since the epoch-to-epoch volatility of the moving vehicle class and the differences between the profiles discussed in Chapters 4 and 5 are evidence for the conclusions drawn.

D.1 Colored Profile

Table D.1: Iteration 5 Colored profile, per epoch training and validation record: training loss (TrL), validation loss (VaL), mean IoU and per-class IoU, with BEST marking each new best checkpoint and the reported checkpoint in bold.

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
1	0.8673	0.2558	0.3031	0.3189	0.0283	0.0000	0.8137	0.0000	0.9078	0.0531	BEST
2	0.3863	0.1270	0.4705	0.7905	0.4222	0.0000	0.9067	0.0000	0.9484	0.2255	BEST
3	0.2896	0.1431	0.4972	0.8423	0.5085	0.0000	0.7920	0.0000	0.9585	0.3794	BEST
4	0.2145	0.1138	0.5324	0.8885	0.7519	0.0000	0.8805	0.0000	0.9482	0.2580	BEST
5	0.2394	0.1124	0.4860	0.7845	0.5108	0.0000	0.8921	0.0000	0.9504	0.2638	
6	0.2167	0.1323	0.5249	0.7806	0.8092	0.0000	0.8894	0.0000	0.9290	0.2657	
7	0.2068	0.1242	0.5173	0.7789	0.7631	0.0000	0.8846	0.0000	0.9310	0.2637	
8	0.2092	0.1628	0.4066	0.5602	0.3138	0.0000	0.8839	0.0000	0.9160	0.1726	
9	0.1825	0.0909	0.5626	0.9346	0.8712	0.0000	0.9358	0.0000	0.9702	0.2266	BEST
10	0.1746	0.0829	0.5594	0.9175	0.8173	0.0000	0.9402	0.0000	0.9717	0.2690	
11	0.1790	0.1304	0.4950	0.8579	0.8070	0.0000	0.7338	0.0000	0.9651	0.1015	
12	0.1533	0.0777	0.5809	0.9427	0.8540	0.0000	0.8928	0.0000	0.9720	0.4046	BEST
13	0.1597	0.0844	0.5786	0.9401	0.8687	0.0000	0.8992	0.0000	0.9671	0.3748	
14	0.1332	0.0889	0.5687	0.9445	0.9003	0.0000	0.9005	0.0000	0.9490	0.2863	
15	0.1389	0.0964	0.5549	0.9338	0.8322	0.0000	0.9085	0.0000	0.9452	0.2648	

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
16	0.1410	0.0640	0.6008	0.9079	0.8184	0.0000	0.8982	0.0000	0.9787	0.6025	BEST
17	0.1397	0.0746	0.5537	0.9389	0.7530	0.0000	0.9028	0.0000	0.9625	0.3191	
18	0.1257	0.0688	0.5872	0.9357	0.8424	0.0000	0.9168	0.0000	0.9816	0.4338	
19	0.1524	0.0626	0.5765	0.9364	0.8257	0.0000	0.9047	0.0000	0.9671	0.4014	
20	0.1278	0.1617	0.5055	0.4730	0.8400	0.0000	0.9295	0.0000	0.8314	0.4644	
21	0.1412	0.0668	0.5754	0.9388	0.8564	0.0000	0.8984	0.0000	0.9696	0.3646	BEST
22	0.1404	0.0650	0.6070	0.9533	0.8644	0.0000	0.9036	0.0000	0.9846	0.5432	
23	0.1166	0.0743	0.6001	0.9241	0.9293	0.0000	0.9011	0.0000	0.9769	0.4690	
24	0.1057	0.0595	0.6120	0.9444	0.8967	0.0000	0.9067	0.0000	0.9818	0.5546	
25	0.1098	0.0700	0.6022	0.9195	0.8906	0.0000	0.9187	0.0000	0.9791	0.5078	
26	0.1121	0.0808	0.5707	0.9424	0.8168	0.0000	0.9146	0.0000	0.9811	0.3402	BEST
27	0.1076	0.0653	0.5977	0.9537	0.8716	0.0000	0.9139	0.0000	0.9812	0.4638	
28	0.0880	0.0671	0.6062	0.9224	0.8612	0.0000	0.9030	0.0000	0.9804	0.5767	
29	0.1123	0.0644	0.6004	0.8790	0.8360	0.0000	0.9188	0.0000	0.9753	0.5935	
30	0.1046	0.0633	0.5979	0.9409	0.8368	0.0000	0.8925	0.0000	0.9832	0.5317	
31	0.1045	0.0703	0.6008	0.9444	0.8427	0.0000	0.9281	0.0000	0.9856	0.5051	BEST
32	0.1158	0.0700	0.5949	0.9464	0.8065	0.0000	0.9107	0.0000	0.9832	0.5171	
33	0.1126	0.0817	0.5978	0.9438	0.8402	0.0000	0.9090	0.0000	0.9839	0.5078	
34	0.1023	0.0601	0.6178	0.9684	0.9068	0.0000	0.9132	0.0000	0.9872	0.5490	
35	0.0972	0.0660	0.6091	0.9446	0.8233	0.0000	0.8921	0.0000	0.9847	0.6192	
36	0.0876	0.0677	0.6028	0.9485	0.8496	0.0000	0.9108	0.0000	0.9864	0.5245	BEST
37	0.0833	0.0661	0.6163	0.9528	0.8112	0.0000	0.8965	0.0000	0.9877	0.6662	
38	0.0994	0.0760	0.5947	0.9301	0.7968	0.0000	0.8921	0.0000	0.9849	0.5590	
39	0.0918	0.0703	0.6110	0.9489	0.8530	0.0000	0.9134	0.0000	0.9870	0.5750	
40	0.0930	0.0689	0.6075	0.9536	0.8578	0.0000	0.8998	0.0000	0.9859	0.5551	
41	0.0973	0.0638	0.6100	0.9567	0.8536	0.0000	0.8933	0.0000	0.9862	0.5804	BEST
42	0.0935	0.0689	0.6116	0.9256	0.8364	0.0000	0.8950	0.0000	0.9835	0.6409	
43	0.0741	0.0596	0.6163	0.9514	0.8337	0.0000	0.8968	0.0000	0.9880	0.6443	
44	0.0886	0.0531	0.6355	0.9698	0.9044	0.0000	0.8948	0.0000	0.9890	0.6907	
45	0.0902	0.0662	0.6120	0.9513	0.8376	0.0000	0.8943	0.0000	0.9864	0.6140	
46	0.0895	0.0714	0.6057	0.9528	0.8133	0.0000	0.9017	0.0000	0.9870	0.5848	BEST
47	0.1073	0.0532	0.6247	0.9523	0.8398	0.0000	0.8955	0.0000	0.9896	0.6957	
48	0.0743	0.0649	0.6209	0.9537	0.8752	0.0000	0.8967	0.0000	0.9875	0.6330	
49	0.0788	0.0553	0.6372	0.9741	0.9279	0.0005	0.8972	0.0000	0.9888	0.6720	
50	0.0756	0.0588	0.6267	0.9662	0.8975	0.0000	0.8955	0.0000	0.9883	0.6393	
51	0.0867	0.0675	0.6125	0.9437	0.8382	0.0000	0.8895	0.0000	0.9866	0.6299	BEST
52	0.0880	0.0696	0.6214	0.9525	0.9002	0.0000	0.9019	0.0000	0.9861	0.6087	
53	0.0788	0.0674	0.6216	0.9463	0.8423	0.0000	0.8920	0.0000	0.9885	0.6822	
54	0.0784	0.0718	0.6109	0.9549	0.8375	0.0000	0.9023	0.0000	0.9885	0.5932	
55	0.0775	0.0591	0.6389	0.9765	0.9377	0.0000	0.8937	0.0000	0.9894	0.6749	
56	0.0797	0.0679	0.6164	0.9657	0.9029	0.0002	0.9012	0.0000	0.9855	0.5593	BEST
57	0.0712	0.0597	0.6172	0.9481	0.8457	0.0000	0.8909	0.0000	0.9871	0.6484	
58	0.0733	0.0668	0.6142	0.9277	0.9003	0.0000	0.8956	0.0000	0.9799	0.5961	
59	0.0796	0.0682	0.6301	0.9456	0.9235	0.0000	0.8975	0.0000	0.9850	0.6592	
60	0.0765	0.0615	0.6360	0.9678	0.9125	0.0000	0.8939	0.0000	0.9896	0.6880	
61	0.0917	0.0657	0.6215	0.9575	0.8822	0.0002	0.8898	0.0000	0.9862	0.6343	BEST
62	0.0962	0.0582	0.6183	0.9315	0.8372	0.0000	0.9024	0.0000	0.9860	0.6711	
63	0.0956	0.0625	0.6108	0.9562	0.8201	0.0001	0.8880	0.0000	0.9880	0.6235	
64	0.0832	0.0845	0.6069	0.9523	0.8303	0.0000	0.8923	0.0000	0.9870	0.5867	
65	0.0965	0.0863	0.5903	0.9351	0.8278	0.0000	0.9068	0.0000	0.9816	0.4806	
66	0.0878	0.0583	0.6200	0.9599	0.8602	0.0000	0.8953	0.0000	0.9878	0.6370	BEST
67	0.0835	0.0690	0.6187	0.9547	0.8354	0.0000	0.8931	0.0000	0.9890	0.6590	
68	0.0972	0.0616	0.6030	0.9564	0.8283	0.0004	0.9014	0.0000	0.9852	0.5491	
69	0.0840	0.0698	0.6087	0.9523	0.8185	0.0007	0.9001	0.0000	0.9884	0.6008	

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
70	0.0876	0.0645	0.6139	0.9611	0.8362	0.0000	0.9043	0.0000	0.9885	0.6071	
71	0.0835	0.0691	0.6160	0.9599	0.8534	0.0000	0.8948	0.0000	0.9879	0.6159	
72	0.0808	0.0678	0.6176	0.9573	0.8295	0.0000	0.8950	0.0000	0.9893	0.6519	
73	0.0803	0.0721	0.6068	0.9496	0.8089	0.0003	0.9036	0.0000	0.9864	0.5990	
74	0.0704	0.0757	0.6120	0.9515	0.8410	0.0000	0.8833	0.0000	0.9858	0.6224	
75	0.0725	0.0619	0.6287	0.9629	0.8795	0.0000	0.8953	0.0000	0.9886	0.6743	
76	0.0710	0.0716	0.6091	0.9327	0.8371	0.0007	0.8231	0.0000	0.9767	0.6937	
77	0.0782	0.0611	0.6152	0.9475	0.7949	0.0005	0.8806	0.0000	0.9879	0.6948	
78	0.0650	0.0423	0.6337	0.9530	0.8709	0.0001	0.8938	0.0000	0.9878	0.7306	
79	0.0848	0.0417	0.6246	0.9588	0.8388	0.0007	0.8911	0.0000	0.9891	0.6938	
80	0.0712	0.0753	0.6045	0.9404	0.8765	0.0001	0.9031	0.0000	0.9831	0.5283	
81	0.0739	0.0634	0.6211	0.9511	0.9008	0.0001	0.8905	0.0000	0.9861	0.6192	
82	0.0756	0.0686	0.6201	0.9604	0.8569	0.0001	0.8897	0.0000	0.9888	0.6451	
83	0.0664	0.0915	0.5713	0.9236	0.8192	0.0004	0.6862	0.0000	0.9544	0.6151	
84	0.0653	0.0523	0.6352	0.9748	0.8878	0.0045	0.8950	0.0000	0.9901	0.6943	
85	0.0636	0.0541	0.6451	0.9800	0.9381	0.0023	0.8935	0.0000	0.9905	0.7111	BEST
86	0.0647	0.0644	0.6243	0.9695	0.8759	0.0019	0.8937	0.0000	0.9896	0.6395	
87	0.0617	0.0543	0.6428	0.9847	0.9185	0.0007	0.9011	0.0000	0.9921	0.7021	
88	0.0479	0.0724	0.6129	0.9440	0.8571	0.0000	0.9081	0.0000	0.9863	0.5944	
89	0.0637	0.0711	0.6164	0.9350	0.8380	0.0018	0.8922	0.0000	0.9864	0.6613	
90	0.0575	0.0685	0.6156	0.9594	0.8351	0.0012	0.8980	0.0000	0.9886	0.6267	
91	0.0628	0.0602	0.6218	0.9645	0.8617	0.0005	0.8937	0.0000	0.9902	0.6421	
92	0.0687	0.0655	0.6162	0.9607	0.8038	0.0000	0.8995	0.0000	0.9893	0.6599	
93	0.0704	0.0600	0.6231	0.9593	0.8216	0.0011	0.9007	0.0000	0.9907	0.6880	
94	0.0648	0.0511	0.6300	0.9595	0.8487	0.0004	0.8947	0.0000	0.9928	0.7142	
95	0.0623	0.0636	0.6204	0.9555	0.8385	0.0005	0.8901	0.0000	0.9903	0.6682	
96	0.0596	0.0782	0.6094	0.9581	0.8408	0.0007	0.8974	0.0000	0.9886	0.5805	
97	0.0538	0.0632	0.6299	0.9681	0.8521	0.0053	0.8733	0.0000	0.9896	0.7211	
98	0.0640	0.0796	0.6163	0.9487	0.8354	0.0015	0.8853	0.0000	0.9877	0.6557	
99	0.0581	0.0746	0.6015	0.9632	0.7954	0.0004	0.7152	0.0000	0.9647	0.7717	
100	0.0514	0.0711	0.6206	0.9669	0.8643	0.0046	0.8852	0.0000	0.9896	0.6335	
101	0.0617	0.0616	0.6293	0.9620	0.8554	0.0021	0.8879	0.0000	0.9902	0.7077	
102	0.0628	0.0679	0.6189	0.9504	0.8388	0.0012	0.8910	0.0000	0.9897	0.6608	
103	0.0481	0.0586	0.6413	0.9714	0.9098	0.0015	0.8876	0.0000	0.9916	0.7270	
104	0.0588	0.0626	0.6431	0.9812	0.9269	0.0037	0.8916	0.0000	0.9908	0.7076	
105	0.0559	0.1025	0.5691	0.9720	0.7272	0.0006	0.6610	0.0000	0.9575	0.6656	
106	0.0614	0.0660	0.6277	0.9707	0.8515	0.0186	0.8998	0.0000	0.9897	0.6637	
107	0.0612	0.0570	0.6366	0.9713	0.8659	0.0176	0.8604	0.0000	0.9901	0.7511	
108	0.0529	0.0469	0.6496	0.9776	0.9132	0.0047	0.8950	0.0000	0.9915	0.7652	BEST
109	0.0492	0.0569	0.6469	0.9776	0.9348	0.0042	0.8971	0.0000	0.9919	0.7224	
110	0.0402	0.0676	0.6274	0.9651	0.8657	0.0429	0.8935	0.0000	0.9893	0.6353	
111	0.0497	0.0603	0.6430	0.9717	0.9240	0.0022	0.8910	0.0000	0.9903	0.7215	
112	0.0515	0.0681	0.6329	0.9739	0.9363	0.0010	0.8838	0.0000	0.9890	0.6465	
113	0.0508	0.0716	0.6259	0.9282	0.9016	0.0016	0.8615	0.0000	0.9863	0.7022	
114	0.0554	0.0680	0.6398	0.9786	0.9463	0.0006	0.8731	0.0000	0.9897	0.6906	
115	0.0433	0.0501	0.6523	0.9848	0.9552	0.0085	0.8900	0.0000	0.9911	0.7368	BEST
116	0.0568	0.0620	0.6461	0.9802	0.9408	0.0020	0.8867	0.0000	0.9916	0.7214	
117	0.0491	0.0641	0.6387	0.9752	0.9467	0.0012	0.8814	0.0000	0.9905	0.6762	
118	0.0435	0.0649	0.6384	0.9753	0.9385	0.0001	0.8457	0.0000	0.9849	0.7245	
119	0.0548	0.0594	0.6515	0.9806	0.9524	0.0113	0.8899	0.0000	0.9911	0.7352	
120	0.0475	0.0550	0.6557	0.9806	0.9512	0.0276	0.8936	0.0000	0.9920	0.7446	BEST
121	0.0407	0.0597	0.6409	0.9772	0.9120	0.0033	0.8726	0.0000	0.9932	0.7282	
122	0.0413	0.0477	0.6582	0.9730	0.8961	0.0793	0.8864	0.0000	0.9932	0.7796	BEST
123	0.0421	0.0538	0.6534	0.9803	0.9405	0.0258	0.8923	0.0000	0.9921	0.7428	

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
124	0.0471	0.0515	0.6437	0.9796	0.8927	0.0012	0.8346	0.0000	0.9890	0.8090	
125	0.0495	0.0635	0.6357	0.9724	0.8936	0.0300	0.8806	0.0000	0.9925	0.6810	
126	0.0484	0.0508	0.6512	0.9758	0.9481	0.0235	0.8852	0.0000	0.9914	0.7345	
127	0.0510	0.0622	0.6447	0.9820	0.9441	0.0216	0.8897	0.0000	0.9916	0.6839	
128	0.0613	0.0560	0.6478	0.9827	0.9410	0.0104	0.8741	0.0000	0.9932	0.7328	
129	0.0453	0.0548	0.6601	0.9834	0.9498	0.0608	0.8915	0.0000	0.9922	0.7430	BEST
130	0.0318	0.0474	0.6767	0.9845	0.9607	0.1142	0.8992	0.0000	0.9939	0.7844	BEST
131	0.0352	0.0659	0.6524	0.9778	0.9459	0.0660	0.8948	0.0000	0.9893	0.6933	
132	0.0398	0.0520	0.6600	0.9832	0.9495	0.0780	0.8765	0.0000	0.9907	0.7419	
133	0.0396	0.0547	0.6549	0.9797	0.9409	0.0510	0.8880	0.0000	0.9920	0.7325	
134	0.0400	0.0641	0.6452	0.9821	0.9480	0.0002	0.8841	0.0000	0.9917	0.7102	
135	0.0384	0.0486	0.6638	0.9845	0.9623	0.0556	0.8848	0.0000	0.9928	0.7668	
136	0.0531	0.0577	0.6441	0.9440	0.9467	0.0070	0.8831	0.0000	0.9869	0.7409	
137	0.0347	0.0604	0.6477	0.9839	0.9528	0.0005	0.8854	0.0000	0.9926	0.7187	
138	0.0338	0.0565	0.6530	0.9822	0.9630	0.0048	0.8919	0.0000	0.9918	0.7370	
139	0.0472	0.0684	0.6529	0.9799	0.9496	0.0341	0.8916	0.0000	0.9909	0.7240	
140	0.0415	0.0519	0.6545	0.9801	0.9324	0.0134	0.8848	0.0000	0.9931	0.7775	
141	0.0357	0.0610	0.6502	0.9855	0.9597	0.0322	0.8759	0.0000	0.9937	0.7046	
142	0.0384	0.0625	0.6561	0.9841	0.9562	0.0722	0.8958	0.0000	0.9913	0.6930	
143	0.0367	0.0531	0.6610	0.9842	0.9558	0.0565	0.8903	0.0000	0.9925	0.7474	
144	0.0379	0.0575	0.6605	0.9835	0.9561	0.0975	0.8714	0.0000	0.9947	0.7204	
145	0.0337	0.0449	0.6650	0.9826	0.9559	0.0646	0.8867	0.0000	0.9940	0.7710	
146	0.0411	0.0529	0.6635	0.9848	0.9482	0.0953	0.8832	0.0000	0.9941	0.7386	
147	0.0382	0.0582	0.6493	0.9843	0.9442	0.0305	0.8966	0.0000	0.9914	0.6981	
148	0.0425	0.0471	0.6575	0.9806	0.9440	0.0261	0.8773	0.0000	0.9945	0.7798	
149	0.0340	0.0492	0.6553	0.9847	0.9537	0.0094	0.8899	0.0000	0.9942	0.7553	
150	0.0315	0.0466	0.6533	0.9785	0.9425	0.0022	0.8711	0.0000	0.9948	0.7837	

D.2 Mixed Profile

Table D.2: Iteration 5 Mixed profile, per epoch training and validation record: training loss (TrL), validation loss (VaL), mean IoU and per-class IoU, with BEST marking each new best checkpoint and the reported checkpoint in bold.

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
1	0.8587	0.3556	0.2560	0.0000	0.1171	0.0000	0.7734	0.0000	0.8605	0.0408	BEST
2	0.4418	0.2195	0.3884	0.5051	0.3481	0.0000	0.8849	0.0000	0.9125	0.0687	BEST
3	0.3668	0.2382	0.3960	0.6347	0.3085	0.0000	0.8261	0.0000	0.9174	0.0851	BEST
4	0.2557	0.1343	0.4978	0.7894	0.7490	0.0000	0.8798	0.0000	0.9414	0.1252	BEST
5	0.2387	0.1978	0.4081	0.5920	0.4054	0.0000	0.8590	0.0000	0.8745	0.1259	
6	0.2494	0.1505	0.5062	0.8261	0.7085	0.0000	0.8774	0.0000	0.9530	0.1782	BEST
7	0.2195	0.1323	0.4892	0.8589	0.5997	0.0000	0.8433	0.0000	0.9411	0.1809	
8	0.2309	0.1193	0.5383	0.9239	0.8053	0.0000	0.8981	0.0000	0.9500	0.1907	BEST
9	0.1890	0.1369	0.5201	0.7950	0.8133	0.0000	0.9194	0.0000	0.9367	0.1761	
10	0.1907	0.1326	0.5162	0.7848	0.7790	0.0000	0.9266	0.0000	0.9340	0.1887	
11	0.1698	0.1404	0.4980	0.7152	0.7562	0.0000	0.9074	0.0000	0.9178	0.1895	
12	0.1899	0.1111	0.5345	0.8827	0.7934	0.0000	0.9254	0.0000	0.9533	0.1863	
13	0.1909	0.1177	0.5444	0.9327	0.8046	0.0000	0.9129	0.0000	0.9653	0.1952	BEST
14	0.1828	0.1137	0.5487	0.9288	0.8282	0.0000	0.9191	0.0000	0.9552	0.2096	BEST
15	0.1517	0.1037	0.5251	0.8952	0.7299	0.0000	0.9167	0.0000	0.9397	0.1940	

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
16	0.1661	0.0979	0.5489	0.9375	0.8195	0.0000	0.9234	0.0000	0.9644	0.1977	BEST
17	0.1459	0.1023	0.5594	0.9469	0.8878	0.0000	0.9150	0.0000	0.9584	0.2078	BEST
18	0.1587	0.1024	0.5433	0.9034	0.8037	0.0000	0.9257	0.0000	0.9643	0.2062	
19	0.1587	0.1383	0.5368	0.8770	0.8095	0.0000	0.9169	0.0000	0.9621	0.1918	
20	0.1446	0.1015	0.5408	0.9042	0.7732	0.0000	0.9147	0.0000	0.9531	0.2401	
21	0.1406	0.1126	0.5590	0.9315	0.8306	0.0000	0.9320	0.0000	0.9728	0.2464	
22	0.1203	0.1441	0.5529	0.9449	0.8303	0.0000	0.9201	0.0000	0.9781	0.1970	
23	0.1266	0.1180	0.5435	0.8938	0.8300	0.0000	0.9177	0.0000	0.9573	0.2052	
24	0.1416	0.1094	0.5490	0.9251	0.8383	0.0000	0.9155	0.0000	0.9622	0.2019	
25	0.1363	0.1239	0.5684	0.9456	0.9313	0.0000	0.9251	0.0000	0.9719	0.2047	BEST
26	0.1278	0.1323	0.5506	0.8803	0.8617	0.0000	0.9228	0.0000	0.9598	0.2294	
27	0.1395	0.1262	0.5605	0.9281	0.8389	0.0000	0.8981	0.0000	0.9764	0.2817	
28	0.1249	0.1193	0.5344	0.8532	0.8280	0.0000	0.9273	0.0000	0.9457	0.1866	
29	0.1193	0.1146	0.5620	0.9370	0.8341	0.0000	0.9218	0.0000	0.9759	0.2653	
30	0.1193	0.1345	0.5660	0.9533	0.8615	0.0000	0.9104	0.0000	0.9798	0.2572	
31	0.1346	0.1482	0.5554	0.9496	0.8241	0.0000	0.9238	0.0000	0.9777	0.2125	
32	0.1307	0.1134	0.5555	0.9257	0.8359	0.0000	0.9039	0.0000	0.9695	0.2531	
33	0.1149	0.1300	0.5593	0.9408	0.8278	0.0000	0.9137	0.0000	0.9757	0.2573	
34	0.1134	0.0984	0.5741	0.9457	0.8291	0.0000	0.9104	0.0000	0.9803	0.3534	BEST
35	0.1150	0.1172	0.5683	0.9477	0.8314	0.0000	0.9200	0.0000	0.9809	0.2982	
36	0.1261	0.0792	0.5749	0.9426	0.8276	0.0000	0.9137	0.0000	0.9778	0.3626	BEST
37	0.1257	0.0944	0.5904	0.9548	0.9313	0.0000	0.9118	0.0000	0.9738	0.3609	BEST
38	0.1299	0.1148	0.5576	0.8949	0.8294	0.0000	0.9319	0.0000	0.9654	0.2816	
39	0.1055	0.1465	0.5536	0.8875	0.8280	0.0000	0.9352	0.0000	0.9690	0.2557	
40	0.1055	0.1237	0.5771	0.9580	0.9066	0.0000	0.9061	0.0000	0.9768	0.2924	
41	0.1268	0.0857	0.5892	0.9134	0.9392	0.0000	0.9408	0.0000	0.9687	0.3626	
42	0.1140	0.1045	0.5748	0.9579	0.8587	0.0000	0.9105	0.0000	0.9774	0.3190	
43	0.1088	0.0781	0.6072	0.9511	0.9331	0.0000	0.9170	0.0000	0.9810	0.4681	BEST
44	0.1165	0.0911	0.5945	0.9616	0.9425	0.0000	0.9186	0.0000	0.9795	0.3591	
45	0.1048	0.0890	0.5870	0.9692	0.9293	0.0000	0.9199	0.0000	0.9766	0.3139	
46	0.1024	0.1196	0.5825	0.9592	0.9275	0.0000	0.9327	0.0000	0.9781	0.2802	
47	0.1212	0.1386	0.5839	0.9673	0.9325	0.0000	0.9195	0.0000	0.9792	0.2887	
48	0.1022	0.0770	0.5949	0.9450	0.8731	0.0000	0.8993	0.0000	0.9796	0.4676	
49	0.1098	0.1159	0.5808	0.9559	0.8533	0.0000	0.8946	0.0000	0.9799	0.3822	
50	0.0943	0.1007	0.5962	0.9713	0.9068	0.0000	0.9320	0.0000	0.9819	0.3812	
51	0.1118	0.0760	0.5997	0.8999	0.9199	0.0000	0.9113	0.0000	0.9711	0.4953	
52	0.0976	0.0815	0.6025	0.9376	0.9070	0.0000	0.9085	0.0000	0.9777	0.4866	
53	0.0971	0.0698	0.5942	0.9338	0.8565	0.0000	0.9069	0.0000	0.9787	0.4837	
54	0.1060	0.1217	0.5838	0.9529	0.9250	0.0000	0.9037	0.0000	0.9775	0.3278	
55	0.0976	0.0874	0.6172	0.9589	0.9231	0.0000	0.8972	0.0000	0.9827	0.5584	BEST
56	0.0974	0.0867	0.6157	0.9622	0.9197	0.0000	0.8999	0.0000	0.9838	0.5442	
57	0.0925	0.1083	0.5919	0.9664	0.9484	0.0000	0.9134	0.0000	0.9791	0.3361	
58	0.1071	0.0856	0.6029	0.9470	0.8708	0.0000	0.8873	0.0000	0.9799	0.5351	
59	0.0878	0.0728	0.6114	0.9572	0.9007	0.0000	0.9032	0.0000	0.9806	0.5378	
60	0.0883	0.0791	0.6016	0.9736	0.9258	0.0000	0.9081	0.0000	0.9805	0.4234	
61	0.0891	0.0984	0.5880	0.9627	0.9152	0.0000	0.9060	0.0000	0.9774	0.3550	
62	0.0925	0.0710	0.6195	0.9712	0.9490	0.0000	0.9012	0.0000	0.9831	0.5320	BEST
63	0.1024	0.0910	0.5907	0.9458	0.9091	0.0000	0.8992	0.0000	0.9769	0.4040	
64	0.0831	0.0813	0.6060	0.9706	0.9295	0.0000	0.9012	0.0000	0.9812	0.4594	
65	0.0902	0.1030	0.5928	0.9378	0.9198	0.0000	0.8987	0.0000	0.9766	0.4167	
66	0.1041	0.0924	0.6004	0.9579	0.9095	0.0000	0.9066	0.0000	0.9800	0.4491	
67	0.0830	0.0837	0.6025	0.9668	0.9318	0.0000	0.9088	0.0000	0.9800	0.4299	
68	0.0946	0.0804	0.6138	0.9615	0.9280	0.0000	0.9043	0.0000	0.9829	0.5202	
69	0.0809	0.0700	0.6103	0.9662	0.8541	0.0000	0.9050	0.0006	0.9844	0.5618	

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
70	0.0798	0.1091	0.5882	0.9421	0.8730	0.0000	0.9002	0.0000	0.9786	0.4233	
71	0.1007	0.0790	0.6167	0.9683	0.8913	0.0000	0.9063	0.0000	0.9848	0.5658	
72	0.0906	0.0922	0.6093	0.9694	0.9444	0.0000	0.8899	0.0000	0.9833	0.4779	
73	0.0815	0.1046	0.5994	0.9691	0.9297	0.0000	0.9068	0.0000	0.9828	0.4075	
74	0.0853	0.0726	0.6165	0.9586	0.9461	0.0000	0.9004	0.0000	0.9813	0.5288	
75	0.0937	0.0782	0.6089	0.9562	0.8893	0.0000	0.9081	0.0000	0.9829	0.5255	
76	0.1022	0.0985	0.6045	0.9628	0.9383	0.0000	0.8921	0.0000	0.9816	0.4565	
77	0.0773	0.1057	0.5823	0.9701	0.8814	0.0000	0.8992	0.0000	0.9790	0.3463	
78	0.0891	0.0808	0.5953	0.8992	0.9111	0.0000	0.9029	0.0139	0.9696	0.4706	
79	0.0879	0.0762	0.6036	0.9333	0.8999	0.0000	0.8881	0.0074	0.9746	0.5221	
80	0.0897	0.0942	0.5933	0.9514	0.8987	0.0000	0.9052	0.0004	0.9783	0.4192	
81	0.0859	0.0760	0.6084	0.9676	0.9077	0.0000	0.9092	0.0156	0.9812	0.4777	
82	0.0741	0.0861	0.5996	0.9489	0.9175	0.0000	0.9065	0.0009	0.9778	0.4457	
83	0.0839	0.0834	0.6152	0.9604	0.9146	0.0000	0.8941	0.0000	0.9826	0.5548	
84	0.0812	0.1016	0.6039	0.9642	0.9413	0.0000	0.9044	0.0000	0.9822	0.4353	
85	0.0739	0.1197	0.5894	0.9664	0.9029	0.0000	0.9040	0.0011	0.9786	0.3729	
86	0.0726	0.1078	0.5971	0.9606	0.9131	0.0000	0.8917	0.0012	0.9803	0.4325	
87	0.0855	0.1254	0.5913	0.9598	0.9306	0.0000	0.8952	0.0001	0.9792	0.3742	
88	0.0789	0.1152	0.5877	0.9391	0.9122	0.0000	0.9072	0.0014	0.9753	0.3788	
89	0.0814	0.0969	0.6038	0.9667	0.9032	0.0000	0.8962	0.0000	0.9834	0.4770	
90	0.0804	0.0755	0.6220	0.9737	0.9117	0.0000	0.9022	0.0005	0.9852	0.5806	BEST
91	0.0772	0.0906	0.6120	0.9354	0.9292	0.0001	0.9000	0.0076	0.9780	0.5337	
92	0.0752	0.0773	0.6062	0.9543	0.8526	0.0000	0.9136	0.0252	0.9820	0.5159	
93	0.0848	0.1123	0.5927	0.9700	0.9078	0.0000	0.9036	0.0050	0.9803	0.3821	
94	0.0706	0.0723	0.6157	0.9468	0.8860	0.0000	0.9064	0.0266	0.9806	0.5632	
95	0.0832	0.1018	0.5939	0.9455	0.9197	0.0000	0.8942	0.0004	0.9770	0.4204	
96	0.0769	0.0953	0.5955	0.9622	0.8826	0.0001	0.9040	0.0052	0.9791	0.4356	
97	0.0751	0.0980	0.6072	0.9569	0.9114	0.0000	0.8996	0.0005	0.9812	0.5010	
98	0.0670	0.0943	0.6088	0.9629	0.9387	0.0000	0.8924	0.0019	0.9816	0.4842	
99	0.0753	0.0948	0.6018	0.9533	0.9244	0.0000	0.8975	0.0001	0.9793	0.4581	
100	0.0780	0.0930	0.6052	0.9728	0.9119	0.0001	0.8954	0.0038	0.9822	0.4704	
101	0.0826	0.0875	0.5991	0.9603	0.9201	0.0000	0.9096	0.0046	0.9802	0.4188	
102	0.0769	0.1031	0.5964	0.9579	0.9226	0.0000	0.9020	0.0014	0.9784	0.4126	
103	0.0745	0.0993	0.6040	0.9563	0.8918	0.0001	0.9047	0.0361	0.9796	0.4595	
104	0.0720	0.0840	0.6107	0.9616	0.9087	0.0000	0.9009	0.0323	0.9805	0.4912	
105	0.0748	0.0687	0.6291	0.9627	0.9437	0.0000	0.8946	0.0169	0.9835	0.6023	BEST
106	0.0809	0.1011	0.5990	0.9643	0.9345	0.0001	0.8923	0.0086	0.9794	0.4138	
107	0.0818	0.1311	0.5899	0.9434	0.9334	0.0001	0.8979	0.0035	0.9754	0.3756	
108	0.0794	0.1321	0.5945	0.9725	0.9385	0.0000	0.8956	0.0022	0.9803	0.3723	
109	0.0837	0.0899	0.6091	0.9606	0.9335	0.0001	0.8960	0.0000	0.9824	0.4911	
110	0.0664	0.0963	0.6105	0.9687	0.9336	0.0002	0.8971	0.0000	0.9831	0.4909	
111	0.0765	0.0885	0.6030	0.9551	0.9253	0.0000	0.8951	0.0037	0.9794	0.4623	
112	0.1200	0.1090	0.5863	0.9173	0.9302	0.0000	0.9281	0.0000	0.9734	0.3554	
113	0.1228	0.0596	0.6111	0.9537	0.9249	0.0000	0.9061	0.0000	0.9814	0.5117	BEST
114	0.0945	0.0951	0.5954	0.9321	0.9528	0.0000	0.9158	0.0000	0.9770	0.3904	
115	0.1056	0.2105	0.5612	0.8544	0.9423	0.0000	0.9317	0.0233	0.9576	0.2192	
116	0.1202	0.0880	0.6211	0.9797	0.9293	0.0000	0.8751	0.0000	0.9849	0.5791	BEST
117	0.1019	0.0885	0.6148	0.9721	0.9443	0.0000	0.8989	0.0000	0.9840	0.5039	
118	0.1070	0.0728	0.6152	0.9368	0.8885	0.0003	0.8848	0.0000	0.9815	0.6148	
119	0.0918	0.0647	0.6146	0.9711	0.9183	0.0000	0.9187	0.0000	0.9804	0.5137	
120	0.0971	0.0851	0.6128	0.9086	0.9453	0.0000	0.8971	0.0000	0.9748	0.5639	
121	0.0813	0.1169	0.5899	0.9651	0.9537	0.0000	0.9094	0.0000	0.9786	0.3225	
122	0.1053	0.0691	0.6138	0.9647	0.9428	0.0000	0.8901	0.0000	0.9835	0.5157	
123	0.1128	0.0979	0.5912	0.9444	0.9021	0.0001	0.8920	0.0000	0.9795	0.4202	

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
124	0.1131	0.1017	0.5852	0.9688	0.9256	0.0000	0.9064	0.0000	0.9730	0.3226	
125	0.0901	0.0899	0.5671	0.9584	0.8273	0.0000	0.8859	0.0000	0.9725	0.3254	
126	0.0935	0.0846	0.5925	0.9708	0.8814	0.0001	0.9174	0.0000	0.9780	0.3996	
127	0.0815	0.0983	0.5980	0.9649	0.9334	0.0006	0.8827	0.0000	0.9815	0.4232	
128	0.0897	0.0526	0.6187	0.9639	0.9409	0.0000	0.9066	0.0000	0.9804	0.5393	
129	0.0881	0.1263	0.5831	0.9331	0.8813	0.0000	0.8947	0.0000	0.9776	0.3948	
130	0.0835	0.0835	0.6103	0.9599	0.8716	0.0000	0.9006	0.0000	0.9800	0.5602	
131	0.0906	0.0874	0.6162	0.9693	0.9237	0.0000	0.9060	0.0000	0.9843	0.5299	
132	0.0750	0.1015	0.5959	0.9400	0.8086	0.0000	0.8994	0.0000	0.9845	0.5387	
133	0.0771	0.0879	0.5967	0.9431	0.8388	0.0001	0.9064	0.0000	0.9830	0.5053	
134	0.0905	0.0795	0.6273	0.9713	0.9528	0.0000	0.8878	0.0000	0.9846	0.5945	BEST
135	0.0840	0.1203	0.5893	0.9503	0.9524	0.0000	0.9240	0.0000	0.9759	0.3227	
136	0.0824	0.1170	0.5918	0.8723	0.9562	0.0002	0.9007	0.0000	0.9633	0.4497	
137	0.0912	0.0992	0.6114	0.9279	0.9552	0.0005	0.9071	0.0000	0.9779	0.5114	
138	0.0763	0.0863	0.6051	0.9728	0.9222	0.0000	0.8947	0.0000	0.9832	0.4630	
139	0.0734	0.0794	0.6277	0.9701	0.9280	0.0074	0.8838	0.0000	0.9845	0.6202	BEST
140	0.0677	0.1339	0.5964	0.9625	0.9215	0.0001	0.8969	0.0000	0.9809	0.4130	
141	0.0918	0.1228	0.6027	0.9711	0.9153	0.0000	0.8972	0.0000	0.9830	0.4526	
142	0.0830	0.1494	0.5661	0.9212	0.8263	0.0027	0.8745	0.0000	0.9769	0.3612	
143	0.0775	0.1041	0.6006	0.9656	0.9587	0.0016	0.8882	0.0008	0.9801	0.4091	
144	0.0702	0.0972	0.6065	0.9667	0.8839	0.0045	0.8974	0.0786	0.9793	0.4350	
145	0.0772	0.0979	0.6066	0.9602	0.9296	0.0001	0.8929	0.0513	0.9792	0.4327	
146	0.0841	0.1130	0.6079	0.9524	0.9314	0.0002	0.8985	0.1012	0.9767	0.3946	
147	0.0785	0.0804	0.6354	0.9677	0.9400	0.0051	0.9026	0.0909	0.9840	0.5573	BEST
148	0.0796	0.0758	0.6170	0.9564	0.9433	0.0101	0.8992	0.0192	0.9810	0.5099	
149	0.0674	0.0665	0.6455	0.9469	0.9426	0.0014	0.8959	0.1658	0.9814	0.5848	BEST
150	0.0673	0.0966	0.6144	0.9658	0.9524	0.0061	0.8826	0.0155	0.9817	0.4964	

D.3 Uncolored Profile

Table D.3: Iteration 5 Uncolored profile, per epoch training and validation record: training loss (TrL), validation loss (VaL), mean IoU and per-class IoU (N/A where a class is absent), with BEST marking each new best checkpoint and the reported checkpoint in bold.

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
1	1.9022	1.0542	0.1841	0.0000	0.0001	N/A	0.2656	N/A	0.5306	0.1243	BEST
2	0.6638	0.6496	0.3413	N/A	0.0000	N/A	0.3232	N/A	0.7677	0.2743	BEST
3	0.5024	0.5366	0.4220	N/A	0.0000	N/A	0.4047	N/A	0.8627	0.4205	BEST
4	0.4523	0.4917	0.4476	N/A	0.0000	N/A	0.4207	N/A	0.8840	0.4858	BEST
5	0.3862	0.5130	0.4424	N/A	0.0000	N/A	0.4468	N/A	0.9127	0.4104	
6	0.3500	0.4255	0.4614	N/A	0.0000	N/A	0.4648	N/A	0.9151	0.4657	BEST
7	0.3218	0.4652	0.5007	N/A	0.0000	N/A	0.4662	N/A	0.9292	0.6074	BEST
8	0.3476	0.4216	0.4718	N/A	0.0000	N/A	0.4631	N/A	0.9153	0.5089	
9	0.2888	0.3854	0.4946	N/A	0.0000	N/A	0.4627	N/A	0.9138	0.6018	
10	0.2787	0.3660	0.4949	N/A	0.0000	N/A	0.4677	N/A	0.9172	0.5949	
11	0.2863	0.3851	0.4861	N/A	0.0000	N/A	0.4629	N/A	0.9085	0.5730	
12	0.2362	0.3675	0.5018	N/A	0.0000	N/A	0.4676	N/A	0.9220	0.6175	BEST
13	0.3086	0.3580	0.4718	N/A	0.0000	N/A	0.4765	N/A	0.9223	0.4885	
14	0.2562	0.3480	0.4812	N/A	0.0000	N/A	0.4937	N/A	0.9240	0.5072	

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
15	0.2056	0.3479	0.5083	N/A	0.0000	N/A	0.4619	N/A	0.9219	0.6493	BEST
16	0.2566	0.3211	0.5127	N/A	0.0000	N/A	0.4602	N/A	0.9359	0.6546	BEST
17	0.2245	0.3512	0.5092	N/A	0.0000	N/A	0.4667	N/A	0.9393	0.6308	
18	0.2188	0.3236	0.5110	N/A	0.0000	N/A	0.4706	N/A	0.9158	0.6578	
19	0.2584	0.3131	0.5099	N/A	0.0000	N/A	0.4629	N/A	0.9480	0.6286	
20	0.2443	0.3436	0.4733	N/A	0.0000	N/A	0.4783	N/A	0.9276	0.4873	
21	0.2008	0.2941	0.5360	N/A	0.0000	N/A	0.4352	N/A	0.9501	0.7585	BEST
22	0.2352	0.3538	0.4810	N/A	0.0000	N/A	0.4727	N/A	0.9316	0.5198	
23	0.2149	0.2972	0.5268	N/A	0.0000	N/A	0.4459	N/A	0.9468	0.7146	
24	0.2073	0.3492	0.5027	N/A	0.0000	N/A	0.4348	N/A	0.9359	0.6403	
25	0.1994	0.3040	0.4779	N/A	0.0000	N/A	0.4658	N/A	0.9290	0.5168	
26	0.1956	0.2522	0.4954	N/A	0.0000	N/A	0.4455	N/A	0.9325	0.6034	
27	0.1816	0.2507	0.5075	N/A	0.0000	N/A	0.4551	N/A	0.9318	0.6430	
28	0.2000	0.2887	0.4793	N/A	0.0000	N/A	0.4660	N/A	0.9299	0.5214	
29	0.1832	0.2479	0.5293	N/A	0.0000	N/A	0.4425	N/A	0.9463	0.7282	
30	0.1796	0.2524	0.5230	N/A	0.0000	N/A	0.4536	N/A	0.9510	0.6875	
31	0.1822	0.2112	0.5189	N/A	0.0000	N/A	0.4360	N/A	0.9458	0.6938	
32	0.2100	0.2167	0.5203	N/A	0.0000	N/A	0.4528	N/A	0.9457	0.6828	
33	0.2132	0.2175	0.5313	N/A	0.0000	N/A	0.4519	N/A	0.9543	0.7191	
34	0.1574	0.2208	0.5204	N/A	0.0000	N/A	0.4464	N/A	0.9496	0.6858	
35	0.1820	0.2365	0.5109	N/A	0.0000	N/A	0.4614	N/A	0.9498	0.6325	
36	0.1591	0.2191	0.5396	N/A	0.0000	N/A	0.4286	N/A	0.9518	0.7779	BEST
37	0.1513	0.2213	0.5546	N/A	0.0000	N/A	0.4271	N/A	0.9565	0.8349	BEST
38	0.1727	0.2172	0.5351	N/A	0.0000	N/A	0.4305	N/A	0.9514	0.7586	
39	0.1824	0.2242	0.5382	N/A	0.0000	N/A	0.4298	N/A	0.9502	0.7726	
40	0.1411	0.1917	0.5530	N/A	0.0000	N/A	0.4286	N/A	0.9563	0.8272	
41	0.1542	0.2391	0.5315	N/A	0.0000	N/A	0.4583	N/A	0.9545	0.7131	
42	0.1673	0.2464	0.5208	N/A	0.0000	N/A	0.4553	N/A	0.9523	0.6757	
43	0.1735	0.1788	0.5499	N/A	0.0000	N/A	0.4281	N/A	0.9577	0.8138	
44	0.1440	0.2337	0.5258	N/A	0.0000	N/A	0.4412	N/A	0.9550	0.7068	
45	0.1755	0.2053	0.5413	N/A	0.0000	N/A	0.4388	N/A	0.9592	0.7670	
46	0.1935	0.2156	0.5419	N/A	0.0000	N/A	0.4432	N/A	0.9542	0.7702	
47	0.1945	0.2143	0.5251	N/A	0.0000	N/A	0.4319	N/A	0.9493	0.7192	
48	0.1722	0.2004	0.5426	N/A	0.0000	N/A	0.4289	N/A	0.9568	0.7848	
49	0.1587	0.2189	0.5445	N/A	0.0000	N/A	0.4297	N/A	0.9554	0.7930	
50	0.1584	0.2039	0.5532	N/A	0.0000	N/A	0.4342	N/A	0.9572	0.8214	
51	0.1609	0.1994	0.5316	N/A	0.0000	N/A	0.4361	N/A	0.9508	0.7393	
52	0.1465	0.2083	0.5311	N/A	0.0000	N/A	0.4312	N/A	0.9516	0.7417	
53	0.1839	0.1861	0.5469	N/A	0.0000	N/A	0.4266	N/A	0.9553	0.8057	
54	0.1567	0.1972	0.5489	N/A	0.0000	N/A	0.4295	N/A	0.9565	0.8095	
55	0.1595	0.1881	0.5528	N/A	0.0000	N/A	0.4294	N/A	0.9589	0.8229	
56	0.1640	0.2053	0.5320	N/A	0.0000	N/A	0.4383	N/A	0.9550	0.7346	
57	0.1730	0.1884	0.5467	N/A	0.0000	N/A	0.4284	N/A	0.9591	0.7993	
58	0.1797	0.2050	0.5366	N/A	0.0000	N/A	0.4317	N/A	0.9529	0.7620	
59	0.1489	0.1883	0.5550	N/A	0.0000	N/A	0.4232	N/A	0.9581	0.8385	BEST
60	0.1678	0.2090	0.5368	N/A	0.0000	N/A	0.4342	N/A	0.9570	0.7560	
61	0.1405	0.1979	0.5483	N/A	0.0000	N/A	0.4312	N/A	0.9590	0.8029	
62	0.1420	0.2137	0.5398	N/A	0.0000	N/A	0.4410	N/A	0.9555	0.7629	
63	0.1909	0.2073	0.5483	N/A	0.0000	N/A	0.4322	N/A	0.9614	0.7996	
64	0.1421	0.2020	0.5626	N/A	0.0000	N/A	0.4257	N/A	0.9619	0.8626	BEST
65	0.1643	0.1885	0.5631	N/A	0.0000	N/A	0.4242	N/A	0.9612	0.8672	BEST
66	0.1655	0.1787	0.5493	N/A	0.0000	N/A	0.4323	N/A	0.9590	0.8060	
67	0.1505	0.1745	0.5510	N/A	0.0000	N/A	0.4287	N/A	0.9587	0.8165	
68	0.1593	0.1848	0.5577	N/A	0.0000	N/A	0.4270	N/A	0.9607	0.8429	

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
69	0.1484	0.1924	0.5549	N/A	0.0000	N/A	0.4267	N/A	0.9602	0.8327	
70	0.1812	0.1687	0.5539	N/A	0.0000	N/A	0.4318	N/A	0.9607	0.8232	
71	0.1817	0.1844	0.5580	N/A	0.0000	N/A	0.4292	N/A	0.9602	0.8427	
72	0.1484	0.1712	0.5535	N/A	0.0000	N/A	0.4234	N/A	0.9578	0.8326	
73	0.1448	0.1603	0.5565	N/A	0.0000	N/A	0.4299	N/A	0.9620	0.8340	
74	0.1749	0.1740	0.5542	N/A	0.0000	N/A	0.4282	N/A	0.9591	0.8296	
75	0.1631	0.1849	0.5540	N/A	0.0000	N/A	0.4310	N/A	0.9606	0.8243	
76	0.1509	0.1853	0.5486	N/A	0.0000	N/A	0.4345	N/A	0.9597	0.8001	
77	0.1614	0.1643	0.5588	N/A	0.0000	N/A	0.4286	N/A	0.9622	0.8443	
78	0.1459	0.1678	0.5625	N/A	0.0000	N/A	0.4276	N/A	0.9616	0.8607	
79	0.1735	0.1716	0.5412	N/A	0.0000	N/A	0.4373	N/A	0.9596	0.7678	
80	0.1490	0.1815	0.5492	N/A	0.0000	N/A	0.4295	N/A	0.9583	0.8089	
81	0.1473	0.1951	0.5593	N/A	0.0000	N/A	0.4315	N/A	0.9615	0.8441	
82	0.1588	0.1960	0.5528	N/A	0.0000	N/A	0.4334	N/A	0.9595	0.8183	
83	0.1533	0.1899	0.5545	N/A	0.0000	N/A	0.4333	N/A	0.9624	0.8225	
84	0.1635	0.1899	0.5515	N/A	0.0000	N/A	0.4351	N/A	0.9613	0.8098	
85	0.1774	0.1803	0.5605	N/A	0.0000	N/A	0.4249	N/A	0.9610	0.8561	
86	0.1385	0.1792	0.5591	N/A	0.0000	N/A	0.4289	N/A	0.9630	0.8445	
87	0.1681	0.1806	0.5469	N/A	0.0000	N/A	0.4376	N/A	0.9615	0.7885	
88	0.1155	0.1775	0.5600	N/A	0.0000	N/A	0.4276	N/A	0.9611	0.8512	
89	0.1450	0.1845	0.5547	N/A	0.0000	N/A	0.4273	N/A	0.9596	0.8316	
90	0.1480	0.1883	0.5714	N/A	0.0000	N/A	0.4228	N/A	0.9637	0.8991	BEST
91	0.1496	0.1766	0.5584	N/A	0.0000	N/A	0.4269	N/A	0.9624	0.8444	
92	0.1636	0.1671	0.5660	N/A	0.0000	N/A	0.4241	N/A	0.9629	0.8771	
93	0.1496	0.1657	0.5675	N/A	0.0000	N/A	0.4239	N/A	0.9632	0.8828	
94	0.1443	0.1755	0.5617	N/A	0.0000	N/A	0.4260	N/A	0.9618	0.8591	
95	0.1839	0.1724	0.5682	N/A	0.0000	N/A	0.4237	N/A	0.9631	0.8861	
96	0.1983	0.1853	0.5460	N/A	0.0000	N/A	0.4352	N/A	0.9565	0.7924	
97	0.1406	0.1743	0.5574	N/A	0.0000	N/A	0.4302	N/A	0.9608	0.8386	
98	0.1370	0.1548	0.5653	N/A	0.0000	N/A	0.4262	N/A	0.9638	0.8714	
99	0.1367	0.1607	0.5558	N/A	0.0000	N/A	0.4314	N/A	0.9616	0.8302	
100	0.1459	0.1615	0.5606	N/A	0.0000	N/A	0.4258	N/A	0.9620	0.8547	
101	0.1425	0.1519	0.5652	N/A	0.0000	N/A	0.4239	N/A	0.9621	0.8746	
102	0.1266	0.1595	0.5611	N/A	0.0000	N/A	0.4271	N/A	0.9618	0.8556	
103	0.1281	0.1474	0.5705	N/A	0.0000	N/A	0.4220	N/A	0.9635	0.8966	
104	0.1109	0.1358	0.5699	N/A	0.0000	N/A	0.4226	N/A	0.9635	0.8937	
105	0.1630	0.1400	0.5586	N/A	0.0000	N/A	0.4243	N/A	0.9617	0.8486	
106	0.1195	0.1406	0.5690	N/A	0.0000	N/A	0.4212	N/A	0.9624	0.8925	
107	0.1567	0.1463	0.5626	N/A	0.0000	N/A	0.4266	N/A	0.9611	0.8629	
108	0.1624	0.1617	0.5614	N/A	0.0000	N/A	0.4279	N/A	0.9608	0.8570	
109	0.1464	0.1601	0.5559	N/A	0.0000	N/A	0.4280	N/A	0.9616	0.8339	
110	0.1372	0.1712	0.5611	N/A	0.0000	N/A	0.4240	N/A	0.9612	0.8591	
111	0.1585	0.1774	0.5603	N/A	0.0000	N/A	0.4284	N/A	0.9622	0.8506	
112	0.1416	0.1716	0.5661	N/A	0.0000	N/A	0.4264	N/A	0.9618	0.8761	
113	0.1636	0.1667	0.5624	N/A	0.0000	N/A	0.4257	N/A	0.9617	0.8620	
114	0.1306	0.1754	0.5582	N/A	0.0000	N/A	0.4310	N/A	0.9611	0.8409	
115	0.1466	0.1652	0.5622	N/A	0.0000	N/A	0.4275	N/A	0.9628	0.8586	
116	0.1463	0.1754	0.5679	N/A	0.0000	N/A	0.4248	N/A	0.9630	0.8838	
117	0.1539	0.1613	0.5641	N/A	0.0000	N/A	0.4273	N/A	0.9637	0.8652	
118	0.1423	0.1701	0.5602	N/A	0.0000	N/A	0.4286	N/A	0.9626	0.8494	
119	0.1509	0.1723	0.5656	N/A	0.0000	N/A	0.4245	N/A	0.9619	0.8758	
120	0.1429	0.1706	0.5719	N/A	0.0000	N/A	0.4241	N/A	0.9643	0.8992	BEST
121	0.1555	0.1645	0.5584	N/A	0.0000	N/A	0.4268	N/A	0.9624	0.8446	
122	0.1352	0.1729	0.5569	N/A	0.0000	N/A	0.4270	N/A	0.9607	0.8401	

Ep	TrL	VaL	mIoU	IoU ₀₂	IoU ₀₃	IoU ₀₄	IoU ₀₅	IoU ₀₆	IoU ₁₁	IoU ₁₈	Note
123	0.1495	0.1640	0.5577	N/A	0.0000	N/A	0.4255	N/A	0.9611	0.8443	
124	0.1505	0.1686	0.5722	N/A	0.0000	N/A	0.4214	N/A	0.9632	0.9044	BEST
125	0.1444	0.1695	0.5729	N/A	0.0000	N/A	0.4199	N/A	0.9628	0.9087	BEST
126	0.1333	0.1507	0.5701	N/A	0.0000	N/A	0.4231	N/A	0.9636	0.8938	
127	0.1444	0.1481	0.5677	N/A	0.0000	N/A	0.4227	N/A	0.9628	0.8851	
128	0.1148	0.1513	0.5643	N/A	0.0000	N/A	0.4245	N/A	0.9612	0.8715	
129	0.1530	0.1577	0.5615	N/A	0.0000	N/A	0.4239	N/A	0.9604	0.8618	
130	0.1210	0.1452	0.5652	N/A	0.0000	N/A	0.4243	N/A	0.9629	0.8734	
131	0.1179	0.1457	0.5680	N/A	0.0000	N/A	0.4243	N/A	0.9632	0.8844	
132	0.1432	0.1591	0.5707	N/A	0.0000	N/A	0.4245	N/A	0.9644	0.8939	
133	0.1444	0.1585	0.5557	N/A	0.0000	N/A	0.4243	N/A	0.9601	0.8385	
134	0.2013	0.1586	0.5614	N/A	0.0000	N/A	0.4249	N/A	0.9627	0.8580	
135	0.1412	0.1732	0.5697	N/A	0.0000	N/A	0.4214	N/A	0.9625	0.8948	
136	0.1261	0.1760	0.5721	N/A	0.0000	N/A	0.4214	N/A	0.9638	0.9032	
137	0.1385	0.1649	0.5698	N/A	0.0000	N/A	0.4220	N/A	0.9641	0.8933	
138	0.1334	0.1560	0.5660	N/A	0.0000	N/A	0.4245	N/A	0.9630	0.8764	
139	0.1231	0.1559	0.5719	N/A	0.0000	N/A	0.4217	N/A	0.9640	0.9017	
140	0.1602	0.1504	0.5680	N/A	0.0000	N/A	0.4206	N/A	0.9628	0.8885	
141	0.1397	0.1701	0.5677	N/A	0.0000	N/A	0.4243	N/A	0.9628	0.8837	
142	0.1463	0.1616	0.5691	N/A	0.0000	N/A	0.4220	N/A	0.9635	0.8910	
143	0.1411	0.1508	0.5697	N/A	0.0000	N/A	0.4225	N/A	0.9636	0.8925	
144	0.1354	0.1520	0.5598	N/A	0.0000	N/A	0.4225	N/A	0.9608	0.8561	
145	0.1127	0.1846	0.5761	N/A	0.0000	N/A	0.4197	N/A	0.9633	0.9213	BEST
146	0.1361	0.1598	0.5733	N/A	0.0000	N/A	0.4221	N/A	0.9639	0.9071	
147	0.1264	0.1682	0.5719	N/A	0.0000	N/A	0.4214	N/A	0.9639	0.9024	
148	0.1600	0.1516	0.5650	N/A	0.0000	N/A	0.4240	N/A	0.9629	0.8732	
149	0.1725	0.1655	0.5693	N/A	0.0000	N/A	0.4213	N/A	0.9625	0.8934	
150	0.1210	0.1415	0.5704	N/A	0.0000	N/A	0.4218	N/A	0.9637	0.8960	

DEPARTMENT OF MATHEMATICAL SCIENCES
CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden
www.chalmers.se



CHALMERS
UNIVERSITY OF TECHNOLOGY